```
  1: ===== FILE: init_project.m =====
  2: function init_project()
  3: % ==============================
  4: % 文件名：init_project.m
  5: % 路径：S-Function_14/init_project.m
  6: % 版本号：V1.0
  7: % 最后修改时间：2025-12-10
  8: % 作者：Auto-generated
  9: % 功能描述：
 10: %   初始化 LPV-MPC 项目运行环境：
 11: %     1) 调用 project_root() 获取根目录
 12: %     2) 将 src 及其子目录递归加入 MATLAB 路径
 13: %     3) 将 simulink 目录加入路径，便于加载模型
 14: % ==============================
 15:
 16: root = project_root();
 17: addpath(root);
 18: addpath(genpath(fullfile(root, 'src')));
 19: addpath(fullfile(root, 'simulink'));
 20:
 21: fprintf('[init_project] Root: %s\n', root);
 22: end
 23:
 24: ===== FILE: project_root.m =====
 25: function root = project_root()
 26: % ==============================
 27: % 文件名：project_root.m
 28: % 路径：S-Function_14/project_root.m
 29: % 版本号：V1.0
 30: % 最后修改时间：2025-12-10
 31: % 作者：Auto-generated
 32: % 功能描述：
 33: %   返回项目根目录的绝对路径，供所有脚本构建相对路径。
 34: %   该函数应位于项目根目录，不随其他脚本迁移。
 35: % ==============================
 36:
 37: persistent cached_root
 38: if isempty(cached_root) || ~isfolder(cached_root)
 39:     [cached_root, ~] = fileparts(mfilename('fullpath'));
 40: end
 41: root = cached_root;
 42: end
 43:
 44: ===== FILE: results_dir.m =====
 45: function dir_out = results_dir(subdir)
 46: % ==============================
 47: % 文件名：results_dir.m
 48: % 路径：S-Function_14/results_dir.m
 49: % 版本号：V1.0
 50: % 最后修改时间：2025-12-10
 51: % 作者：Auto-generated
 52: % 功能描述：
 53: %   构建结果输出目录的绝对路径，并在不存在时自动创建。
 54: %   示例：out_dir = results_dir('gru/performance_offline');
 55: % ==============================
 56:
 57: if nargin < 1 || isempty(subdir)
 58:     subdir = '';
 59: end
 60:
```

```
root = project_root();
dir_out = fullfile(root, 'results', subdir);

if ~exist(dir_out, 'dir')
    mkdir(dir_out);
end
end

===== FILE: src/ModernTCN/modern_tcn_model.py =====
"""ONNX 友好的 ModernTCN-small 多任务模型。

第一版不直接照搬官方 ModernTCN 的全部组件，而是保留其核心思想：
大核 depthwise temporal convolution + pointwise channel mixing + residual。
这样导出的 ONNX 主要由 Conv1d、BatchNorm、ReLU、Linear、Mean、Concat、
Reshape/Transpose 等标准算子构成，便于后续导入 MATLAB R2024b。
"""

from __future__ import annotations

from dataclasses import dataclass, asdict
from typing import Dict, Tuple

import torch
from torch import nn


@dataclass
class ModernTCNConfig:
    """ModernTCN-small 的默认训练与结构配置。"""

    input_dim: int = 19
    seq_len: int = 128
    channels: int = 64
    blocks: int = 5
    kernel_size: int = 31
    temporal_padding: str = "same"
    dropout: float = 0.15
    expansion: int = 2
    readout_input_stats: bool = True
    turn_head_source: str = "full"
    turn_feature_indices: Tuple[int, ...] = (1, 4, 5, 6, 7, 9, 10, 11, 16)
    lambda_turn: float = 0.05
    lambda_theta: float = 0.35
    lambda_theta_flat: float = 0.20
    theta_flat_loss_mode: str = "near_zero"
    theta_flat_zero_tol_deg: float = 0.3
    lambda_theta_near_flat: float = 0.0
    theta_near_flat_deg: float = 0.5
    lambda_theta_error_excess: float = 0.0
    lambda_theta_flat_excess: float = 0.0
    lambda_theta_near_flat_excess: float = 0.0
    lambda_theta_true_zero_excess: float = 0.0
    lambda_theta_active_excess: float = 0.0
    lambda_theta_small_neg: float = 0.0
    lambda_theta_small_neg_excess: float = 0.0
    theta_excess_target_deg: float = 1.0
    theta_flat_excess_target_deg: float = 0.5
    theta_true_zero_tol_deg: float = 1e-4
    theta_small_neg_min_deg: float = -4.0
    theta_small_neg_max_deg: float = -2.0
```

```
121:     theta_gate_mode: str = "none"
122:     theta_gate_power: float = 1.0
123:     theta_gate_floor: float = 0.0
124:     main_class_multipliers: Tuple[float, float, float] = (1.20, 1.00, 0.95)
125:     turn_class_multipliers: Tuple[float, float, float] = (1.00, 1.10, 1.00)
126:     main_class_weight_method: str = "sqrt_inverse"
127:     turn_class_weight_method: str = "sqrt_inverse"
128:     main_neg_slope_weight: float = 2.0
129:     main_pos_slope_weight: float = 1.0
130:     theta_neg_weight: float = 2.0
131:     theta_pos_weight: float = 1.0
132:     turn_transition_weight: float = 1.0
133:     select_turn_weight: float = 0.30
134:     select_turn_transition_weight: float = 1.00
135:     select_turn_transition_target: float = 0.75
136:     select_turn_left_weight: float = 0.00
137:     select_turn_left_target: float = 0.80
138:     select_turn_lr_weight: float = 0.00
139:     select_turn_lr_target: float = 0.80
140:     select_theta_weight: float = 0.15
141:     select_theta_ref_deg: float = 5.0
142:     select_theta_p95_weight: float = 0.0
143:     select_theta_p95_target_deg: float = 1.0
144:     select_theta_flat_p95_weight: float = 0.0
145:     select_theta_flat_p95_target_deg: float = 1.0
146:     select_theta_near_flat_p95_weight: float = 0.0
147:     select_theta_near_flat_p95_target_deg: float = 1.0
148:     select_theta_true_zero_p95_weight: float = 0.0
149:     select_theta_true_zero_p95_target_deg: float = 1.0
150:     select_theta_flat_peak_weight: float = 0.0
151:     select_theta_flat_peak_target_deg: float = 3.0
152:     select_theta_small_neg_p95_weight: float = 0.0
153:     select_theta_small_neg_p95_target_deg: float = 1.0
154:     select_theta_extreme_p95_weight: float = 0.0
155:     select_theta_extreme_p95_target_deg: float = 1.0
156:     select_theta_edge_p95_weight: float = 0.0
157:     select_theta_edge_p95_target_deg: float = 1.2
158:     select_theta_small_nonzero_p95_weight: float = 0.0
159:     select_theta_small_nonzero_p95_target_deg: float = 1.0
160:     select_theta_flat_bias_weight: float = 0.0
161:     select_theta_flat_bias_target_deg: float = 0.2
162: 
163:     def to_dict(self) -> Dict[str, object]:
164:         return asdict(self)
165: 
166: 
167: class CausalDepthwiseConv1d(nn.Module):
168:     """Depthwise causal Conv1d with ONNX-friendly Conv + Slice semantics."""
169: 
170:     def __init__(self, channels: int, kernel_size: int) -> None:
171:         super().__init__()
172:         if kernel_size < 1:
173:             raise ValueError("kernel_size 必须 >= 1。")
174:         self.trim = kernel_size - 1
175:         self.conv = nn.Conv1d(
176:             channels,
177:             channels,
178:             kernel_size,
179:             padding=self.trim,
180:             groups=channels,
```

```
 181:         )
 182:
 183:     def forward(self, x: torch.Tensor) -> torch.Tensor:
 184:         y = self.conv(x)
 185:         if self.trim == 0:
 186:             return y
 187:         return y[..., : x.size(-1)]
 188:
 189:
 190: class ModernTCNBlock(nn.Module):
 191:     """大核 depthwise conv + pointwise MLP 的残差块。"""
 192:
 193:     def __init__(
 194:         self,
 195:         channels: int,
 196:         kernel_size: int,
 197:         dropout: float,
 198:         expansion: int,
 199:         temporal_padding: str = "same",
 200:     ) -> None:
 201:         super().__init__()
 202:         hidden = int(channels * expansion)
 203:         temporal_padding = str(temporal_padding).lower()
 204:         if temporal_padding == "same":
 205:             if kernel_size % 2 != 1:
 206:                 raise ValueError("same padding 模式下 kernel_size 必须为奇数。")
 207:             padding = kernel_size // 2
 208:             self.depthwise = nn.Conv1d(channels, channels, kernel_size, padding=padding,
groups=channels)
 209:         elif temporal_padding == "causal":
 210:             self.depthwise = CausalDepthwiseConv1d(channels, kernel_size)
 211:         else:
 212:             raise ValueError(f"未知 temporal_padding: {temporal_padding}")
 213:         self.temporal_padding = temporal_padding
 214:         self.bn1 = nn.BatchNorm1d(channels)
 215:         self.pw1 = nn.Conv1d(channels, hidden, kernel_size=1)
 216:         self.pw2 = nn.Conv1d(hidden, channels, kernel_size=1)
 217:         self.bn2 = nn.BatchNorm1d(channels)
 218:         self.act = nn.ReLU(inplace=False)
 219:         self.drop = nn.Dropout(dropout)
 220:         # layer_scale 是常数形状参数，导出 ONNX 后只对应 Mul，MATLAB 侧更容易处理。
 221:         self.layer_scale = nn.Parameter(torch.ones(1, channels, 1) * 1e-2)
 222:
 223:     def forward(self, x: torch.Tensor) -> torch.Tensor:
 224:         residual = x
 225:         y = self.depthwise(x)
 226:         y = self.bn1(y)
 227:         y = self.act(y)
 228:         y = self.pw1(y)
 229:         y = self.act(y)
 230:         y = self.drop(y)
 231:         y = self.pw2(y)
 232:         y = self.bn2(y)
 233:         y = self.drop(y)
 234:         return residual + y * self.layer_scale
 235:
 236:
 237: class ModernTCNSmall(nn.Module):
 238:     """固定输入 `[batch, time=128, features=19]` 的三输出多任务网络。"""
 239:
 240:     def __init__(self, cfg: ModernTCNConfig) -> None:
```

```
241:         super().__init__()
242:         self.cfg = cfg
243:         self.stem = nn.Sequential(
244:             nn.Conv1d(cfg.input_dim, cfg.channels, kernel_size=1),
245:             nn.BatchNorm1d(cfg.channels),
246:             nn.ReLU(inplace=False),
247:         )
248:         self.blocks = nn.ModuleList(
249:             [
250:                 ModernTCNBlock(
251:                     cfg.channels,
252:                     cfg.kernel_size,
253:                     cfg.dropout,
254:                     cfg.expansion,
255:                     temporal_padding=cfg.temporal_padding,
256:                 )
257:                 for _ in range(cfg.blocks)
258:             ]
259:         )
260:         feature_dim = cfg.channels * 3
261:         if cfg.readout_input_stats:
262:             feature_dim += cfg.input_dim * 5
263:         input_stats_dim = cfg.input_dim * 5
264:         turn_source = cfg.turn_head_source.lower()
265:         if turn_source == "full":
266:             turn_dim = feature_dim
267:         elif turn_source == "inputstats":
268:             turn_dim = input_stats_dim
269:         elif turn_source == "kinematic_stats":
270:             turn_indices = self._make_turn_stats_indices(cfg.input_dim, cfg.turn_feature_indices)
271:             self.register_buffer("turn_stats_indices", turn_indices, persistent=False)
272:             turn_dim = int(turn_indices.numel())
273:         else:
274:             raise ValueError(f"未知 turn_head_source: {cfg.turn_head_source}")
275:         self.turn_head_source = turn_source
276:
277:         # 三个任务头严格对应 MATLAB 端后处理契约。
278:         self.main_head = nn.Linear(feature_dim, 3)
279:         self.turn_head = nn.Sequential(
280:             nn.Linear(turn_dim, 64),
281:             nn.ReLU(inplace=False),
282:             nn.Linear(64, 3),
283:         )
284:         self.theta_head = nn.Linear(feature_dim, 1)
285:
286:     def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
287:         # x: [B, T, F]；Conv1d 使用 [B, F, T]。
288:         x_ct = x.transpose(1, 2)
289:         z = self.stem(x_ct)
290:         for block in self.blocks:
291:             z = block(z)
292:
293:         h_last = z[:, :, -1]
294:         h_mean = z.mean(dim=2)
295:         h_max = z.amax(dim=2)
296:         pieces = [h_last, h_mean, h_max]
297:         input_stats = self._input_stats(x_ct)
298:         if self.cfg.readout_input_stats:
299:             pieces.append(input_stats)
300:         h = torch.cat(pieces, dim=1)
```

```
        logits_main = self.main_head(h)
        if self.turn_head_source == "full":
            h_turn = h
        elif self.turn_head_source == "inputstats":
            h_turn = input_stats
        else:
            h_turn = input_stats.index_select(1, self.turn_stats_indices)
        logits_turn = self.turn_head(h_turn)
        theta_hat = self.theta_head(h)
        theta_hat = self._apply_theta_gate(theta_hat, logits_main)
        return logits_main, logits_turn, theta_hat

    def _apply_theta_gate(self, theta_hat: torch.Tensor, logits_main: torch.Tensor) ->
torch.Tensor:
        """Optionally suppress theta when the main head is confident the window is flat/stall."""

        mode = str(getattr(self.cfg, "theta_gate_mode", "none")).lower()
        if mode in ("", "none"):
            return theta_hat
        if mode != "main_slope_prob":
            raise ValueError(f"未知 theta_gate_mode: {self.cfg.theta_gate_mode}")
        slope_prob = torch.softmax(logits_main, dim=1)[:, 2:3]
        floor = float(getattr(self.cfg, "theta_gate_floor", 0.0))
        power = float(getattr(self.cfg, "theta_gate_power", 1.0))
        gate = floor + (1.0 - floor) * torch.pow(torch.clamp(slope_prob, min=1e-6), power)
        return theta_hat * gate

    @staticmethod
    def _input_stats(x_ct: torch.Tensor) -> torch.Tensor:
        """复用窗口级统计线索，但仍只来自同一 19 维输入窗口。"""

        x_last = x_ct[:, :, -1]
        x_mean = x_ct.mean(dim=2)
        x_std = torch.sqrt(torch.mean((x_ct - x_mean.unsqueeze(2)) ** 2, dim=2) + 1e-8)
        x_max = x_ct.amax(dim=2)
        x_min = x_ct.amin(dim=2)
        return torch.cat([x_last, x_mean, x_std, x_max, x_min], dim=1)

    @staticmethod
    def _make_turn_stats_indices(input_dim: int, feature_indices: Tuple[int, ...]) -> torch.Tensor:
        """Return gather indices for [last, mean, std, max, min] input stats."""

        idx = []
        for stat_block in range(5):
            base = stat_block * input_dim
            idx.extend(base + int(i) for i in feature_indices)
        return torch.tensor(idx, dtype=torch.long)


def build_model_from_checkpoint_dict(ckpt: Dict[str, object]) -> ModernTCNSmall:
    """按 checkpoint 中的配置恢复模型结构。"""

    cfg_dict = dict(ckpt["model_config"])
    cfg_dict.setdefault("temporal_padding", "same")
    cfg_dict["main_class_multipliers"] = tuple(cfg_dict["main_class_multipliers"])
    cfg_dict["turn_class_multipliers"] = tuple(cfg_dict["turn_class_multipliers"])
    if "turn_feature_indices" in cfg_dict:
        cfg_dict["turn_feature_indices"] = tuple(cfg_dict["turn_feature_indices"])
    cfg = ModernTCNConfig(**cfg_dict)
    model = ModernTCNSmall(cfg)
    model.load_state_dict(ckpt["model_state"])
```

```
361:     return model
362:
363: ===== FILE: src/ModernTCN/modern_tcn_data.py =====
364: """读取固定 ModernTCN 数据集，并保持 MATLAB 侧已有 split/scaler 不变。
365:
366: 本文件默认读取 `data/tcn/ModernTCN_dataset_v4_industrial.mat` 中已经生成好的
367: 窗口、标签、样本权重和元信息。这里不能重新划分 train/val/test，也不能
368: 重新拟合 scaler；Python 端训练必须直接使用 MAT 文件里的归一化后 X。
369: """
370:
371: from __future__ import annotations
372:
373: from dataclasses import dataclass
374: from pathlib import Path
375: from typing import Dict, Iterable, List, Optional
376:
377: import h5py
378: import numpy as np
379: import torch
380: from torch.utils.data import Dataset
381:
382:
383: DATASET_RELATIVE_PATH = Path("data") / "tcn" / "ModernTCN_dataset_v4_industrial.mat"
384:
385:
386: @dataclass(frozen=True)
387: class ModernTCNContract:
388:     """记录第一阶段实验必须锁定的数据契约。"""
389:
390:     dataset_file: str
391:     seq_len: int
392:     input_dim: int
393:     output_contract: str
394:     split_policy: str
395:     scaler_policy: str
396:     vehicle_type: str = "diagonal_dual_steer_drive_agv"
397:     active_drive_steer_wheels: str = "LF,RR"
398:     passive_support_wheels: str = "RF,LR"
399:     feature_policy: str = "keep_current_algorithm_inputs_unchanged"
400:     label_time_policy: str = "current_window_end"
401:     horizon_steps: int = 0
402:     horizon_seconds: float = 0.0
403:     confidence_policy: str = "derive_classification_confidence_from_softmax_and_export"
404:
405:
406: @dataclass
407: class SplitArrays:
408:     """单个 split 的全部训练/评估数组。"""
409:
410:     X: np.ndarray
411:     y_main: np.ndarray
412:     y_turn: np.ndarray
413:     y_theta: np.ndarray
414:     mask_theta: np.ndarray
415:     main_weight: np.ndarray
416:     turn_weight: np.ndarray
417:     theta_weight: np.ndarray
418:     turn_purity: np.ndarray
419:     turn_transition: np.ndarray
420:     run_id: np.ndarray
```

```
421:
422:
423: class AGVWindowDataset(Dataset):
424:     """PyTorch Dataset，直接封装 MAT 文件中的窗口数据。"""
425:
426:     def __init__(self, split: SplitArrays) -> None:
427:         self.split = split
428:
429:     def __len__(self) -> int:
430:         return int(self.split.X.shape[0])
431:
432:     def __getitem__(self, idx: int) -> Dict[str, torch.Tensor]:
433:         # 这里保持输入格式为 [time, feature]，模型内部再转成 Conv1d 需要的 [B, C, T]。
434:         return {
435:             "X": torch.from_numpy(self.split.X[idx]).float(),
436:             "y_main": torch.tensor(self.split.y_main[idx], dtype=torch.long),
437:             "y_turn": torch.tensor(self.split.y_turn[idx], dtype=torch.long),
438:             "y_theta": torch.tensor(self.split.y_theta[idx], dtype=torch.float32),
439:             "mask_theta": torch.tensor(self.split.mask_theta[idx], dtype=torch.float32),
440:             "main_weight": torch.tensor(self.split.main_weight[idx], dtype=torch.float32),
441:             "turn_weight": torch.tensor(self.split.turn_weight[idx], dtype=torch.float32),
442:             "theta_weight": torch.tensor(self.split.theta_weight[idx], dtype=torch.float32),
443:             "turn_purity": torch.tensor(self.split.turn_purity[idx], dtype=torch.float32),
444:             "turn_transition": torch.tensor(self.split.turn_transition[idx], dtype=torch.bool),
445:             "run_id": torch.tensor(self.split.run_id[idx], dtype=torch.float32),
446:         }
447:
448:
449: def find_project_root(start: Optional[Path] = None) -> Path:
450:     """从当前文件向上寻找项目根目录。"""
451:
452:     if start is None:
453:         start = Path(__file__).resolve()
454:     start = start.resolve()
455:     candidates: Iterable[Path] = (start, *start.parents)
456:     for p in candidates:
457:         if (p / "init_project.m").exists() and (p / "data" / "tcn").exists():
458:             return p
459:     raise FileNotFoundError("无法定位项目根目录：未找到 init_project.m 和 data/tcn。")
460:
461:
462: def default_dataset_file(project_root: Optional[Path] = None) -> Path:
463:     """返回当前默认 ModernTCN 数据集路径。"""
464:
465:     root = project_root or find_project_root()
466:     return root / DATASET_RELATIVE_PATH
467:
468:
469: def load_modern_tcn_dataset(
470:     dataset_file: Optional[Path] = None,
471:     limit_train: int = 0,
472:     limit_val: int = 0,
473:     limit_test: int = 0,
474: ) -> Dict[str, object]:
475:     """读取固定 ModernTCN 数据集。
476:
477:     MATLAB v7.3 MAT 文件本质上是 HDF5；数值数组在 HDF5 中维度顺序和 MATLAB
478:     `size(dataset.X_train) == [N, 128, 19]` 相反，因此这里显式转回
479:     `[window, time, feature]`。
480:     """
```

```
 481:
 482:     dataset_file = Path(dataset_file) if dataset_file else default_dataset_file()
 483:     if not dataset_file.exists():
 484:         raise FileNotFoundError(f"找不到固定 ModernTCN 数据集：{dataset_file}")
 485:
 486:     with h5py.File(dataset_file, "r") as f:
 487:         root = f["dataset"]
 488:         train = _read_split(root, "train", limit_train)
 489:         val = _read_split(root, "val", limit_val)
 490:         test = _read_split(root, "test", limit_test)
 491:         feat_names = _read_feat_names(f, root)
 492:         scaler = _read_scaler(root)
 493:
 494:     contract = ModernTCNContract(
 495:         dataset_file=str(dataset_file),
 496:         seq_len=int(train.X.shape[1]),
 497:         input_dim=int(train.X.shape[2]),
 498:         output_contract="logits_main3_logits_turn3_theta1",
 499:         split_policy="use_existing_run_level_split_from_mat",
 500:         scaler_policy="use_existing_normalized_X_and_existing_scaler_only",
 501:     )
 502:
 503:     _check_contract(contract)
 504:     return {
 505:         "train": train,
 506:         "val": val,
 507:         "test": test,
 508:         "feat_names": feat_names,
 509:         "scaler": scaler,
 510:         "contract": contract,
 511:     }
 512:
 513:
 514: def class_weights(labels: np.ndarray, num_classes: int, method: str, multipliers: List[float]) ->
torch.Tensor:
 515:     """复刻 MATLAB baseline 中的类别权重计算方式。"""
 516:
 517:     labels = np.asarray(labels).reshape(-1)
 518:     counts = np.array([(labels == i).sum() for i in range(num_classes)], dtype=np.float64)
 519:     counts_safe = np.maximum(counts, 1.0)
 520:     method = method.lower()
 521:     if method == "inverse":
 522:         w = 1.0 / counts_safe
 523:     elif method == "sqrt_inverse":
 524:         w = 1.0 / np.sqrt(counts_safe)
 525:     elif method == "balanced":
 526:         w = counts.sum() / (num_classes * counts_safe)
 527:     else:
 528:         w = np.ones(num_classes, dtype=np.float64)
 529:     w[counts == 0] = 0.0
 530:     if np.any(w > 0):
 531:         w = w / np.mean(w[w > 0])
 532:     else:
 533:         w = np.ones(num_classes, dtype=np.float64)
 534:     w = w * np.asarray(multipliers, dtype=np.float64)
 535:     if np.any(w > 0):
 536:         w = w / np.mean(w[w > 0])
 537:     return torch.tensor(w, dtype=torch.float32)
 538:
 539:
 540: def _read_split(root: h5py.Group, split_name: str, limit: int) -> SplitArrays:
```

```
    X = _read_h5_array(root[f"X_{split_name}"])
    # MATLAB 主工况标签为 1/2/3；PyTorch CE 使用 0/1/2。
    y_main = _read_vector(root[f"y_main_{split_name}"]).astype(np.int64) - 1
    # MATLAB 转弯标签为 -1/0/1；PyTorch CE 使用 0/1/2。
    y_turn = _read_vector(root[f"y_turn_{split_name}"]).astype(np.int64) + 1
    y_theta = _read_vector(root[f"y_theta_{split_name}"]).astype(np.float32)
    mask_theta = _read_vector(root[f"mask_theta_{split_name}"]).astype(np.float32)
    main_weight = _read_vector(root[f"main_sample_weight_{split_name}"]).astype(np.float32)
    turn_weight = _read_vector(root[f"turn_sample_weight_{split_name}"]).astype(np.float32)
    theta_weight = _read_vector(root[f"theta_sample_weight_{split_name}"]).astype(np.float32)
    turn_purity = _read_vector(root[f"turn_purity_{split_name}"]).astype(np.float32)
    turn_transition = _read_vector(root[f"turn_transition_{split_name}"]).astype(bool)
    run_id = _read_vector(root[f"run_id_{split_name}"]).astype(np.float32)

    if limit and limit > 0:
        sl = slice(0, int(limit))
        X = X[sl]
        y_main = y_main[sl]
        y_turn = y_turn[sl]
        y_theta = y_theta[sl]
        mask_theta = mask_theta[sl]
        main_weight = main_weight[sl]
        turn_weight = turn_weight[sl]
        theta_weight = theta_weight[sl]
        turn_purity = turn_purity[sl]
        turn_transition = turn_transition[sl]
        run_id = run_id[sl]

    return SplitArrays(
        X=X,
        y_main=y_main,
        y_turn=y_turn,
        y_theta=y_theta,
        mask_theta=mask_theta,
        main_weight=main_weight,
        turn_weight=turn_weight,
        theta_weight=theta_weight,
        turn_purity=turn_purity,
        turn_transition=turn_transition,
        run_id=run_id,
    )


def _read_h5_array(ds: h5py.Dataset) -> np.ndarray:
    raw = np.asarray(ds, dtype=np.float32)
    if raw.ndim != 3:
        raise ValueError(f"期望 3D 窗口数组，实际 shape={raw.shape}")
    return np.transpose(raw, (2, 1, 0)).copy()


def _read_vector(ds: h5py.Dataset) -> np.ndarray:
    return np.asarray(ds).reshape(-1).copy()


def _read_scaler(root: h5py.Group) -> Dict[str, np.ndarray]:
    scaler_group = root["scaler"]
    return {
        "mean": _read_vector(scaler_group["mean"]).astype(np.float32),
        "std": _read_vector(scaler_group["std"]).astype(np.float32),
    }
```

```
601:
602:
603: def _read_feat_names(f: h5py.File, root: h5py.Group) -> List[str]:
604:     names: List[str] = []
605:     refs = np.asarray(root["feat_names"]).reshape(-1)
606:     for ref in refs:
607:         names.append(_decode_matlab_char(f[ref]))
608:     return names
609:
610:
611: def _decode_matlab_char(ds: h5py.Dataset) -> str:
612:     raw = np.asarray(ds).reshape(-1)
613:     return "".join(chr(int(c)) for c in raw if int(c) != 0)
614:
615:
616: def _check_contract(contract: ModernTCNContract) -> None:
617:     if contract.seq_len != 128:
618:         raise ValueError(f"ModernTCN 第一阶段要求 seq_len=128，实际为 {contract.seq_len}")
619:     if contract.input_dim != 19:
620:         raise ValueError(f"ModernTCN 第一阶段要求 input_dim=19，实际为 {contract.input_dim}")
621:     if contract.horizon_steps != 0:
622:         raise ValueError(f"ModernTCN 当前模型固定为当前状态估计 horizon_steps=0，实际为
{contract.horizon_steps}")
623:
624: ===== FILE: src/ModernTCN/modern_tcn_metrics.py =====
625: """ModernTCN 第一阶段指标、损失和门槛判定。
626:
627: 指标定义和 `run_TCN_GRU_transition_rich_v3_baseline.m` 对齐：输出
628: acc_main、acc_turn、acc_turn_transition、theta_mae_deg、flat/stall/slope
629: recall、uphill/downhill recall。seed42 门槛只用于决定是否进入三 seed，
630: 不用于训练集或验证集重划分。
631: """
632:
633: from __future__ import annotations
634:
635: from dataclasses import dataclass
636: from typing import Dict, Iterable, List, Tuple
637:
638: import numpy as np
639: import torch
640: import torch.nn.functional as F
641:
642:
643: @dataclass(frozen=True)
644: class GateThresholds:
645:     acc_main: float = 0.90
646:     flat_recall: float = 0.90
647:     slope_recall: float = 0.88
648:     acc_turn_transition: float = 0.75
649:     theta_mae_deg: float = 0.70
650:
651:
652: def multitask_loss(
653:     logits_main: torch.Tensor,
654:     logits_turn: torch.Tensor,
655:     theta_hat: torch.Tensor,
656:     batch: Dict[str, torch.Tensor],
657:     class_weights_main: torch.Tensor,
658:     class_weights_turn: torch.Tensor,
659:     cfg,
660: ) -> Tuple[torch.Tensor, Dict[str, float]]:
```

```
661:     """计算与 MATLAB TCN/GRU baseline 对齐的多任务损失。"""
662:
663:     y_main = batch["y_main"]
664:     y_turn = batch["y_turn"]
665:     theta = batch["y_theta"]
666:     mask_theta = batch["mask_theta"]
667:
668:     main_sample_weight = batch["main_weight"] * _main_slope_sample_weight(y_main, theta, cfg)
669:     turn_sample_weight = batch["turn_weight"] * (
670:         1.0 + (cfg.turn_transition_weight - 1.0) * batch["turn_transition"].float()
671:     )
672:     theta_sample_weight = batch["theta_weight"] * _theta_sign_weight(theta, cfg)
673:
674:     loss_main = _weighted_ce(logits_main, y_main, class_weights_main, main_sample_weight)
675:     loss_turn = _weighted_ce(logits_turn, y_turn, class_weights_turn, turn_sample_weight)
676:     theta_hat = theta_hat.reshape(-1)
677:     loss_theta = _masked_weighted_mse(theta_hat, theta, mask_theta, theta_sample_weight)
678:     near_flat_limit = torch.deg2rad(torch.tensor(float(getattr(cfg, "theta_near_flat_deg", 0.5)),
device=theta.device))
679:     near_flat_mask = (torch.abs(theta) <= near_flat_limit).float() * mask_theta
680:     loss_theta_near_flat = _masked_weighted_mse(
681:         theta_hat, torch.zeros_like(theta), near_flat_mask, theta_sample_weight
682:     )
683:     true_zero_limit = torch.deg2rad(
684:         torch.tensor(float(getattr(cfg, "theta_true_zero_tol_deg", 1e-4)), device=theta.device)
685:     )
686:     true_zero_mask = (torch.abs(theta) <= true_zero_limit).float() * mask_theta
687:     flat_zero_mask = _theta_flat_zero_mask(theta, y_main, mask_theta, near_flat_mask,
true_zero_mask, cfg)
688:     loss_theta_flat = _masked_weighted_mse(theta_hat, torch.zeros_like(theta), flat_zero_mask,
theta_sample_weight)
689:     active_mask = (torch.abs(theta) >= torch.deg2rad(torch.tensor(2.0,
device=theta.device))).float() * mask_theta
690:     small_neg_min = torch.deg2rad(
691:         torch.tensor(float(getattr(cfg, "theta_small_neg_min_deg", -4.0)), device=theta.device)
692:     )
693:     small_neg_max = torch.deg2rad(
694:         torch.tensor(float(getattr(cfg, "theta_small_neg_max_deg", -2.0)), device=theta.device)
695:     )
696:     small_neg_mask = ((theta >= small_neg_min) & (theta <= small_neg_max)).float() * mask_theta
697:     loss_theta_error_excess = _masked_weighted_abs_excess_mse(
698:         theta_hat, theta, mask_theta, theta_sample_weight, float(getattr(cfg,
"theta_excess_target_deg", 1.0))
699:     )
700:     loss_theta_flat_excess = _masked_weighted_abs_excess_mse(
701:         theta_hat,
702:         torch.zeros_like(theta),
703:         flat_zero_mask,
704:         theta_sample_weight,
705:         float(getattr(cfg, "theta_flat_excess_target_deg", 0.5)),
706:     )
707:     loss_theta_near_flat_excess = _masked_weighted_abs_excess_mse(
708:         theta_hat,
709:         torch.zeros_like(theta),
710:         near_flat_mask,
711:         theta_sample_weight,
712:         float(getattr(cfg, "theta_flat_excess_target_deg", 0.5)),
713:     )
714:     loss_theta_true_zero_excess = _masked_weighted_abs_excess_mse(
715:         theta_hat,
716:         torch.zeros_like(theta),
717:         true_zero_mask,
718:         theta_sample_weight,
719:         float(getattr(cfg, "theta_flat_excess_target_deg", 0.5)),
720:     )
```

```
 721:     loss_theta_active_excess = _masked_weighted_abs_excess_mse(
 722:         theta_hat, theta, active_mask, theta_sample_weight, float(getattr(cfg,
"theta_excess_target_deg", 1.0))
 723:     )
 724:     loss_theta_small_neg = _masked_weighted_mse(theta_hat, theta, small_neg_mask,
theta_sample_weight)
 725:     loss_theta_small_neg_excess = _masked_weighted_abs_excess_mse(
 726:         theta_hat, theta, small_neg_mask, theta_sample_weight, float(getattr(cfg,
"theta_excess_target_deg", 1.0))
 727:     )
 728:
 729:     total = (
 730:         loss_main
 731:         + cfg.lambda_turn * loss_turn
 732:         + cfg.lambda_theta * loss_theta
 733:         + cfg.lambda_theta_flat * loss_theta_flat
 734:         + float(getattr(cfg, "lambda_theta_near_flat", 0.0)) * loss_theta_near_flat
 735:         + float(getattr(cfg, "lambda_theta_error_excess", 0.0)) * loss_theta_error_excess
 736:         + float(getattr(cfg, "lambda_theta_flat_excess", 0.0)) * loss_theta_flat_excess
 737:         + float(getattr(cfg, "lambda_theta_near_flat_excess", 0.0)) * loss_theta_near_flat_excess
 738:         + float(getattr(cfg, "lambda_theta_true_zero_excess", 0.0)) * loss_theta_true_zero_excess
 739:         + float(getattr(cfg, "lambda_theta_active_excess", 0.0)) * loss_theta_active_excess
 740:         + float(getattr(cfg, "lambda_theta_small_neg", 0.0)) * loss_theta_small_neg
 741:         + float(getattr(cfg, "lambda_theta_small_neg_excess", 0.0)) * loss_theta_small_neg_excess
 742:     )
 743:     parts = {
 744:         "loss_total": float(total.detach().cpu()),
 745:         "loss_main": float(loss_main.detach().cpu()),
 746:         "loss_turn": float(loss_turn.detach().cpu()),
 747:         "loss_theta": float(loss_theta.detach().cpu()),
 748:         "loss_theta_flat": float(loss_theta_flat.detach().cpu()),
 749:         "loss_theta_near_flat": float(loss_theta_near_flat.detach().cpu()),
 750:         "loss_theta_error_excess": float(loss_theta_error_excess.detach().cpu()),
 751:         "loss_theta_flat_excess": float(loss_theta_flat_excess.detach().cpu()),
 752:         "loss_theta_near_flat_excess": float(loss_theta_near_flat_excess.detach().cpu()),
 753:         "loss_theta_true_zero_excess": float(loss_theta_true_zero_excess.detach().cpu()),
 754:         "loss_theta_active_excess": float(loss_theta_active_excess.detach().cpu()),
 755:         "loss_theta_small_neg": float(loss_theta_small_neg.detach().cpu()),
 756:         "loss_theta_small_neg_excess": float(loss_theta_small_neg_excess.detach().cpu()),
 757:     }
 758:     return total, parts
 759:
 760:
 761: def _theta_flat_zero_mask(
 762:     theta: torch.Tensor,
 763:     y_main: torch.Tensor,
 764:     mask_theta: torch.Tensor,
 765:     near_flat_mask: torch.Tensor,
 766:     true_zero_mask: torch.Tensor,
 767:     cfg,
 768: ) -> torch.Tensor:
 769:     """Return the samples where the auxiliary flat-theta loss may force zero.
 770:
 771:     ``main_flat`` is kept for legacy reproduction. New balanced training uses
 772:     ``near_zero`` so |theta|<2 deg can still be classified as flat without
 773:     teaching the regression head to collapse those angles to zero.
 774:     """
 775:
 776:     mode = str(getattr(cfg, "theta_flat_loss_mode", "near_zero")).lower()
 777:     if mode in ("", "none", "off"):
 778:         return torch.zeros_like(mask_theta)
 779:     if mode in ("main_flat", "flat", "legacy"):
 780:         return (y_main == 0).float() * mask_theta
```

```
    if mode in ("near_flat", "theta_near_flat"):
        return near_flat_mask
    if mode in ("true_zero", "zero"):
        return true_zero_mask
    if mode in ("near_zero", "very_near_zero"):
        tol = torch.deg2rad(
            torch.tensor(float(getattr(cfg, "theta_flat_zero_tol_deg", 0.3)), device=theta.device)
        )
        return (torch.abs(theta) <= tol).float() * mask_theta
    raise ValueError(f"Unknown theta_flat_loss_mode: {mode}")


def compute_metrics(
    logits_main: np.ndarray,
    logits_turn: np.ndarray,
    theta_hat: np.ndarray,
    split,
    loss_total: float = float("nan"),
) -> Dict[str, object]:
    """根据完整 split 的预测结果计算评估指标。"""

    prob_main = _softmax_np(np.asarray(logits_main))
    prob_turn = _softmax_np(np.asarray(logits_turn))
    pred_main = prob_main.argmax(axis=1)
    pred_turn_cls = prob_turn.argmax(axis=1)
    pred_turn_raw = pred_turn_cls - 1
    y_main = split.y_main.reshape(-1)
    y_turn_raw = split.y_turn.reshape(-1) - 1
    theta_true = split.y_theta.reshape(-1)
    theta_hat = np.asarray(theta_hat).reshape(-1)
    main_confidence = prob_main.max(axis=1)
    turn_confidence = prob_turn.max(axis=1)

    cm_main = _confusion_matrix(y_main, pred_main, 3)
    cm_turn = _confusion_matrix(y_turn_raw + 1, pred_turn_raw + 1, 3)
    recall_main = np.diag(cm_main) / np.maximum(cm_main.sum(axis=1), 1)
    recall_turn = np.diag(cm_turn) / np.maximum(cm_turn.sum(axis=1), 1)
    transition_mask = split.turn_transition.astype(bool)
    pure_mask = np.isfinite(split.turn_purity) & (split.turn_purity >= 0.8) & (~transition_mask)
    slope_idx = np.where(split.mask_theta.reshape(-1) == 1)[0]

    metrics: Dict[str, object] = {
        "loss_total": float(loss_total),
        "acc_main": float(np.mean(pred_main == y_main)),
        "acc_turn": float(np.mean(pred_turn_raw == y_turn_raw)),
        "acc_turn_pure": _masked_acc(pred_turn_raw, y_turn_raw, pure_mask),
        "acc_turn_transition": _masked_acc(pred_turn_raw, y_turn_raw, transition_mask),
        "flat_recall": float(recall_main[0]),
        "stall_recall": float(recall_main[1]),
        "slope_recall": float(recall_main[2]),
        "recall_main": [float(x) for x in recall_main],
        "turn_right_recall": float(recall_turn[0]),
        "turn_straight_recall": float(recall_turn[1]),
        "turn_left_recall": float(recall_turn[2]),
        "recall_turn": [float(x) for x in recall_turn],
        "cm_turn": cm_turn.tolist(),
        "n_turn_transition": int(transition_mask.sum()),
        "n_turn_pure": int(pure_mask.sum()),
        "cm_main": cm_main.tolist(),
    }
```

```
    metrics.update(_confidence_summary("main", main_confidence, pred_main == y_main))
    metrics.update(_confidence_summary("turn", turn_confidence, pred_turn_raw == y_turn_raw))

    if slope_idx.size == 0:
        metrics.update(
            {
                "theta_mae_rad": 0.0,
                "theta_mae_deg": 0.0,
                "uphill_recall": float("nan"),
                "downhill_recall": float("nan"),
                "slope_sign_acc": float("nan"),
            }
        )
    else:
        theta_err = np.abs(theta_hat[slope_idx] - theta_true[slope_idx])
        uphill_idx = slope_idx[theta_true[slope_idx] > 0]
        downhill_idx = slope_idx[theta_true[slope_idx] < 0]
        metrics.update(
            {
                "theta_mae_rad": float(theta_err.mean()),
                "theta_mae_deg": float(np.rad2deg(theta_err.mean())),
                "uphill_recall": _slope_sub_recall(pred_main, uphill_idx),
                "downhill_recall": _slope_sub_recall(pred_main, downhill_idx),
                "slope_sign_acc": float(np.mean(np.sign(theta_hat[slope_idx]) ==
np.sign(theta_true[slope_idx]))),
            }
        )
    metrics.update(_theta_control_metrics(theta_hat, theta_true, y_main, y_turn_raw,
split.mask_theta.reshape(-1)))
    return metrics


def selection_score(metrics: Dict[str, object], cfg=None) -> float:
    """验证集选模分数：主工况边界优先，兼顾转弯过渡和 theta。"""

    select_theta_weight = float(getattr(cfg, "select_theta_weight", 0.15))
    select_theta_ref_deg = max(float(getattr(cfg, "select_theta_ref_deg", 5.0)), 1e-6)
    theta_norm = float(metrics["theta_mae_deg"]) / select_theta_ref_deg
    flat_penalty = max(0.0, 0.90 - float(metrics["flat_recall"]))
    slope_penalty = max(0.0, 0.88 - float(metrics["slope_recall"]))
    turn_t = float(metrics["acc_turn_transition"])
    select_turn_weight = float(getattr(cfg, "select_turn_weight", 0.30))
    select_turn_t_weight = float(getattr(cfg, "select_turn_transition_weight", 1.00))
    select_turn_t_target = float(getattr(cfg, "select_turn_transition_target", 0.75))
    select_turn_left_weight = float(getattr(cfg, "select_turn_left_weight", 0.00))
    select_turn_left_target = float(getattr(cfg, "select_turn_left_target", 0.80))
    select_turn_lr_weight = float(getattr(cfg, "select_turn_lr_weight", 0.00))
    select_turn_lr_target = float(getattr(cfg, "select_turn_lr_target", 0.80))
    turn_t_penalty = 0.0 if np.isnan(turn_t) else max(0.0, select_turn_t_target - turn_t)
    turn_right = float(metrics.get("turn_right_recall", float("nan")))
    turn_left = float(metrics.get("turn_left_recall", float("nan")))
    turn_left_penalty = 0.0 if np.isnan(turn_left) else max(0.0, select_turn_left_target -
turn_left)
    if np.isnan(turn_right) or np.isnan(turn_left):
        turn_lr_penalty = 0.0
    else:
        turn_lr_penalty = max(0.0, select_turn_lr_target - min(turn_right, turn_left))
    flat_p95 = float(metrics.get("theta_flat_abs_p95_deg", float("nan")))
    flat_p95_target = float(getattr(cfg, "select_theta_flat_p95_target_deg", 1.0))
    flat_p95_penalty = 0.0 if np.isnan(flat_p95) else max(0.0, flat_p95 - flat_p95_target) / 5.0
    theta_p95_penalty = _metric_excess_penalty(
        metrics, "theta_abs_le_8_p95_abs_err_deg", float(getattr(cfg,
"select_theta_p95_target_deg", 1.0)), 5.0
    )
```

```
    near_flat_p95_penalty = _metric_excess_penalty(
        metrics,
        "theta_near_flat_abs_p95_deg",
        float(getattr(cfg, "select_theta_near_flat_p95_target_deg", 1.0)),
        5.0,
    )
    true_zero_p95_penalty = _metric_excess_penalty(
        metrics,
        "theta_true_zero_abs_p95_deg",
        float(getattr(cfg, "select_theta_true_zero_p95_target_deg", 1.0)),
        5.0,
    )
    flat_peak_penalty = _metric_excess_penalty(
        metrics,
        "theta_flat_abs_max_deg",
        float(getattr(cfg, "select_theta_flat_peak_target_deg", 3.0)),
        10.0,
    )
    small_neg_p95_penalty = _metric_excess_penalty(
        metrics,
        "theta_neg_4_2_p95_abs_err_deg",
        float(getattr(cfg, "select_theta_small_neg_p95_target_deg", 1.0)),
        5.0,
    )
    extreme_p95_penalty = 0.5 * (
        _metric_excess_penalty(
            metrics,
            "theta_neg_8_6_p95_abs_err_deg",
            float(getattr(cfg, "select_theta_extreme_p95_target_deg", 1.0)),
            5.0,
        )
        + _metric_excess_penalty(
            metrics,
            "theta_pos_6_8_p95_abs_err_deg",
            float(getattr(cfg, "select_theta_extreme_p95_target_deg", 1.0)),
            5.0,
        )
    )
    edge_p95_penalty = 0.5 * (
        _metric_excess_penalty(
            metrics,
            "theta_neg_10_8_p95_abs_err_deg",
            float(getattr(cfg, "select_theta_edge_p95_target_deg", 1.2)),
            5.0,
        )
        + _metric_excess_penalty(
            metrics,
            "theta_pos_8_10_p95_abs_err_deg",
            float(getattr(cfg, "select_theta_edge_p95_target_deg", 1.2)),
            5.0,
        )
    )
    small_nonzero_p95_penalty = 0.5 * (
        _metric_excess_penalty(
            metrics,
            "theta_neg_2_0p5_p95_abs_err_deg",
            float(getattr(cfg, "select_theta_small_nonzero_p95_target_deg", 1.0)),
            5.0,
        )
        + _metric_excess_penalty(
```

```
 961:             metrics,
 962:             "theta_pos_0p5_2_p95_abs_err_deg",
 963:             float(getattr(cfg, "select_theta_small_nonzero_p95_target_deg", 1.0)),
 964:             5.0,
 965:         )
 966:     )
 967:     flat_bias = abs(float(metrics.get("theta_flat_bias_deg", float("nan"))))
 968:     flat_bias_target = float(getattr(cfg, "select_theta_flat_bias_target_deg", 0.2))
 969:     flat_bias_penalty = 0.0 if np.isnan(flat_bias) else max(0.0, flat_bias - flat_bias_target) /
5.0
 970:     return (
 971:         float(metrics["loss_total"])
 972:         + 1.00 * (1.0 - float(metrics["acc_main"]))
 973:         + select_turn_weight * (1.0 - float(metrics["acc_turn"]))
 974:         + select_theta_weight * theta_norm
 975:         + 2.00 * flat_penalty
 976:         + 2.00 * slope_penalty
 977:         + select_turn_t_weight * turn_t_penalty
 978:         + select_turn_left_weight * turn_left_penalty
 979:         + select_turn_lr_weight * turn_lr_penalty
 980:         + float(getattr(cfg, "select_theta_p95_weight", 0.0)) * theta_p95_penalty
 981:         + float(getattr(cfg, "select_theta_flat_p95_weight", 0.0)) * flat_p95_penalty
 982:         + float(getattr(cfg, "select_theta_near_flat_p95_weight", 0.0)) * near_flat_p95_penalty
 983:         + float(getattr(cfg, "select_theta_true_zero_p95_weight", 0.0)) * true_zero_p95_penalty
 984:         + float(getattr(cfg, "select_theta_flat_peak_weight", 0.0)) * flat_peak_penalty
 985:         + float(getattr(cfg, "select_theta_small_neg_p95_weight", 0.0)) * small_neg_p95_penalty
 986:         + float(getattr(cfg, "select_theta_extreme_p95_weight", 0.0)) * extreme_p95_penalty
 987:         + float(getattr(cfg, "select_theta_edge_p95_weight", 0.0)) * edge_p95_penalty
 988:         + float(getattr(cfg, "select_theta_small_nonzero_p95_weight", 0.0)) *
small_nonzero_p95_penalty
 989:         + float(getattr(cfg, "select_theta_flat_bias_weight", 0.0)) * flat_bias_penalty
 990:     )
 991:
 992:
 993: def seed42_gate(metrics: Dict[str, object], thresholds: GateThresholds = GateThresholds()) ->
Tuple[bool, List[str]]:
 994:     """判断 seed42 是否进入三 seed。"""
 995:
 996:     checks = [
 997:         ("acc_main", float(metrics["acc_main"]), ">=", thresholds.acc_main),
 998:         ("flat_recall", float(metrics["flat_recall"]), ">=", thresholds.flat_recall),
 999:         ("slope_recall", float(metrics["slope_recall"]), ">=", thresholds.slope_recall),
1000:         ("acc_turn_transition", float(metrics["acc_turn_transition"]), ">=",
thresholds.acc_turn_transition),
1001:         ("theta_mae_deg", float(metrics["theta_mae_deg"]), "<=", thresholds.theta_mae_deg),
1002:     ]
1003:     failed: List[str] = []
1004:     for name, value, op, threshold in checks:
1005:         passed = value >= threshold if op == ">=" else value <= threshold
1006:         if not passed:
1007:             failed.append(f"{name} {op} {threshold:.4f} 未满足，实际 {value:.4f}")
1008:     return len(failed) == 0, failed
1009:
1010:
1011: def _metric_excess_penalty(metrics: Dict[str, object], key: str, target: float, scale: float) ->
float:
1012:     value = float(metrics.get(key, float("nan")))
1013:     if np.isnan(value):
1014:         return 0.0
1015:     return max(0.0, value - target) / max(scale, 1e-6)
1016:
1017:
1018: def metric_row(seed: int, best_epoch: int, metrics: Dict[str, object], paths: Dict[str, str]) ->
Dict[str, object]:
1019:     return {
1020:         "model": "ModernTCN-small",
```

```
1021:         "seed": seed,
1022:         "best_epoch": best_epoch,
1023:         "acc_main": metrics["acc_main"],
1024:         "acc_turn": metrics["acc_turn"],
1025:         "acc_turn_pure": metrics["acc_turn_pure"],
1026:         "acc_turn_transition": metrics["acc_turn_transition"],
1027:         "main_confidence_mean": metrics.get("main_confidence_mean", float("nan")),
1028:         "main_low_conf_0p60_ratio": metrics.get("main_low_conf_0p60_ratio", float("nan")),
1029:         "main_low_conf_0p70_ratio": metrics.get("main_low_conf_0p70_ratio", float("nan")),
1030:         "turn_confidence_mean": metrics.get("turn_confidence_mean", float("nan")),
1031:         "turn_low_conf_0p60_ratio": metrics.get("turn_low_conf_0p60_ratio", float("nan")),
1032:         "turn_low_conf_0p70_ratio": metrics.get("turn_low_conf_0p70_ratio", float("nan")),
1033:         "turn_right_recall": metrics["turn_right_recall"],
1034:         "turn_straight_recall": metrics["turn_straight_recall"],
1035:         "turn_left_recall": metrics["turn_left_recall"],
1036:         "theta_mae_deg": metrics["theta_mae_deg"],
1037:         "theta_abs_le_8_mae_deg": metrics.get("theta_abs_le_8_mae_deg", float("nan")),
1038:         "theta_abs_le_8_rmse_deg": metrics.get("theta_abs_le_8_rmse_deg", float("nan")),
1039:         "theta_abs_le_8_p95_abs_err_deg": metrics.get("theta_abs_le_8_p95_abs_err_deg",
float("nan")),
1040:         "theta_abs_le_8_max_abs_err_deg": metrics.get("theta_abs_le_8_max_abs_err_deg",
float("nan")),
1041:         "theta_abs_le_8_bias_deg": metrics.get("theta_abs_le_8_bias_deg", float("nan")),
1042:         "theta_abs_le_10_mae_deg": metrics.get("theta_abs_le_10_mae_deg", float("nan")),
1043:         "theta_abs_le_10_rmse_deg": metrics.get("theta_abs_le_10_rmse_deg", float("nan")),
1044:         "theta_abs_le_10_p95_abs_err_deg": metrics.get("theta_abs_le_10_p95_abs_err_deg",
float("nan")),
1045:         "theta_abs_le_10_max_abs_err_deg": metrics.get("theta_abs_le_10_max_abs_err_deg",
float("nan")),
1046:         "theta_abs_le_10_bias_deg": metrics.get("theta_abs_le_10_bias_deg", float("nan")),
1047:         "theta_pos_8_10_mae_deg": metrics.get("theta_pos_8_10_mae_deg", float("nan")),
1048:         "theta_pos_8_10_rmse_deg": metrics.get("theta_pos_8_10_rmse_deg", float("nan")),
1049:         "theta_pos_8_10_p95_abs_err_deg": metrics.get("theta_pos_8_10_p95_abs_err_deg",
float("nan")),
1050:         "theta_pos_8_10_max_abs_err_deg": metrics.get("theta_pos_8_10_max_abs_err_deg",
float("nan")),
1051:         "theta_pos_8_10_bias_deg": metrics.get("theta_pos_8_10_bias_deg", float("nan")),
1052:         "theta_pos_8_10_n": metrics.get("theta_pos_8_10_n", float("nan")),
1053:         "theta_neg_10_8_mae_deg": metrics.get("theta_neg_10_8_mae_deg", float("nan")),
1054:         "theta_neg_10_8_rmse_deg": metrics.get("theta_neg_10_8_rmse_deg", float("nan")),
1055:         "theta_neg_10_8_p95_abs_err_deg": metrics.get("theta_neg_10_8_p95_abs_err_deg",
float("nan")),
1056:         "theta_neg_10_8_max_abs_err_deg": metrics.get("theta_neg_10_8_max_abs_err_deg",
float("nan")),
1057:         "theta_neg_10_8_bias_deg": metrics.get("theta_neg_10_8_bias_deg", float("nan")),
1058:         "theta_neg_10_8_n": metrics.get("theta_neg_10_8_n", float("nan")),
1059:         "theta_pos_6_8_mae_deg": metrics.get("theta_pos_6_8_mae_deg", float("nan")),
1060:         "theta_pos_6_8_rmse_deg": metrics.get("theta_pos_6_8_rmse_deg", float("nan")),
1061:         "theta_pos_6_8_p95_abs_err_deg": metrics.get("theta_pos_6_8_p95_abs_err_deg",
float("nan")),
1062:         "theta_pos_6_8_max_abs_err_deg": metrics.get("theta_pos_6_8_max_abs_err_deg",
float("nan")),
1063:         "theta_pos_6_8_bias_deg": metrics.get("theta_pos_6_8_bias_deg", float("nan")),
1064:         "theta_pos_6_8_n": metrics.get("theta_pos_6_8_n", float("nan")),
1065:         "theta_neg_8_6_mae_deg": metrics.get("theta_neg_8_6_mae_deg", float("nan")),
1066:         "theta_neg_8_6_rmse_deg": metrics.get("theta_neg_8_6_rmse_deg", float("nan")),
1067:         "theta_neg_8_6_p95_abs_err_deg": metrics.get("theta_neg_8_6_p95_abs_err_deg",
float("nan")),
1068:         "theta_neg_8_6_max_abs_err_deg": metrics.get("theta_neg_8_6_max_abs_err_deg",
float("nan")),
1069:         "theta_neg_8_6_bias_deg": metrics.get("theta_neg_8_6_bias_deg", float("nan")),
1070:         "theta_neg_8_6_n": metrics.get("theta_neg_8_6_n", float("nan")),
1071:         "theta_active_abs_ge_2_mae_deg": metrics.get("theta_active_abs_ge_2_mae_deg",
float("nan")),
1072:         "theta_active_abs_ge_2_p95_abs_err_deg":
metrics.get("theta_active_abs_ge_2_p95_abs_err_deg", float("nan")),
```

```
1073:         "theta_neg_4_2_mae_deg": metrics.get("theta_neg_4_2_mae_deg", float("nan")),
1074:         "theta_neg_4_2_p95_abs_err_deg": metrics.get("theta_neg_4_2_p95_abs_err_deg",
float("nan")),
1075:         "theta_neg_4_2_bias_deg": metrics.get("theta_neg_4_2_bias_deg", float("nan")),
1076:         "theta_neg_2_0p5_mae_deg": metrics.get("theta_neg_2_0p5_mae_deg", float("nan")),
1077:         "theta_neg_2_0p5_p95_abs_err_deg": metrics.get("theta_neg_2_0p5_p95_abs_err_deg",
float("nan")),
1078:         "theta_neg_2_0p5_bias_deg": metrics.get("theta_neg_2_0p5_bias_deg", float("nan")),
1079:         "theta_pos_0p5_2_mae_deg": metrics.get("theta_pos_0p5_2_mae_deg", float("nan")),
1080:         "theta_pos_0p5_2_p95_abs_err_deg": metrics.get("theta_pos_0p5_2_p95_abs_err_deg",
float("nan")),
```

```
1081:         "theta_pos_0p5_2_bias_deg": metrics.get("theta_pos_0p5_2_bias_deg", float("nan")),
1082:         "theta_flat_abs_p95_deg": metrics.get("theta_flat_abs_p95_deg", float("nan")),
1083:         "theta_flat_abs_max_deg": metrics.get("theta_flat_abs_max_deg", float("nan")),
1084:         "theta_flat_bias_deg": metrics.get("theta_flat_bias_deg", float("nan")),
1085:         "theta_near_flat_abs_p95_deg": metrics.get("theta_near_flat_abs_p95_deg", float("nan")),
1086:         "theta_near_flat_abs_max_deg": metrics.get("theta_near_flat_abs_max_deg", float("nan")),
1087:         "theta_near_flat_bias_deg": metrics.get("theta_near_flat_bias_deg", float("nan")),
1088:         "theta_true_zero_mae_deg": metrics.get("theta_true_zero_mae_deg", float("nan")),
1089:         "theta_true_zero_abs_p95_deg": metrics.get("theta_true_zero_abs_p95_deg", float("nan")),
1090:         "theta_true_zero_abs_max_deg": metrics.get("theta_true_zero_abs_max_deg", float("nan")),
1091:         "theta_true_zero_bias_deg": metrics.get("theta_true_zero_bias_deg", float("nan")),
1092:         "theta_flat_turn_abs_p95_deg": metrics.get("theta_flat_turn_abs_p95_deg", float("nan")),
1093:         "theta_flat_turn_abs_max_deg": metrics.get("theta_flat_turn_abs_max_deg", float("nan")),
1094:         "flat_recall": metrics["flat_recall"],
1095:         "stall_recall": metrics["stall_recall"],
1096:         "slope_recall": metrics["slope_recall"],
1097:         "uphill_recall": metrics["uphill_recall"],
1098:         "downhill_recall": metrics["downhill_recall"],
1099:         "checkpoint_file": paths.get("checkpoint_file", ""),
1100:         "onnx_file": paths.get("onnx_file", ""),
1101:         "report_file": paths.get("report_file", ""),
1102:     }
1103:
1104:
1105: def _weighted_ce(logits: torch.Tensor, labels: torch.Tensor, class_weights: torch.Tensor,
sample_weights: torch.Tensor) -> torch.Tensor:
1106:     class_weights = class_weights.to(logits.device)
1107:     per_sample = F.cross_entropy(logits, labels, weight=class_weights, reduction="none")
1108:     denom = (class_weights[labels] * sample_weights).sum().clamp_min(1e-8)
1109:     return (per_sample * sample_weights).sum() / denom
1110:
1111:
1112: def _masked_weighted_mse(pred: torch.Tensor, target: torch.Tensor, mask: torch.Tensor, weight:
torch.Tensor) -> torch.Tensor:
1113:     wm = mask * weight
1114:     return (((pred - target) ** 2) * wm).sum() / wm.sum().clamp_min(1e-8)
1115:
1116:
1117: def _masked_weighted_abs_excess_mse(
1118:     pred: torch.Tensor,
1119:     target: torch.Tensor,
1120:     mask: torch.Tensor,
1121:     weight: torch.Tensor,
1122:     target_deg: float,
1123: ) -> torch.Tensor:
1124:     wm = mask * weight
1125:     target_rad = torch.deg2rad(torch.tensor(float(target_deg), device=pred.device,
dtype=pred.dtype))
1126:     excess = torch.relu(torch.abs(pred - target) - target_rad)
1127:     return ((excess ** 2) * wm).sum() / wm.sum().clamp_min(1e-8)
1128:
1129:
1130: def _main_slope_sample_weight(y_main: torch.Tensor, theta: torch.Tensor, cfg) -> torch.Tensor:
1131:     w = torch.ones_like(theta)
1132:     w = torch.where((y_main == 2) & (theta < 0), torch.full_like(w, cfg.main_neg_slope_weight), w)
1133:     w = torch.where((y_main == 2) & (theta > 0), torch.full_like(w, cfg.main_pos_slope_weight), w)
1134:     return w
1135:
1136:
1137: def _theta_sign_weight(theta: torch.Tensor, cfg) -> torch.Tensor:
1138:     w = torch.ones_like(theta)
1139:     w = torch.where(theta < 0, torch.full_like(w, cfg.theta_neg_weight), w)
1140:     w = torch.where(theta > 0, torch.full_like(w, cfg.theta_pos_weight), w)
```

```
    return w


def _confusion_matrix(truth: np.ndarray, pred: np.ndarray, num_classes: int) -> np.ndarray:
    cm = np.zeros((num_classes, num_classes), dtype=np.int64)
    for t, p in zip(truth.reshape(-1), pred.reshape(-1)):
        if 0 <= int(t) < num_classes and 0 <= int(p) < num_classes:
            cm[int(t), int(p)] += 1
    return cm


def _softmax_np(logits: np.ndarray) -> np.ndarray:
    logits = np.asarray(logits, dtype=np.float64)
    logits = logits - np.max(logits, axis=1, keepdims=True)
    exp_logits = np.exp(logits)
    return exp_logits / np.maximum(exp_logits.sum(axis=1, keepdims=True), 1e-12)


def _confidence_summary(prefix: str, confidence: np.ndarray, correct: np.ndarray) -> Dict[str,
object]:
    confidence = np.asarray(confidence, dtype=np.float64).reshape(-1)
    correct = np.asarray(correct).reshape(-1).astype(bool)
    out: Dict[str, object] = {
        f"{prefix}_confidence_mean": float(np.mean(confidence)) if confidence.size else
float("nan"),
         f"{prefix}_confidence_error_mean": float(np.mean(confidence[~correct])) if np.any(~correct)
else float("nan"),
        f"{prefix}_low_conf_0p60_ratio": float(np.mean(confidence < 0.60)) if confidence.size else
float("nan"),
        f"{prefix}_low_conf_0p70_ratio": float(np.mean(confidence < 0.70)) if confidence.size else
float("nan"),
    }
    edges = np.array([0.0, 0.60, 0.70, 0.80, 0.90, 1.0000001], dtype=np.float64)
    bins = []
    for i in range(len(edges) - 1):
        lo = edges[i]
        hi = edges[i + 1]
        mask = (confidence >= lo) & (confidence < hi)
        n = int(mask.sum())
        bins.append(
            {
                "bin": f"[{lo:.2f},{min(hi, 1.0):.2f})",
                "n": n,
                "error_rate": float(np.mean(~correct[mask])) if n else float("nan"),
                "mean_confidence": float(np.mean(confidence[mask])) if n else float("nan"),
            }
        )
    out[f"{prefix}_confidence_bins"] = bins
    return out


def _masked_acc(pred: np.ndarray, truth: np.ndarray, mask: np.ndarray) -> float:
    mask = mask.astype(bool)
    if not np.any(mask):
        return float("nan")
    return float(np.mean(pred[mask] == truth[mask]))


def _slope_sub_recall(pred_main: np.ndarray, idx: Iterable[int]) -> float:
    idx = np.asarray(list(idx), dtype=np.int64)
    if idx.size == 0:
        return float("nan")
    return float(np.mean(pred_main[idx] == 2))


```

```
1201: def _theta_control_metrics(
1202:     theta_hat: np.ndarray,
1203:     theta_true: np.ndarray,
1204:     y_main: np.ndarray,
1205:     y_turn_raw: np.ndarray,
1206:     mask_theta: np.ndarray,
1207: ) -> Dict[str, float]:
1208:     """Metrics for whether theta_hat is safe enough to be used by scheduling."""
1209:
1210:     flat_mask = y_main == 0
1211:     near_flat_mask = np.abs(theta_true) <= np.deg2rad(0.5)
1212:     true_zero_mask = np.isclose(theta_true, 0.0, atol=np.deg2rad(1e-6))
1213:     flat_turn_mask = flat_mask & near_flat_mask & (y_turn_raw != 0)
1214:     slope_mask = np.asarray(mask_theta).reshape(-1).astype(bool)
1215:
1216:     out: Dict[str, float] = {}
1217:     out.update(_theta_zone("theta_flat", theta_hat, theta_true, flat_mask))
1218:     out.update(_theta_zone("theta_near_flat", theta_hat, theta_true, near_flat_mask))
1219:     out.update(_theta_zone("theta_true_zero", theta_hat, theta_true, true_zero_mask))
1220:     out.update(_theta_zone("theta_flat_turn", theta_hat, theta_true, flat_turn_mask))
1221:     out.update(_theta_zone("theta_slope_control", theta_hat, theta_true, slope_mask))
1222:     theta_deg = np.rad2deg(theta_true)
1223:     out.update(_theta_error_zone("theta_all", theta_hat, theta_true, slope_mask))
1224:     out.update(_theta_error_zone("theta_active_abs_ge_2", theta_hat, theta_true, slope_mask &
(np.abs(theta_deg) >= 2.0)))
1225:     out.update(_theta_error_zone("theta_abs_le_8", theta_hat, theta_true, slope_mask &
(np.abs(theta_deg) <= 8.0)))
1226:     out.update(_theta_error_zone("theta_abs_le_10", theta_hat, theta_true, slope_mask &
(np.abs(theta_deg) <= 10.0)))
1227:     out.update(_theta_error_zone("theta_pos_8_10", theta_hat, theta_true, slope_mask &
(theta_deg >= 8.0) & (theta_deg <= 10.0)))
1228:     out.update(_theta_error_zone("theta_neg_10_8", theta_hat, theta_true, slope_mask &
(theta_deg >= -10.0) & (theta_deg <= -8.0)))
1229:     out.update(_theta_error_zone("theta_pos_6_8", theta_hat, theta_true, slope_mask &
(theta_deg >= 6.0) & (theta_deg <= 8.0)))
1230:     out.update(_theta_error_zone("theta_neg_8_6", theta_hat, theta_true, slope_mask &
(theta_deg >= -8.0) & (theta_deg <= -6.0)))
1231:     out.update(_theta_error_zone("theta_neg_4_2", theta_hat, theta_true, slope_mask &
(theta_deg >= -4.0) & (theta_deg <= -2.0)))
1232:     out.update(_theta_error_zone("theta_neg_2_0p5", theta_hat, theta_true, slope_mask &
(theta_deg >= -2.0) & (theta_deg <= -0.5)))
1233:     out.update(_theta_error_zone("theta_pos_0p5_2", theta_hat, theta_true, slope_mask &
(theta_deg >= 0.5) & (theta_deg <= 2.0)))
1234:     return out
1235:
1236:
1237: def _theta_error_zone(prefix: str, theta_hat: np.ndarray, theta_true: np.ndarray, mask: np.ndarray)
-> Dict[str, float]:
1238:     mask = np.asarray(mask).reshape(-1).astype(bool)
1239:     if not np.any(mask):
1240:         return {
1241:             f"{prefix}_mae_deg": float("nan"),
1242:             f"{prefix}_rmse_deg": float("nan"),
1243:             f"{prefix}_p95_abs_err_deg": float("nan"),
1244:             f"{prefix}_max_abs_err_deg": float("nan"),
1245:             f"{prefix}_bias_deg": float("nan"),
1246:             f"{prefix}_n": 0.0,
1247:         }
1248:     err_deg = np.rad2deg(theta_hat[mask] - theta_true[mask])
1249:     return {
1250:         f"{prefix}_mae_deg": float(np.mean(np.abs(err_deg))),
1251:         f"{prefix}_rmse_deg": float(np.sqrt(np.mean(err_deg ** 2))),
1252:         f"{prefix}_p95_abs_err_deg": float(np.percentile(np.abs(err_deg), 95)),
1253:         f"{prefix}_max_abs_err_deg": float(np.max(np.abs(err_deg))),
1254:         f"{prefix}_bias_deg": float(np.mean(err_deg)),
1255:         f"{prefix}_n": float(np.count_nonzero(mask)),
```

```
1256:     }
1257:
1258:
1259: def _theta_zone(prefix: str, theta_hat: np.ndarray, theta_true: np.ndarray, mask: np.ndarray) ->
Dict[str, float]:
1260:     mask = np.asarray(mask).reshape(-1).astype(bool)
```

```
    if not np.any(mask):
        return {
            f"{prefix}_mae_deg": float("nan"),
            f"{prefix}_abs_p95_deg": float("nan"),
            f"{prefix}_abs_max_deg": float("nan"),
            f"{prefix}_bias_deg": float("nan"),
            f"{prefix}_n": 0.0,
        }
    err = theta_hat[mask] - theta_true[mask]
    abs_theta_deg = np.abs(np.rad2deg(theta_hat[mask]))
    return {
        f"{prefix}_mae_deg": float(np.rad2deg(np.mean(np.abs(err)))),
        f"{prefix}_abs_p95_deg": float(np.percentile(abs_theta_deg, 95)),
        f"{prefix}_abs_max_deg": float(np.max(abs_theta_deg)),
        f"{prefix}_bias_deg": float(np.rad2deg(np.mean(err))),
        f"{prefix}_n": float(np.count_nonzero(mask)),
    }

===== FILE: src/ModernTCN/train_modern_tcn.py =====
"""训练 ModernTCN-small，并输出与 TCN/GRU baseline 对齐的测试指标。

使用方式：
    python ModernTCN/train_modern_tcn.py --seed 42

重要约束：
    1. 默认读取 data/tcn/ModernTCN_dataset_v4_industrial.mat。
    2. 不重新划分 split，不重新拟合 scaler。
    3. 多 seed 实验建议通过 results/modern_tcn/scripts 下的脚本统一启动。
"""

from __future__ import annotations

import argparse
import csv
import json
import math
import random
import time
from pathlib import Path
from typing import Dict, Iterable, Tuple

import numpy as np
import torch
from torch.utils.data import DataLoader

from modern_tcn_data import AGVWindowDataset, class_weights, find_project_root,
load_modern_tcn_dataset
from modern_tcn_metrics import compute_metrics, metric_row, multitask_loss, seed42_gate,
selection_score
from modern_tcn_model import ModernTCNConfig, ModernTCNSmall


def parse_args() -> argparse.Namespace:
    p = argparse.ArgumentParser(description="ModernTCN-small 第一阶段训练脚本")
    p.add_argument("--seed", type=int, default=42)
    p.add_argument("--dataset-file", type=str, default="")
    p.add_argument("--run-tag", type=str, default="")
    p.add_argument("--epochs", type=int, default=120)
    p.add_argument("--batch-size", type=int, default=256)
    p.add_argument("--lr", type=float, default=1e-3)
    p.add_argument("--weight-decay", type=float, default=1e-4)
    p.add_argument("--patience", type=int, default=25)
```

```
    p.add_argument("--min-epochs", type=int, default=30)
    p.add_argument("--channels", type=int, default=64)
    p.add_argument("--blocks", type=int, default=5)
    p.add_argument("--kernel-size", type=int, default=31)
    p.add_argument(
        "--temporal-padding",
        type=str,
        default="same",
        choices=["same", "causal"],
        help="Temporal convolution padding mode. 默认 same，causal 仅用于因果消融实验。",
    )
    p.add_argument("--dropout", type=float, default=0.15)
    p.add_argument("--turn-head-source", type=str, default="full", choices=["full", "inputstats",
"kinematic_stats"])
    p.add_argument("--lambda-turn", type=float, default=0.05)
    p.add_argument("--lambda-theta", type=float, default=0.35)
    p.add_argument("--lambda-theta-flat", type=float, default=0.20)
    p.add_argument(
        "--theta-flat-loss-mode",
        type=str,
        default="near_zero",
        choices=["near_zero", "true_zero", "near_flat", "main_flat", "none"],
    )
    p.add_argument("--theta-flat-zero-tol-deg", type=float, default=0.3)
    p.add_argument("--lambda-theta-near-flat", type=float, default=0.0)
    p.add_argument("--theta-near-flat-deg", type=float, default=0.5)
    p.add_argument("--lambda-theta-error-excess", type=float, default=0.0)
    p.add_argument("--lambda-theta-flat-excess", type=float, default=0.0)
    p.add_argument("--lambda-theta-near-flat-excess", type=float, default=0.0)
    p.add_argument("--lambda-theta-true-zero-excess", type=float, default=0.0)
    p.add_argument("--lambda-theta-active-excess", type=float, default=0.0)
    p.add_argument("--lambda-theta-small-neg", type=float, default=0.0)
    p.add_argument("--lambda-theta-small-neg-excess", type=float, default=0.0)
    p.add_argument("--theta-excess-target-deg", type=float, default=1.0)
    p.add_argument("--theta-flat-excess-target-deg", type=float, default=0.5)
    p.add_argument("--theta-small-neg-min-deg", type=float, default=-4.0)
    p.add_argument("--theta-small-neg-max-deg", type=float, default=-2.0)
    p.add_argument("--theta-gate-mode", type=str, default="none", choices=["none",
"main_slope_prob"])
    p.add_argument("--theta-gate-power", type=float, default=1.0)
    p.add_argument("--theta-gate-floor", type=float, default=0.0)
    p.add_argument("--theta-neg-weight", type=float, default=1.0)
    p.add_argument("--theta-pos-weight", type=float, default=1.0)
    p.add_argument("--turn-transition-weight", type=float, default=1.0)
    p.add_argument("--turn-class-multipliers", type=float, nargs=3, default=[1.00, 1.10, 1.00])
    p.add_argument("--select-turn-weight", type=float, default=0.30)
    p.add_argument("--select-turn-transition-weight", type=float, default=1.00)
    p.add_argument("--select-turn-transition-target", type=float, default=0.75)
    p.add_argument("--select-turn-left-weight", type=float, default=0.00)
    p.add_argument("--select-turn-left-target", type=float, default=0.80)
    p.add_argument("--select-turn-lr-weight", type=float, default=0.00)
    p.add_argument("--select-turn-lr-target", type=float, default=0.80)
    p.add_argument("--select-theta-weight", type=float, default=0.15)
    p.add_argument("--select-theta-ref-deg", type=float, default=5.0)
    p.add_argument("--select-theta-p95-weight", type=float, default=0.0)
    p.add_argument("--select-theta-p95-target-deg", type=float, default=1.0)
    p.add_argument("--select-theta-flat-p95-weight", type=float, default=0.0)
    p.add_argument("--select-theta-flat-p95-target-deg", type=float, default=1.0)
    p.add_argument("--select-theta-near-flat-p95-weight", type=float, default=0.0)
    p.add_argument("--select-theta-near-flat-p95-target-deg", type=float, default=1.0)
    p.add_argument("--select-theta-true-zero-p95-weight", type=float, default=0.0)
    p.add_argument("--select-theta-true-zero-p95-target-deg", type=float, default=1.0)
```

```
    p.add_argument("--select-theta-extreme-p95-weight", type=float, default=0.0)
    p.add_argument("--select-theta-extreme-p95-target-deg", type=float, default=1.0)
    p.add_argument("--select-theta-edge-p95-weight", type=float, default=0.0)
    p.add_argument("--select-theta-edge-p95-target-deg", type=float, default=1.2)
    p.add_argument("--select-theta-small-nonzero-p95-weight", type=float, default=0.0)
    p.add_argument("--select-theta-small-nonzero-p95-target-deg", type=float, default=1.0)
    p.add_argument("--select-theta-flat-bias-weight", type=float, default=0.0)
    p.add_argument("--select-theta-flat-bias-target-deg", type=float, default=0.2)
    p.add_argument("--device", type=str, default="auto", choices=["auto", "cpu", "cuda"])
    p.add_argument("--num-workers", type=int, default=0)
    p.add_argument("--limit-train", type=int, default=0, help="仅用于 smoke test，正式实验必须为 0。
")
    p.add_argument("--limit-val", type=int, default=0, help="仅用于 smoke test，正式实验必须为 0。")
    p.add_argument("--limit-test", type=int, default=0, help="仅用于 smoke test，正式实验必须为 0。")
    p.add_argument("--dry-run", action="store_true", help="只读取数据并跑一次前向，不保存训练结果。")
    return p.parse_args()


def main() -> Dict[str, object]:
    args = parse_args()
    return train_one_seed(args)


def train_one_seed(args: argparse.Namespace) -> Dict[str, object]:
    root = find_project_root()
    run_tag = args.run_tag or f"modern_tcn_v4_industrial_seed{args.seed}"
    out_dir = root / "results" / "modern_tcn" / run_tag
    dataset_file = Path(args.dataset_file) if args.dataset_file else None

    _set_seed(args.seed)
    device = _select_device(args.device)
    data = load_modern_tcn_dataset(
        dataset_file=dataset_file,
        limit_train=args.limit_train,
        limit_val=args.limit_val,
        limit_test=args.limit_test,
    )
    train_split = data["train"]
    val_split = data["val"]
    test_split = data["test"]
    contract = data["contract"]

    cfg = ModernTCNConfig(
        input_dim=contract.input_dim,
        seq_len=contract.seq_len,
        channels=args.channels,
        blocks=args.blocks,
        kernel_size=args.kernel_size,
        temporal_padding=getattr(args, "temporal_padding", "same"),
        dropout=args.dropout,
        turn_head_source=args.turn_head_source,
        lambda_turn=args.lambda_turn,
        lambda_theta=args.lambda_theta,
        lambda_theta_flat=args.lambda_theta_flat,
        theta_flat_loss_mode=args.theta_flat_loss_mode,
        theta_flat_zero_tol_deg=args.theta_flat_zero_tol_deg,
        lambda_theta_near_flat=args.lambda_theta_near_flat,
        theta_near_flat_deg=args.theta_near_flat_deg,
        lambda_theta_error_excess=args.lambda_theta_error_excess,
        lambda_theta_flat_excess=args.lambda_theta_flat_excess,
        lambda_theta_near_flat_excess=args.lambda_theta_near_flat_excess,
```

```
1441:         lambda_theta_true_zero_excess=args.lambda_theta_true_zero_excess,
1442:         lambda_theta_active_excess=args.lambda_theta_active_excess,
1443:         lambda_theta_small_neg=args.lambda_theta_small_neg,
1444:         lambda_theta_small_neg_excess=args.lambda_theta_small_neg_excess,
1445:         theta_excess_target_deg=args.theta_excess_target_deg,
1446:         theta_flat_excess_target_deg=args.theta_flat_excess_target_deg,
1447:         theta_small_neg_min_deg=args.theta_small_neg_min_deg,
1448:         theta_small_neg_max_deg=args.theta_small_neg_max_deg,
1449:         theta_gate_mode=args.theta_gate_mode,
1450:         theta_gate_power=args.theta_gate_power,
1451:         theta_gate_floor=args.theta_gate_floor,
1452:         theta_neg_weight=args.theta_neg_weight,
1453:         theta_pos_weight=args.theta_pos_weight,
1454:         turn_transition_weight=args.turn_transition_weight,
1455:         turn_class_multipliers=tuple(args.turn_class_multipliers),
1456:         select_turn_weight=args.select_turn_weight,
1457:         select_turn_transition_weight=args.select_turn_transition_weight,
1458:         select_turn_transition_target=getattr(args, "select_turn_transition_target", 0.75),
1459:         select_turn_left_weight=args.select_turn_left_weight,
1460:         select_turn_left_target=args.select_turn_left_target,
1461:         select_turn_lr_weight=getattr(args, "select_turn_lr_weight", 0.0),
1462:         select_turn_lr_target=getattr(args, "select_turn_lr_target", 0.80),
1463:         select_theta_weight=args.select_theta_weight,
1464:         select_theta_ref_deg=args.select_theta_ref_deg,
1465:         select_theta_p95_weight=args.select_theta_p95_weight,
1466:         select_theta_p95_target_deg=getattr(args, "select_theta_p95_target_deg", 1.0),
1467:         select_theta_flat_p95_weight=args.select_theta_flat_p95_weight,
1468:         select_theta_flat_p95_target_deg=getattr(args, "select_theta_flat_p95_target_deg", 1.0),
1469:         select_theta_near_flat_p95_weight=args.select_theta_near_flat_p95_weight,
1470:         select_theta_near_flat_p95_target_deg=getattr(args,
"select_theta_near_flat_p95_target_deg", 1.0),
1471:         select_theta_true_zero_p95_weight=args.select_theta_true_zero_p95_weight,
1472:         select_theta_true_zero_p95_target_deg=getattr(args,
"select_theta_true_zero_p95_target_deg", 1.0),
1473:         select_theta_extreme_p95_weight=args.select_theta_extreme_p95_weight,
1474:         select_theta_extreme_p95_target_deg=getattr(args, "select_theta_extreme_p95_target_deg",
1.0),
1475:         select_theta_edge_p95_weight=getattr(args, "select_theta_edge_p95_weight", 0.0),
1476:         select_theta_edge_p95_target_deg=getattr(args, "select_theta_edge_p95_target_deg", 1.2),
1477:         select_theta_small_nonzero_p95_weight=args.select_theta_small_nonzero_p95_weight,
1478:         select_theta_small_nonzero_p95_target_deg=getattr(args,
"select_theta_small_nonzero_p95_target_deg", 1.0),
1479:         select_theta_flat_bias_weight=args.select_theta_flat_bias_weight,
1480:         select_theta_flat_bias_target_deg=getattr(args, "select_theta_flat_bias_target_deg", 0.2),
1481:     )
1482:     model = ModernTCNSmall(cfg).to(device)
1483:
1484:     # smoke test 只验证数据契约和模型维度，不写任何模型文件。
1485:     if args.dry_run:
1486:         xb = torch.from_numpy(train_split.X[:4]).float().to(device)
1487:         with torch.no_grad():
1488:             outputs = model(xb)
1489:         print("[ModernTCN dry-run] 数据和模型前向检查通过")
1490:         print(f"  X: {tuple(xb.shape)}")
1491:         print(f"  logits_main/logits_turn/theta: {[tuple(o.shape) for o in outputs]}")
1492:         return {"status": "dry_run_ok"}
1493:
1494:     out_dir.mkdir(parents=True, exist_ok=True)
1495:     checkpoint_file = out_dir / f"modern_tcn_seed{args.seed}.pt"
1496:     summary_csv = out_dir / f"modern_tcn_seed{args.seed}_summary.csv"
1497:     history_csv = out_dir / f"modern_tcn_seed{args.seed}_history.csv"
1498:     report_file = out_dir / "ModernTCN_train_report.md"
1499:
1500:     train_loader = DataLoader(
```

```
        AGVWindowDataset(train_split),
        batch_size=args.batch_size,
        shuffle=True,
        num_workers=args.num_workers,
        pin_memory=(device.type == "cuda"),
    )
    val_loader = DataLoader(AGVWindowDataset(val_split), batch_size=args.batch_size, shuffle=False,
num_workers=0)
    test_loader = DataLoader(AGVWindowDataset(test_split), batch_size=args.batch_size,
shuffle=False, num_workers=0)

    class_w_main = class_weights(
        train_split.y_main,
        3,
        cfg.main_class_weight_method,
        list(cfg.main_class_multipliers),
    ).to(device)
    class_w_turn = class_weights(
        train_split.y_turn,
        3,
        cfg.turn_class_weight_method,
        list(cfg.turn_class_multipliers),
    ).to(device)

    opt = torch.optim.AdamW(model.parameters(), lr=args.lr, weight_decay=args.weight_decay)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=max(args.epochs, 1))

    print("[ModernTCN] 第一阶段训练开始")
    print(f"  seed={args.seed}, device={device}, out={out_dir}")
    print(f"  dataset={contract.dataset_file}")
    print(f"  train/val/test={len(train_split.X)}/{len(val_split.X)}/{len(test_split.X)}")
    print(
        f"  model channels={cfg.channels}, blocks={cfg.blocks}, kernel={cfg.kernel_size}, "
        f"temporal_padding={cfg.temporal_padding}"
    )

    best_score = math.inf
    best_epoch = 0
    best_state = None
    best_val_metrics: Dict[str, object] = {}
    patience_count = 0
    history_rows = []
    t0 = time.time()

    for epoch in range(1, args.epochs + 1):
        train_loss = _train_epoch(model, train_loader, opt, device, class_w_main, class_w_turn,
cfg)
        scheduler.step()
        val_loss, val_logits_main, val_logits_turn, val_theta = _predict_full(
            model, val_loader, val_split, device, class_w_main, class_w_turn, cfg
        )
        val_metrics = compute_metrics(val_logits_main, val_logits_turn, val_theta, val_split,
val_loss)
        score = selection_score(val_metrics, cfg)
        history_rows.append(_history_row(epoch, opt.param_groups[0]["lr"], train_loss, val_metrics,
score))

        if score < best_score:
            best_score = score
            best_epoch = epoch
            best_state = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}
            best_val_metrics = dict(val_metrics)
            patience_count = 0
        else:
            patience_count += 1
```

```
1561:
1562:         if epoch == 1 or epoch % 5 == 0 or epoch == args.epochs:
1563:             print(
1564:                 f"  epoch {epoch:03d} | train={train_loss:.4f} val={val_metrics['loss_total']:.4f}
"
1565:                 f"main={val_metrics['acc_main']:.4f} turn={val_metrics['acc_turn']:.4f} "
1566:                 f"turnL={val_metrics['turn_left_recall']:.4f}
turnT={val_metrics['acc_turn_transition']:.4f} "
1567:                 f"theta={val_metrics['theta_mae_deg']:.4f} score={score:.4f}"
1568:             )
1569:
1570:         if epoch >= args.min_epochs and patience_count >= args.patience:
1571:             print(f"[ModernTCN] 早停：epoch={epoch}, best_epoch={best_epoch}")
1572:             break
1573:
1574:     if best_state is None:
1575:         raise RuntimeError("训练未产生有效 checkpoint。")
1576:
1577:     model.load_state_dict(best_state)
1578:     test_loss, logits_main, logits_turn, theta_hat = _predict_full(
1579:         model, test_loader, test_split, device, class_w_main, class_w_turn, cfg
1580:     )
1581:     test_metrics = compute_metrics(logits_main, logits_turn, theta_hat, test_split, test_loss)
1582:     train_seconds = time.time() - t0
1583:
1584:     torch.save(
1585:         {
1586:             "model_state": best_state,
1587:             "model_config": cfg.to_dict(),
1588:             "seed": args.seed,
1589:             "best_epoch": best_epoch,
1590:             "best_val_score": best_score,
1591:             "best_val_metrics": best_val_metrics,
1592:             "test_metrics": test_metrics,
1593:             "contract": contract.__dict__,
1594:             "feat_names": data["feat_names"],
1595:             "scaler": data["scaler"],
1596:             "train_seconds": train_seconds,
1597:         },
1598:         checkpoint_file,
1599:     )
1600:
1601:     paths = {"checkpoint_file": str(checkpoint_file), "report_file": str(report_file)}
1602:     row = metric_row(args.seed, best_epoch, test_metrics, paths)
1603:     _write_csv(summary_csv, [row])
1604:     _write_csv(history_csv, history_rows)
1605:     _write_report(report_file, args, cfg, contract.__dict__, row, test_metrics, best_val_metrics,
train_seconds)
1606:
1607:     print("[ModernTCN] 训练完成")
1608:     print(f"  checkpoint: {checkpoint_file}")
1609:     print(f"  summary: {summary_csv}")
1610:     print(f"  report: {report_file}")
1611:     print(
1612:         f"  test main={test_metrics['acc_main']:.4f},
turnT={test_metrics['acc_turn_transition']:.4f}, "
1613:         f"turnL={test_metrics['turn_left_recall']:.4f}, theta={test_metrics['theta_mae_deg']:.4f},
flat={test_metrics['flat_recall']:.4f}, "
1614:         f"slope={test_metrics['slope_recall']:.4f}"
1615:     )
1616:
1617:     if args.seed == 42:
1618:         passed, failures = seed42_gate(test_metrics)
1619:         print(f"  seed42 gate pass={int(passed)}")
1620:         for msg in failures:
```

```
            print(f"    - {msg}")

    return {
        "checkpoint_file": str(checkpoint_file),
        "summary_csv": str(summary_csv),
        "report_file": str(report_file),
        "test_metrics": test_metrics,
        "best_epoch": best_epoch,
    }


def _train_epoch(model, loader, opt, device, class_w_main, class_w_turn, cfg) -> float:
    model.train()
    total_loss = 0.0
    total_n = 0
    for batch in loader:
        batch = _to_device(batch, device)
        opt.zero_grad(set_to_none=True)
        logits_main, logits_turn, theta_hat = model(batch["X"])
        loss, _ = multitask_loss(logits_main, logits_turn, theta_hat, batch, class_w_main,
class_w_turn, cfg)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        opt.step()
        n = int(batch["X"].shape[0])
        total_loss += float(loss.detach().cpu()) * n
        total_n += n
    return total_loss / max(total_n, 1)


@torch.no_grad()
def _predict_full(model, loader, split, device, class_w_main, class_w_turn, cfg) -> Tuple[float,
np.ndarray, np.ndarray, np.ndarray]:
    model.eval()
    logits_main_all = []
    logits_turn_all = []
    theta_all = []
    loss_sum = 0.0
    n_sum = 0
    for batch in loader:
        batch = _to_device(batch, device)
        logits_main, logits_turn, theta_hat = model(batch["X"])
        loss, _ = multitask_loss(logits_main, logits_turn, theta_hat, batch, class_w_main,
class_w_turn, cfg)
        n = int(batch["X"].shape[0])
        loss_sum += float(loss.detach().cpu()) * n
        n_sum += n
        logits_main_all.append(logits_main.detach().cpu().numpy())
        logits_turn_all.append(logits_turn.detach().cpu().numpy())
        theta_all.append(theta_hat.detach().cpu().numpy())
    return (
        loss_sum / max(n_sum, 1),
        np.concatenate(logits_main_all, axis=0),
        np.concatenate(logits_turn_all, axis=0),
        np.concatenate(theta_all, axis=0).reshape(-1),
    )


def _to_device(batch: Dict[str, torch.Tensor], device: torch.device) -> Dict[str, torch.Tensor]:
    return {k: v.to(device, non_blocking=True) for k, v in batch.items()}


def _set_seed(seed: int) -> None:
```

```
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = False
    torch.backends.cudnn.deterministic = True


def _select_device(mode: str) -> torch.device:
    if mode == "cuda":
        return torch.device("cuda")
    if mode == "cpu":
        return torch.device("cpu")
    return torch.device("cuda" if torch.cuda.is_available() else "cpu")


def _history_row(epoch: int, lr: float, train_loss: float, val_metrics: Dict[str, object], score:
float) -> Dict[str, object]:
    return {
        "epoch": epoch,
        "lr": lr,
        "train_loss": train_loss,
        "val_loss": val_metrics["loss_total"],
        "val_score": score,
        "val_acc_main": val_metrics["acc_main"],
        "val_acc_turn": val_metrics["acc_turn"],
        "val_main_confidence_mean": val_metrics.get("main_confidence_mean", float("nan")),
        "val_turn_confidence_mean": val_metrics.get("turn_confidence_mean", float("nan")),
        "val_main_low_conf_0p60_ratio": val_metrics.get("main_low_conf_0p60_ratio", float("nan")),
        "val_turn_low_conf_0p60_ratio": val_metrics.get("turn_low_conf_0p60_ratio", float("nan")),
        "val_turn_left_recall": val_metrics["turn_left_recall"],
        "val_turn_right_recall": val_metrics["turn_right_recall"],
        "val_acc_turn_transition": val_metrics["acc_turn_transition"],
        "val_theta_mae_deg": val_metrics["theta_mae_deg"],
        "val_theta_abs_le_10_p95_abs_err_deg": val_metrics["theta_abs_le_10_p95_abs_err_deg"],
        "val_theta_neg_10_8_p95_abs_err_deg": val_metrics["theta_neg_10_8_p95_abs_err_deg"],
        "val_theta_pos_8_10_p95_abs_err_deg": val_metrics["theta_pos_8_10_p95_abs_err_deg"],
        "val_theta_abs_le_8_p95_abs_err_deg": val_metrics["theta_abs_le_8_p95_abs_err_deg"],
        "val_theta_neg_8_6_p95_abs_err_deg": val_metrics["theta_neg_8_6_p95_abs_err_deg"],
        "val_theta_pos_6_8_p95_abs_err_deg": val_metrics["theta_pos_6_8_p95_abs_err_deg"],
        "val_theta_neg_2_0p5_p95_abs_err_deg": val_metrics["theta_neg_2_0p5_p95_abs_err_deg"],
        "val_theta_pos_0p5_2_p95_abs_err_deg": val_metrics["theta_pos_0p5_2_p95_abs_err_deg"],
        "val_theta_flat_abs_p95_deg": val_metrics["theta_flat_abs_p95_deg"],
        "val_theta_flat_bias_deg": val_metrics["theta_flat_bias_deg"],
        "val_theta_near_flat_abs_p95_deg": val_metrics["theta_near_flat_abs_p95_deg"],
        "val_theta_true_zero_abs_p95_deg": val_metrics["theta_true_zero_abs_p95_deg"],
        "val_flat_recall": val_metrics["flat_recall"],
        "val_stall_recall": val_metrics["stall_recall"],
        "val_slope_recall": val_metrics["slope_recall"],
    }


def _write_csv(path: Path, rows: Iterable[Dict[str, object]]) -> None:
    rows = list(rows)
    if not rows:
        return
    with path.open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)

```

```
1741:
1742: def _write_report(
1743:     path: Path,
1744:     args: argparse.Namespace,
1745:     cfg: ModernTCNConfig,
1746:     contract: Dict[str, object],
1747:     row: Dict[str, object],
1748:     test_metrics: Dict[str, object],
1749:     val_metrics: Dict[str, object],
1750:     train_seconds: float,
1751: ) -> None:
1752:     passed, failures = seed42_gate(test_metrics) if args.seed == 42 else (False, [])
1753:     with path.open("w", encoding="utf-8") as f:
1754:         f.write("# ModernTCN-small 第一阶段训练报告\n\n")
1755:         f.write("## 固定约束\n\n")
1756:         f.write(f"- dataset: `{contract['dataset_file']}`\n")
1757:         f.write(f"- vehicle: `{contract.get('vehicle_type', '')}`;
active=`{contract.get('active_drive_steer_wheels', '')}`;
passive=`{contract.get('passive_support_wheels', '')}`\n")
1758:         f.write(f"- feature_policy: `{contract.get('feature_policy', '')}`\n")
1759:         f.write(f"- label_time_policy: `{contract.get('label_time_policy', '')}`,
horizon_steps={contract.get('horizon_steps', 0)}\n")
1760:         f.write("- split: 使用 MAT 文件已有 run-level split，不重划分。\n")
1761:         f.write("- scaler: 使用 MAT 文件已有归一化后 X，不重新拟合。\n")
1762:         f.write(f"- confidence_policy: `{contract.get('confidence_policy', '')}`\n")
1763:         f.write(f"- input: `[batch, time={contract['seq_len']},
feature={contract['input_dim']}]`\n")
1764:         f.write("- output: `logits_main`, `logits_turn`, `theta_hat`\n\n")
1765:         f.write("## 配置\n\n")
1766:         f.write("```json\n")
1767:         f.write(json.dumps(cfg.to_dict(), indent=2, ensure_ascii=False))
1768:         f.write("\n```\n\n")
1769:         f.write("## 测试集指标\n\n")
1770:         f.write("| metric | value |\n|---|---:|\n")
1771:         for key in [
1772:             "acc_main",
1773:             "acc_turn",
1774:             "acc_turn_pure",
1775:             "acc_turn_transition",
1776:             "main_confidence_mean",
1777:             "main_low_conf_0p60_ratio",
1778:             "main_low_conf_0p70_ratio",
1779:             "turn_confidence_mean",
1780:             "turn_low_conf_0p60_ratio",
1781:             "turn_low_conf_0p70_ratio",
1782:             "turn_right_recall",
1783:             "turn_straight_recall",
1784:             "turn_left_recall",
1785:             "theta_mae_deg",
1786:             "theta_abs_le_10_p95_abs_err_deg",
1787:             "theta_neg_10_8_p95_abs_err_deg",
1788:             "theta_pos_8_10_p95_abs_err_deg",
1789:             "theta_abs_le_8_p95_abs_err_deg",
1790:             "theta_neg_8_6_p95_abs_err_deg",
1791:             "theta_pos_6_8_p95_abs_err_deg",
1792:             "theta_neg_2_0p5_p95_abs_err_deg",
1793:             "theta_pos_0p5_2_p95_abs_err_deg",
1794:             "theta_flat_abs_p95_deg",
1795:             "theta_flat_bias_deg",
1796:             "theta_near_flat_abs_p95_deg",
1797:             "theta_true_zero_abs_p95_deg",
1798:             "theta_near_flat_bias_deg",
1799:             "theta_flat_turn_abs_p95_deg",
1800:             "flat_recall",
```

```
            "stall_recall",
            "slope_recall",
            "uphill_recall",
            "downhill_recall",
        ]:
            f.write(f"| {key} | {float(row[key]):.4f} |\n")
        f.write(f"\n- best_epoch: {row['best_epoch']}\n")
        f.write(f"- train_seconds: {train_seconds:.1f}\n\n")
        f.write("## 置信度分桶\n\n")
        _write_confidence_bins(f, "main", test_metrics)
        _write_confidence_bins(f, "turn", test_metrics)
        if args.seed == 42:
            f.write("## seed42 进入三 seed 判定\n\n")
            f.write(f"- pass: `{int(passed)}`\n")
            if failures:
                for msg in failures:
                    f.write(f"- {msg}\n")
            else:
                f.write("- seed42 已满足进入 `[42, 73, 101]` 的最低门槛。\n")
        f.write("\n## 验证集最佳点\n\n")
        f.write("```json\n")
        f.write(json.dumps(val_metrics, indent=2, ensure_ascii=False))
        f.write("\n```\n")


def _write_confidence_bins(f, prefix: str, metrics: Dict[str, object]) -> None:
    f.write(f"### {prefix}\n\n")
    f.write("| confidence bin | n | error rate | mean confidence |\n|---|---:|---:|---:|\n")
    for row in metrics.get(f"{prefix}_confidence_bins", []):
        f.write(
            f"| {row['bin']} | {int(row['n'])} | "
            f"{float(row['error_rate']):.4f} | {float(row['mean_confidence']):.4f} |\n"
        )
    f.write("\n")


if __name__ == "__main__":
    main()

===== FILE: src/ModernTCN/run_modern_tcn_theta10_v2_multiseed.py =====
"""Run ModernTCN multi-seed training on the theta10 uniform V2 dataset.

This entry point is intentionally separate from the older phase1 runner:
it always trains every requested seed, uses the horizon=0 V2 dataset by
default, and writes one aggregate CSV/report after all seeds finish.
"""

from __future__ import annotations

import argparse
import csv
import math
from pathlib import Path
from statistics import mean, pstdev
from typing import Dict, Iterable, List

from modern_tcn_data import find_project_root
from train_modern_tcn import train_one_seed


```

```
1861: DEFAULT_DATASET = "data/tcn/ModernTCN_dataset_agv_dualsteer_theta10_uniform_conf_h0_v2.mat"
1862: DEFAULT_SEEDS = [11, 21, 42, 73, 101]
1863: KEY_METRICS = [
1864:     "acc_main",
1865:     "flat_recall",
1866:     "slope_recall",
1867:     "acc_turn",
1868:     "acc_turn_transition",
1869:     "turn_right_recall",
1870:     "turn_left_recall",
1871:     "theta_mae_deg",
1872:     "theta_abs_le_10_p95_abs_err_deg",
1873:     "theta_neg_10_8_p95_abs_err_deg",
1874:     "theta_pos_8_10_p95_abs_err_deg",
1875:     "theta_neg_8_6_p95_abs_err_deg",
1876:     "theta_pos_6_8_p95_abs_err_deg",
1877:     "theta_neg_2_0p5_p95_abs_err_deg",
1878:     "theta_pos_0p5_2_p95_abs_err_deg",
1879:     "theta_true_zero_abs_p95_deg",
1880:     "theta_flat_abs_p95_deg",
1881:     "theta_flat_bias_deg",
1882: ]
1883:
1884:
1885: def parse_args() -> argparse.Namespace:
1886:     p = argparse.ArgumentParser(description="ModernTCN theta10 V2 multi-seed training")
1887:     p.add_argument("--dataset-file", type=str, default=DEFAULT_DATASET)
1888:     p.add_argument("--seeds", type=int, nargs="+", default=DEFAULT_SEEDS)
1889:     p.add_argument("--run-tag-prefix", type=str, default="modern_tcn_theta10_uniform_h0_v2")
1890:     p.add_argument("--epochs", type=int, default=180)
1891:     p.add_argument("--batch-size", type=int, default=256)
1892:     p.add_argument("--lr", type=float, default=1e-3)
1893:     p.add_argument("--weight-decay", type=float, default=1e-4)
1894:     p.add_argument("--patience", type=int, default=35)
1895:     p.add_argument("--min-epochs", type=int, default=50)
1896:     p.add_argument("--channels", type=int, default=64)
1897:     p.add_argument("--blocks", type=int, default=5)
1898:     p.add_argument("--kernel-size", type=int, default=31)
1899:     p.add_argument(
1900:         "--temporal-padding",
1901:         type=str,
1902:         default="same",
1903:         choices=["same", "causal"],
1904:         help="Temporal convolution padding mode. 默认 same，causal 仅用于因果消融实验。",
1905:     )
1906:     p.add_argument("--dropout", type=float, default=0.15)
1907:     p.add_argument("--turn-head-source", type=str, default="full", choices=["full", "inputstats",
"kinematic_stats"])
1908:     p.add_argument("--lambda-turn", type=float, default=0.08)
1909:     p.add_argument("--lambda-theta", type=float, default=0.55)
1910:     p.add_argument("--lambda-theta-flat", type=float, default=0.12)
1911:     p.add_argument("--theta-flat-loss-mode", type=str, default="near_zero", choices=["near_zero",
"true_zero", "near_flat", "main_flat", "none"])
1912:     p.add_argument("--theta-flat-zero-tol-deg", type=float, default=0.3)
1913:     p.add_argument("--lambda-theta-near-flat", type=float, default=0.0)
1914:     p.add_argument("--theta-near-flat-deg", type=float, default=0.5)
1915:     p.add_argument("--lambda-theta-error-excess", type=float, default=0.05)
1916:     p.add_argument("--lambda-theta-flat-excess", type=float, default=0.0)
1917:     p.add_argument("--lambda-theta-near-flat-excess", type=float, default=0.0)
1918:     p.add_argument("--lambda-theta-true-zero-excess", type=float, default=0.10)
1919:     p.add_argument("--lambda-theta-active-excess", type=float, default=0.10)
1920:     p.add_argument("--lambda-theta-small-neg", type=float, default=0.0)
```

```
1921:     p.add_argument("--lambda-theta-small-neg-excess", type=float, default=0.0)
1922:     p.add_argument("--theta-excess-target-deg", type=float, default=1.0)
1923:     p.add_argument("--theta-flat-excess-target-deg", type=float, default=0.5)
1924:     p.add_argument("--theta-small-neg-min-deg", type=float, default=-4.0)
1925:     p.add_argument("--theta-small-neg-max-deg", type=float, default=-2.0)
1926:     p.add_argument("--theta-gate-mode", type=str, default="none", choices=["none",
"main_slope_prob"])
1927:     p.add_argument("--theta-gate-power", type=float, default=1.0)
1928:     p.add_argument("--theta-gate-floor", type=float, default=0.0)
1929:     p.add_argument("--theta-neg-weight", type=float, default=1.0)
1930:     p.add_argument("--theta-pos-weight", type=float, default=1.0)
1931:     p.add_argument("--turn-transition-weight", type=float, default=1.4)
1932:     p.add_argument("--turn-class-multipliers", type=float, nargs=3, default=[1.08, 1.00, 1.08])
1933:     p.add_argument("--select-turn-weight", type=float, default=0.30)
1934:     p.add_argument("--select-turn-transition-weight", type=float, default=1.20)
1935:     p.add_argument("--select-turn-transition-target", type=float, default=0.82)
1936:     p.add_argument("--select-turn-left-weight", type=float, default=0.0)
1937:     p.add_argument("--select-turn-left-target", type=float, default=0.88)
1938:     p.add_argument("--select-turn-lr-weight", type=float, default=0.20)
1939:     p.add_argument("--select-turn-lr-target", type=float, default=0.88)
1940:     p.add_argument("--select-theta-weight", type=float, default=0.30)
1941:     p.add_argument("--select-theta-ref-deg", type=float, default=2.0)
1942:     p.add_argument("--select-theta-p95-weight", type=float, default=0.80)
1943:     p.add_argument("--select-theta-p95-target-deg", type=float, default=1.20)
1944:     p.add_argument("--select-theta-flat-p95-weight", type=float, default=0.35)
1945:     p.add_argument("--select-theta-flat-p95-target-deg", type=float, default=0.70)
1946:     p.add_argument("--select-theta-near-flat-p95-weight", type=float, default=0.20)
1947:     p.add_argument("--select-theta-near-flat-p95-target-deg", type=float, default=0.70)
1948:     p.add_argument("--select-theta-true-zero-p95-weight", type=float, default=0.45)
1949:     p.add_argument("--select-theta-true-zero-p95-target-deg", type=float, default=0.50)
1950:     p.add_argument("--select-theta-extreme-p95-weight", type=float, default=0.60)
1951:     p.add_argument("--select-theta-extreme-p95-target-deg", type=float, default=1.20)
1952:     p.add_argument("--select-theta-edge-p95-weight", type=float, default=0.70)
1953:     p.add_argument("--select-theta-edge-p95-target-deg", type=float, default=1.50)
1954:     p.add_argument("--select-theta-small-nonzero-p95-weight", type=float, default=0.80)
1955:     p.add_argument("--select-theta-small-nonzero-p95-target-deg", type=float, default=1.00)
1956:     p.add_argument("--select-theta-flat-bias-weight", type=float, default=0.30)
1957:     p.add_argument("--select-theta-flat-bias-target-deg", type=float, default=0.15)
1958:     p.add_argument("--device", type=str, default="auto", choices=["auto", "cpu", "cuda"])
1959:     p.add_argument("--num-workers", type=int, default=0)
1960:     p.add_argument("--limit-train", type=int, default=0, help="Smoke-test only. Keep 0 for formal
training.")
1961:     p.add_argument("--limit-val", type=int, default=0, help="Smoke-test only. Keep 0 for formal
training.")
1962:     p.add_argument("--limit-test", type=int, default=0, help="Smoke-test only. Keep 0 for formal
training.")
1963:     p.add_argument("--dry-run", action="store_true", help="Check data/model wiring without
training.")
1964:     return p.parse_args()
1965:
1966:
1967: def main() -> None:
1968:     args = parse_args()
1969:     root = find_project_root()
1970:     dataset_file = _resolve_dataset(root, args.dataset_file)
1971:
1972:     print("[ModernTCN V2 multi-seed]")
1973:     print(f"  dataset: {dataset_file}")
1974:     print(f"  seeds: {args.seeds}")
1975:     print(f"  temporal_padding: {args.temporal_padding}")
1976:     print(f"  theta_gate_mode: {args.theta_gate_mode}")
1977:     print(f"  theta_flat_loss_mode: {args.theta_flat_loss_mode},
tol={args.theta_flat_zero_tol_deg:g} deg")
1978:
1979:     rows: List[Dict[str, str]] = []
1980:     for index, seed in enumerate(args.seeds, start=1):
```

```
1981:         print(f"\n[ModernTCN V2 multi-seed] seed {seed} ({index}/{len(args.seeds)})")
1982:         train_args = _build_train_args(args, seed, dataset_file)
1983:         result = train_one_seed(train_args)
1984:         if args.dry_run:
1985:             break
1986:         row = _read_single_row(Path(result["summary_csv"]))
1987:         row["run_tag"] = train_args.run_tag
1988:         row["checkpoint_file"] = result["checkpoint_file"]
1989:         rows.append(row)
1990:
1991:     if args.dry_run:
1992:         print("[ModernTCN V2 multi-seed] dry-run finished.")
1993:         return
1994:
1995:     out_dir = root / "results" / "modern_tcn"
1996:     summary_csv = out_dir / f"{args.run_tag_prefix}_multiseed_summary.csv"
1997:     report_file = out_dir / f"{args.run_tag_prefix}_multiseed_report.md"
1998:     _write_csv(summary_csv, rows)
1999:     _write_report(report_file, args, dataset_file, rows)
2000:     print("\n[ModernTCN V2 multi-seed] all seeds finished")
2001:     print(f"  summary: {summary_csv}")
2002:     print(f"  report: {report_file}")
2003:
2004:
2005: def _resolve_dataset(root: Path, dataset_file: str) -> Path:
2006:     path = Path(dataset_file)
2007:     if not path.is_absolute():
2008:         path = root / path
2009:     if not path.exists():
2010:         raise FileNotFoundError(f"Dataset not found: {path}")
2011:     return path
2012:
2013:
2014: def _build_train_args(args: argparse.Namespace, seed: int, dataset_file: Path) ->
argparse.Namespace:
2015:     return argparse.Namespace(
2016:         seed=seed,
2017:         dataset_file=str(dataset_file),
2018:         run_tag=f"{args.run_tag_prefix}_seed{seed}",
2019:         epochs=args.epochs,
2020:         batch_size=args.batch_size,
2021:         lr=args.lr,
2022:         weight_decay=args.weight_decay,
2023:         patience=args.patience,
2024:         min_epochs=args.min_epochs,
2025:         channels=args.channels,
2026:         blocks=args.blocks,
2027:         kernel_size=args.kernel_size,
2028:         temporal_padding=args.temporal_padding,
2029:         dropout=args.dropout,
2030:         turn_head_source=args.turn_head_source,
2031:         lambda_turn=args.lambda_turn,
2032:         lambda_theta=args.lambda_theta,
2033:         lambda_theta_flat=args.lambda_theta_flat,
2034:         theta_flat_loss_mode=args.theta_flat_loss_mode,
2035:         theta_flat_zero_tol_deg=args.theta_flat_zero_tol_deg,
2036:         lambda_theta_near_flat=args.lambda_theta_near_flat,
2037:         theta_near_flat_deg=args.theta_near_flat_deg,
2038:         lambda_theta_error_excess=args.lambda_theta_error_excess,
2039:         lambda_theta_flat_excess=args.lambda_theta_flat_excess,
2040:         lambda_theta_near_flat_excess=args.lambda_theta_near_flat_excess,
```

```
2041:         lambda_theta_true_zero_excess=args.lambda_theta_true_zero_excess,
2042:         lambda_theta_active_excess=args.lambda_theta_active_excess,
2043:         lambda_theta_small_neg=args.lambda_theta_small_neg,
2044:         lambda_theta_small_neg_excess=args.lambda_theta_small_neg_excess,
2045:         theta_excess_target_deg=args.theta_excess_target_deg,
2046:         theta_flat_excess_target_deg=args.theta_flat_excess_target_deg,
2047:         theta_small_neg_min_deg=args.theta_small_neg_min_deg,
2048:         theta_small_neg_max_deg=args.theta_small_neg_max_deg,
2049:         theta_gate_mode=args.theta_gate_mode,
2050:         theta_gate_power=args.theta_gate_power,
2051:         theta_gate_floor=args.theta_gate_floor,
2052:         theta_neg_weight=args.theta_neg_weight,
2053:         theta_pos_weight=args.theta_pos_weight,
2054:         turn_transition_weight=args.turn_transition_weight,
2055:         turn_class_multipliers=args.turn_class_multipliers,
2056:         select_turn_weight=args.select_turn_weight,
2057:         select_turn_transition_weight=args.select_turn_transition_weight,
2058:         select_turn_transition_target=args.select_turn_transition_target,
2059:         select_turn_left_weight=args.select_turn_left_weight,
2060:         select_turn_left_target=args.select_turn_left_target,
2061:         select_turn_lr_weight=args.select_turn_lr_weight,
2062:         select_turn_lr_target=args.select_turn_lr_target,
2063:         select_theta_weight=args.select_theta_weight,
2064:         select_theta_ref_deg=args.select_theta_ref_deg,
2065:         select_theta_p95_weight=args.select_theta_p95_weight,
2066:         select_theta_p95_target_deg=args.select_theta_p95_target_deg,
2067:         select_theta_flat_p95_weight=args.select_theta_flat_p95_weight,
2068:         select_theta_flat_p95_target_deg=args.select_theta_flat_p95_target_deg,
2069:         select_theta_near_flat_p95_weight=args.select_theta_near_flat_p95_weight,
2070:         select_theta_near_flat_p95_target_deg=args.select_theta_near_flat_p95_target_deg,
2071:         select_theta_true_zero_p95_weight=args.select_theta_true_zero_p95_weight,
2072:         select_theta_true_zero_p95_target_deg=args.select_theta_true_zero_p95_target_deg,
2073:         select_theta_extreme_p95_weight=args.select_theta_extreme_p95_weight,
2074:         select_theta_extreme_p95_target_deg=args.select_theta_extreme_p95_target_deg,
2075:         select_theta_edge_p95_weight=args.select_theta_edge_p95_weight,
2076:         select_theta_edge_p95_target_deg=args.select_theta_edge_p95_target_deg,
2077:         select_theta_small_nonzero_p95_weight=args.select_theta_small_nonzero_p95_weight,
2078:         select_theta_small_nonzero_p95_target_deg=args.select_theta_small_nonzero_p95_target_deg,
2079:         select_theta_flat_bias_weight=args.select_theta_flat_bias_weight,
2080:         select_theta_flat_bias_target_deg=args.select_theta_flat_bias_target_deg,
2081:         device=args.device,
2082:         num_workers=args.num_workers,
2083:         limit_train=args.limit_train,
2084:         limit_val=args.limit_val,
2085:         limit_test=args.limit_test,
2086:         dry_run=args.dry_run,
2087:     )
2088: 
2089: 
2090: def _read_single_row(path: Path) -> Dict[str, str]:
2091:     with path.open("r", newline="", encoding="utf-8") as f:
2092:         rows = list(csv.DictReader(f))
2093:     if len(rows) != 1:
2094:         raise RuntimeError(f"Expected one summary row in {path}, got {len(rows)}")
2095:     return rows[0]
2096: 
2097: 
2098: def _write_csv(path: Path, rows: Iterable[Dict[str, str]]) -> None:
2099:     rows = list(rows)
2100:     if not rows:
```

```
2101:         return
2102:     fieldnames: List[str] = []
2103:     for row in rows:
2104:         for key in row.keys():
2105:             if key not in fieldnames:
2106:                 fieldnames.append(key)
2107:     with path.open("w", newline="", encoding="utf-8") as f:
2108:         writer = csv.DictWriter(f, fieldnames=fieldnames)
2109:         writer.writeheader()
2110:         writer.writerows(rows)
2111:
2112:
2113: def _write_report(path: Path, args: argparse.Namespace, dataset_file: Path, rows: List[Dict[str,
str]]) -> None:
2114:     with path.open("w", encoding="utf-8") as f:
2115:         f.write("# ModernTCN theta10 V2 multi-seed report\n\n")
2116:         f.write(f"- dataset: `{dataset_file}`\n")
2117:         f.write(f"- seeds: `{args.seeds}`\n")
2118:         f.write(f"- epochs: `{args.epochs}`, batch_size: `{args.batch_size}`\n")
2119:         f.write(f"- temporal_padding: `{args.temporal_padding}`\n")
2120:         f.write(f"- theta_gate_mode: `{args.theta_gate_mode}`\n")
2121:         f.write(f"- theta_flat_loss_mode: `{args.theta_flat_loss_mode}`, zero_tol_deg:
`{args.theta_flat_zero_tol_deg}`\n")
2122:         f.write(f"- lambda_theta: `{args.lambda_theta}`, lambda_turn: `{args.lambda_turn}`\n\n")
2123:         f.write(f"- turn_lr_target: `{args.select_turn_lr_target}`, turn_lr_weight:
`{args.select_turn_lr_weight}`\n\n")
2124:         f.write("## Aggregate\n\n")
2125:         f.write("| metric | mean | std |\n|---|---:|---:|\n")
2126:         for metric in KEY_METRICS:
2127:             values = [_to_float(row.get(metric, "")) for row in rows]
2128:             values = [x for x in values if math.isfinite(x)]
2129:             if not values:
2130:                 continue
2131:             std_value = pstdev(values) if len(values) > 1 else 0.0
2132:             f.write(f"| {metric} | {mean(values):.4f} | {std_value:.4f} |\n")
2133:         f.write("\n## Per seed\n\n")
2134:         f.write("| seed | acc_main | acc_turn_transition | turn_L/R | theta_mae | theta_10_p95 |
edge_neg_p95 | edge_pos_p95 | checkpoint |\n")
2135:         f.write("|---:|---:|---:|---:|---:|---:|---:|---:|---|\n")
2136:         for row in rows:
2137:             turn_lr = _turn_lr_ratio(row)
2138:             f.write(
2139:                 f"| {int(float(row['seed']))} | {_fmt(row, 'acc_main')} | {_fmt(row,
'acc_turn_transition')} | "
2140:                 f"{turn_lr:.4f} | {_fmt(row, 'theta_mae_deg')} | {_fmt(row,
'theta_abs_le_10_p95_abs_err_deg')} | "
2141:                 f"{_fmt(row, 'theta_neg_10_8_p95_abs_err_deg')} | {_fmt(row,
'theta_pos_8_10_p95_abs_err_deg')} | "
2142:                 f"`{row.get('checkpoint_file', '')}` |\n"
2143:             )
2144:
2145:
2146: def _to_float(value: str) -> float:
2147:     try:
2148:         return float(value)
2149:     except (TypeError, ValueError):
2150:         return float("nan")
2151:
2152:
2153: def _fmt(row: Dict[str, str], key: str) -> str:
2154:     value = _to_float(row.get(key, ""))
2155:     return "nan" if not math.isfinite(value) else f"{value:.4f}"
2156:
2157:
2158: def _turn_lr_ratio(row: Dict[str, str]) -> float:
2159:     right = _to_float(row.get("turn_right_recall", ""))
```

```
2160:     left = _to_float(row.get("turn_left_recall", ""))
```

```
2161:     if not math.isfinite(right) or not math.isfinite(left) or max(right, left) <= 0:
2162:         return float("nan")
2163:     return min(right, left) / max(right, left)
2164:
2165:
2166: if __name__ == "__main__":
2167:     main()
2168:
2169: ===== FILE: src/ModernTCN/export_modern_tcn_onnx.py =====
2170: """将训练好的 ModernTCN checkpoint 导出为 ONNX，并保存 PyTorch 参考输出。
2171:
2172: 导出前强制 `model.eval()`，固定输入形状 `[batch, 128, 19]`，不启用
2173: dynamic axes。第一版默认 opset=17；如果 MATLAB 导入报算子不支持，再按
2174: 错误信息降级或替换模型算子。
2175: """
2176:
2177: from __future__ import annotations
2178:
2179: import argparse
2180: import importlib.util
2181: import json
2182: from pathlib import Path
2183:
2184: import numpy as np
2185: import torch
2186: from scipy.io import savemat
2187:
2188: from modern_tcn_data import find_project_root, load_modern_tcn_dataset
2189: from modern_tcn_model import build_model_from_checkpoint_dict
2190:
2191:
2192: def parse_args() -> argparse.Namespace:
2193:     p = argparse.ArgumentParser(description="导出 ModernTCN ONNX")
2194:     p.add_argument("--checkpoint", type=str, required=True)
2195:     p.add_argument("--onnx-file", type=str, default="")
2196:     p.add_argument("--opset", type=int, default=17)
2197:     p.add_argument("--sample-count", type=int, default=16)
2198:     return p.parse_args()
2199:
2200:
2201: def main() -> None:
2202:     args = parse_args()
2203:     if importlib.util.find_spec("onnx") is None:
2204:         raise SystemExit("缺少 onnx。请先运行：python -m pip install onnx")
2205:     if importlib.util.find_spec("onnxscript") is None:
2206:         raise SystemExit("缺少 onnxscript。请先运行：python -m pip install onnxscript")
2207:     root = find_project_root()
2208:     checkpoint = Path(args.checkpoint)
2209:     if not checkpoint.exists():
2210:         raise FileNotFoundError(f"找不到 checkpoint：{checkpoint}")
2211:
2212:     # checkpoint 内含普通 Python dict/list，显式关闭 weights_only 以兼容新版 PyTorch 默认策略。
2213:     ckpt = torch.load(checkpoint, map_location="cpu", weights_only=False)
2214:     model = build_model_from_checkpoint_dict(ckpt)
2215:     model.eval()
2216:
2217:     out_dir = checkpoint.parent
2218:     onnx_file = Path(args.onnx_file) if args.onnx_file else out_dir /
checkpoint.name.replace(".pt", ".onnx")
2219:     sample_file = out_dir / checkpoint.name.replace(".pt", "_pytorch_reference.mat")
2220:     meta_file = out_dir / checkpoint.name.replace(".pt", "_onnx_export.json")
```

```
2221:
2222:     dummy = torch.zeros(1, ckpt["model_config"]["seq_len"], ckpt["model_config"]["input_dim"],
dtype=torch.float32)
2223:     with torch.no_grad():
2224:         torch.onnx.export(
2225:             model,
2226:             dummy,
2227:             onnx_file,
2228:             export_params=True,
2229:             opset_version=args.opset,
2230:             do_constant_folding=True,
2231:             input_names=["input_window"],
2232:             output_names=["logits_main", "logits_turn", "theta_hat"],
2233:             dynamo=False,
2234:         )
2235:
2236:     # 保存一组 test 窗口的 PyTorch 输出，供 ONNXRuntime 和 MATLAB 做三方一致性。
2237:     data = load_modern_tcn_dataset(Path(ckpt["contract"]["dataset_file"]))
2238:     X_sample = data["test"].X[: args.sample_count].astype(np.float32)
2239:     with torch.no_grad():
2240:         lm, lt, th = model(torch.from_numpy(X_sample).float())
2241:     savemat(
2242:         sample_file,
2243:         {
2244:             "X_sample": X_sample,
2245:             "logits_main_pytorch": lm.numpy().astype(np.float32),
2246:             "logits_turn_pytorch": lt.numpy().astype(np.float32),
2247:             "theta_hat_pytorch": th.numpy().astype(np.float32),
2248:         },
2249:     )
2250:
2251:     meta = {
2252:         "checkpoint": str(checkpoint),
2253:         "onnx_file": str(onnx_file),
2254:         "sample_file": str(sample_file),
2255:         "opset": args.opset,
2256:         "input_shape": [1, ckpt["model_config"]["seq_len"], ckpt["model_config"]["input_dim"]],
2257:         "output_names": ["logits_main", "logits_turn", "theta_hat"],
2258:         "note": "导出前已调用 model.eval()；dropout 在 ONNX 推理中关闭；第一版固定 batch=1，离线批量检
查脚本会逐窗口推理；当前默认使用 legacy TorchScript ONNX exporter 以获得更稳定的 PyTorch/ONNXRuntime 数值一致
性。",
2259:     }
2260:     meta_file.write_text(json.dumps(meta, indent=2, ensure_ascii=False), encoding="utf-8")
2261:     print("[ModernTCN ONNX] 导出完成")
2262:     print(f"  onnx: {onnx_file}")
2263:     print(f"  sample: {sample_file}")
2264:     print(f"  meta: {meta_file}")
2265:
2266:
2267: if __name__ == "__main__":
2268:     main()
2269:
2270: ===== FILE: src/ModernTCN/check_onnxruntime_consistency.py =====
2271: """比较 PyTorch 参考输出和 ONNXRuntime 输出。
2272:
2273: 该脚本是三方一致性检查的第二步：
2274:     1. PyTorch 输出：由 export_modern_tcn_onnx.py 保存到 MAT。
2275:     2. ONNXRuntime 输出：本脚本读取同一 ONNX 和同一 X_sample。
2276:     3. MATLAB 输出：由 ModernTCN_check_matlab_onnx.m 完成。
2277: """
2278:
2279: from __future__ import annotations
2280:
```

```
import argparse
import json
from pathlib import Path
from typing import Dict

import numpy as np
from scipy.io import loadmat


def parse_args() -> argparse.Namespace:
    p = argparse.ArgumentParser(description="ModernTCN ONNXRuntime 一致性检查")
    p.add_argument("--onnx-file", type=str, required=True)
    p.add_argument("--sample-file", type=str, required=True)
    p.add_argument("--max-abs-tol", type=float, default=1e-4)
    p.add_argument("--mean-abs-tol", type=float, default=1e-5)
    return p.parse_args()


def main() -> Dict[str, object]:
    args = parse_args()
    try:
        import onnxruntime as ort
    except ImportError as exc:
        raise SystemExit("缺少 onnxruntime。请先运行：python -m pip install onnx onnxruntime") from
exc

    onnx_file = Path(args.onnx_file)
    sample_file = Path(args.sample_file)
    sample = loadmat(sample_file)
    X = sample["X_sample"].astype(np.float32)

    session = ort.InferenceSession(str(onnx_file), providers=["CPUExecutionProvider"])
    outputs = _run_onnx(session, X)
    refs = [sample["logits_main_pytorch"], sample["logits_turn_pytorch"],
sample["theta_hat_pytorch"]]
    names = ["logits_main", "logits_turn", "theta_hat"]

    result = {
        "onnx_file": str(onnx_file),
        "sample_file": str(sample_file),
        "threshold": {"max_abs_error": args.max_abs_tol, "mean_abs_error": args.mean_abs_tol},
        "outputs": {},
    }
    passed = True
    for name, out, ref in zip(names, outputs, refs):
        diff = np.abs(np.asarray(out) - np.asarray(ref))
        max_abs = float(diff.max())
        mean_abs = float(diff.mean())
        ok = max_abs <= args.max_abs_tol and mean_abs <= args.mean_abs_tol
        passed = passed and ok
        result["outputs"][name] = {"max_abs_error": max_abs, "mean_abs_error": mean_abs, "pass":
ok}
    result["pass"] = passed

    out_json = onnx_file.with_name(onnx_file.stem + "_onnxruntime_consistency.json")
    out_md = onnx_file.with_name(onnx_file.stem + "_onnxruntime_consistency.md")
    out_json.write_text(json.dumps(result, indent=2, ensure_ascii=False), encoding="utf-8")
    _write_md(out_md, result)
    print(f"[ModernTCN ONNXRuntime] pass={int(passed)}")
    print(f"  json: {out_json}")
    print(f"  report: {out_md}")
    return result

```

```
2341:
2342: def _run_onnx(session, X: np.ndarray):
2343:     """兼容固定 batch=1 的 ONNX 导出。
2344:
2345:     第一阶段为了 MATLAB/Simulink 单窗口部署更稳，ONNX 输入固定为
2346:     [1, 128, 19]。一致性检查使用 16 个测试窗口时，需要逐窗口推理后拼接。
2347:     如果后续改为 dynamic batch，这里也能直接一次性推理。
2348:     """
2349:
2350:     input_shape = session.get_inputs()[0].shape
2351:     batch_dim = input_shape[0]
2352:     output_names = ["logits_main", "logits_turn", "theta_hat"]
2353:     if isinstance(batch_dim, int) and batch_dim == 1 and X.shape[0] != 1:
2354:         chunks = []
2355:         for i in range(X.shape[0]):
2356:             yi = session.run(output_names, {"input_window": X[i : i + 1]})
2357:             chunks.append(yi)
2358:         return [np.concatenate([c[j] for c in chunks], axis=0) for j in range(3)]
2359:     return session.run(output_names, {"input_window": X})
2360:
2361:
2362: def _write_md(path: Path, result: Dict[str, object]) -> None:
2363:     with path.open("w", encoding="utf-8") as f:
2364:         f.write("# ModernTCN ONNXRuntime 一致性检查\n\n")
2365:         f.write(f"- onnx: `{result['onnx_file']}`\n")
2366:         f.write(f"- sample: `{result['sample_file']}`\n")
2367:         f.write(f"- pass: `{int(result['pass'])}`\n\n")
2368:         f.write("| output | max abs error | mean abs error | pass |\n")
2369:         f.write("|---|---:|---:|---:|\n")
2370:         for name, row in result["outputs"].items():
2371:             f.write(f"| {name} | {row['max_abs_error']:.6g} | {row['mean_abs_error']:.6g} |
{int(row['pass'])} |\n")
2372:
2373:
2374: if __name__ == "__main__":
2375:     main()
2376:
2377: ===== FILE: src/ModernTCN/ModernTCN_default_config.m =====
2378: function cfg = ModernTCN_default_config(root)
2379: %MODERNTCN_DEFAULT_CONFIG 当前冻结的 ModernTCN 部署配置。
2380: %
2381: % 这里只保存当前闭环仿真使用的默认模型。需要临时测试其他 checkpoint 时，仍可在调用脚本中传入 cfg
2382: % 覆盖 seed、dataset_file、run_tag 或 onnx_file。
2383:
2384: if nargin < 1 || isempty(root)
2385:     root = local_project_root();
2386: end
2387:
2388: cfg = struct();
2389: cfg.seed = 21;
2390: cfg.run_tag = 'modern_tcn_theta10_uniform_h0_v2_seed21';
2391: cfg.dataset_file = fullfile(root, 'data', 'tcn', ...
2392:     'ModernTCN_dataset_agv_dualsteer_theta10_uniform_conf_h0_v2.mat');
2393: cfg.raw_train_file = fullfile(root, 'data', 'tcn', ...
2394:     'ModernTCN_train_data_agv_dualsteer_theta10_uniform_conf_h0_v2.mat');
2395: cfg.onnx_file = fullfile(root, 'results', 'modern_tcn', cfg.run_tag, ...
2396:     'modern_tcn_seed21.onnx');
2397: cfg.reference_dir = fullfile(root, 'results', 'modern_tcn', cfg.run_tag);
2398: cfg.pytorch_reference_file = fullfile(root, 'results', 'modern_tcn', cfg.run_tag, ...
2399:     'modern_tcn_seed21_pytorch_reference.mat');
2400:
```

```
% Deployment-side theta conditioning for closed-loop scheduling. A concrete
% checkpoint must set these fields after the next dataset/model is selected.
cfg.theta_output_gain = 1.0;
cfg.theta_abs_limit = deg2rad(12.0);
cfg.theta_rate_limit = deg2rad(5.0);
cfg.theta_mpc_deadzone = deg2rad(2.0);
end

function root = local_project_root()
if exist('project_root', 'file') == 2
    root = project_root();
else
    root = pwd;
end
end

===== FILE: src/ModernTCN/ModernTCN_load_predictor.m =====
function predictor = ModernTCN_load_predictor(seed, onnx_file)
%MODERNTCN_LOAD_PREDICTOR 加载一个 ModernTCN ONNX 模型，返回在线推理句柄。
%
% 功能说明：
%   本函数是后续 Simulink 接入前的 MATLAB 在线推理入口。它只负责把指定
%   seed 的 ONNX 文件导入为 dlnetwork，并记录标签映射、输入窗口尺寸等信息。
%   真正的单窗口推理由 ModernTCN_predict_window 完成。
%
% 用法：
%   init_project;
%   addpath(fullfile(project_root(), 'src', 'ModernTCN'));
%   predictor = ModernTCN_load_predictor();
%
% 输入：
%   seed      : 可选，默认 21。默认定位当前推荐的 V4 theta-head B 候选。
%   onnx_file : 可选，自定义 ONNX 路径。为空时按 seed 自动定位。
%
% 输出：
%   predictor : 结构体，包含 net、seed、onnx_file、input_size 和标签映射。

root = local_project_root();
default_cfg = ModernTCN_default_config(root);
if nargin < 1 || isempty(seed)
    seed = default_cfg.seed;
end

if nargin < 2 || isempty(onnx_file)
    if seed == default_cfg.seed
        onnx_file = default_cfg.onnx_file;
    else
        onnx_file = fullfile(root, 'results', 'modern_tcn', default_cfg.run_tag, ...
            sprintf('modern_tcn_seed%d.onnx', seed));
    end
end

if exist(onnx_file, 'file') ~= 2
    error('ModernTCN:MissingONNX', '找不到 ONNX 文件: %s', onnx_file);
end

% MATLAB ONNX importer 会为部分算子生成 custom layer。这里把生成位置固定
% 在 src/ModernTCN/generated_layers，避免污染项目根目录。
layer_root = fullfile(root, 'src', 'ModernTCN', 'generated_layers');
if exist(layer_root, 'dir') ~= 7
```

```
2461:     mkdir(layer_root);
2462: end
2463: addpath(layer_root);
2464:
2465: old_dir = pwd;
2466: cleanup = onCleanup(@() cd(old_dir));
2467: cd(layer_root);
2468: net = importNetworkFromONNX(onnx_file, Namespace=local_onnx_namespace(onnx_file));
2469:
2470: predictor = struct();
2471: predictor.seed = seed;
2472: predictor.onnx_file = string(onnx_file);
2473: predictor.layer_root = string(layer_root);
2474: predictor.net = net;
2475: predictor.input_size = [128 19];              % [time, feature]
2476: predictor.onnx_input_size = [1 128 19];       % [batch, time, feature]
2477: predictor.main_labels = [1 2 3];              % 1=flat, 2=stall, 3=slope
2478: predictor.turn_labels = [-1 0 1];             % -1=right, 0=straight, 1=left
2479: predictor.theta_unit = "rad";
2480: predictor.theta_output_unit = "deg";
2481: end
2482:
2483: function root = local_project_root()
2484: if exist('project_root', 'file') == 2
2485:     root = project_root();
2486: else
2487:     root = pwd;
2488: end
2489: end
2490:
2491: function namespace = local_onnx_namespace(onnx_file)
2492: if contains(lower(char(onnx_file)), 'causal')
2493:     namespace = "modern_tcn_causal_onnx_layers";
2494: else
2495:     namespace = "modern_tcn_onnx_layers";
2496: end
2497: end
2498:
2499: ===== FILE: src/ModernTCN/ModernTCN_predict_window.m =====
2500: function output = ModernTCN_predict_window(predictor, X_window)
2501: %MODERNTCN_PREDICT_WINDOW 使用已加载的 ModernTCN 对单个时间窗口做在线推理。
2502: %
2503: % 功能说明：
2504: %   输入一个已经归一化、特征顺序与训练数据一致的 128x19 窗口，输出主工况、
2505: %   转弯状态、坡度角预测值和对应概率。该函数只做推理和后处理，不做 scaler
2506: %   归一化，也不改变特征顺序。
2507: %
2508: % 输入格式：
2509: %   X_window 可以是 [128,19] 或 [1,128,19]。
2510: %   [128,19] = [time, feature]，这是后续 MATLAB/Simulink wrapper 推荐格式。
2511: %
2512: % 输出标签：
2513: %   main_state = 1/2/3，分别对应 flat/stall/slope。
2514: %   turn_state = -1/0/1，分别对应 right/straight/left。
2515:
2516: if nargin < 2
2517:     error('ModernTCN:MissingInput', '需要 predictor 和 X_window 两个输入。');
2518: end
2519: if ~isstruct(predictor) || ~isfield(predictor, 'net')
2520:     error('ModernTCN:InvalidPredictor', 'predictor 必须来自 ModernTCN_load_predictor。');
```

```
2521: end
2522:
2523: X = local_prepare_single_window(X_window, predictor.input_size);
2524: [logits_main, logits_turn, theta_hat_rad] = local_predict_one(predictor.net, X);
2525:
2526: main_prob = local_softmax(logits_main);
2527: turn_prob = local_softmax(logits_turn);
2528:
2529: [main_confidence, main_idx] = max(main_prob, [], 2);
2530: [turn_confidence, turn_idx] = max(turn_prob, [], 2);
2531:
2532: output = struct();
2533: output.seed = predictor.seed;
2534: output.main_state = predictor.main_labels(main_idx);
2535: output.turn_state = predictor.turn_labels(turn_idx);
2536: output.theta_hat_rad = double(theta_hat_rad(1));
2537: output.theta_hat_deg = rad2deg(double(theta_hat_rad(1)));
2538: output.main_prob = main_prob(1,:);
2539: output.turn_prob = turn_prob(1,:);
2540: output.main_confidence = main_confidence(1);
2541: output.turn_confidence = turn_confidence(1);
2542: output.logits_main = logits_main(1,:);
2543: output.logits_turn = logits_turn(1,:);
2544: end
2545:
2546: function X = local_prepare_single_window(X_window, expected_size)
2547: % 统一转换成 ONNX 需要的 [batch,time,feature] = [1,128,19]。
2548: X = single(X_window);
2549: sz = size(X);
2550:
2551: if ismatrix(X)
2552:     if isequal(sz, expected_size)
2553:         X = reshape(X, [1 expected_size]);
2554:     elseif isequal(sz, fliplr(expected_size))
2555:         error(['ModernTCN:WrongWindowShape'], ...
2556:             ['输入尺寸是 [%d,%d]，看起来像 [feature,time]。', ...
2557:              '请转置成 [time,feature] = [128,19] 后再调用。'], sz(1), sz(2));
2558:     else
2559:         error('ModernTCN:WrongWindowShape', ...
2560:             '输入窗口必须是 [128,19] 或 [1,128,19]，当前尺寸是 %s。', mat2str(sz));
2561:     end
2562: elseif ndims(X) == 3
2563:     if ~isequal(sz, [1 expected_size])
2564:         error('ModernTCN:WrongWindowShape', ...
2565:             '三维输入必须是 [1,128,19]，当前尺寸是 %s。', mat2str(sz));
2566:     end
2567: else
2568:     error('ModernTCN:WrongWindowShape', ...
2569:         '输入窗口维度不支持，必须是二维或三维数组。');
2570: end
2571: end
2572:
2573: function [logits_main, logits_turn, theta_hat] = local_predict_one(net, X)
2574: % 兼容不同 MATLAB ONNX importer 对 [batch,time,feature] 的解释。
2575: try
2576:     [logits_main, logits_turn, theta_hat] = predict(net, X);
2577: catch
2578:     try
2579:         [logits_main, logits_turn, theta_hat] = predict(net, dlarray(X, "BTC"));
2580:     catch
```

```
2581:         % 如果 importer 按 [feature,time,batch] 读取，则转为 CTB。
2582:         Xctb = permute(X, [3 2 1]);
2583:         [logits_main, logits_turn, theta_hat] = predict(net, dlarray(Xctb, "CTB"));
2584:     end
2585: end
2586:
2587: logits_main = local_to_batch_first(local_extract(logits_main), 3);
2588: logits_turn = local_to_batch_first(local_extract(logits_turn), 3);
2589: theta_hat = local_to_batch_first(local_extract(theta_hat), 1);
2590: end
2591:
2592: function A = local_extract(A)
2593: if isa(A, 'dlarray')
2594:     A = gather(extractdata(A));
2595: else
2596:     A = gather(A);
2597: end
2598: end
2599:
2600: function A = local_to_batch_first(A, width)
2601: A = squeeze(A);
2602: if size(A, 2) ~= width && size(A, 1) == width
2603:     A = A.';
2604: end
2605: if width == 1
2606:     A = reshape(A, [], 1);
2607: end
2608: A = single(A);
2609: end
2610:
2611: function P = local_softmax(Z)
2612: % 数值稳定 softmax，避免依赖不同 MATLAB 版本中的同名函数行为。
2613: Z = single(Z);
2614: Z = Z - max(Z, [], 2);
2615: E = exp(Z);
2616: P = E ./ sum(E, 2);
2617: P = single(P);
2618: end
2619:
2620: ===== FILE: src/ModernTCN/ModernTCN_online_step.m =====
2621: function out = ModernTCN_online_step(y_raw, reset, seed, request_predict)
2622: %MODERNTCN_ONLINE_STEP 从单帧 y_raw 在线更新 ModernTCN 状态并按需推理。
2623: %
2624: % 功能说明：
2625: %   该函数是 ModernTCN 接入 Simulink 前的核心在线封装。它的输入不是
2626: %   已经归一化的 [128,19] 窗口，而是仿真模型每个采样周期输出的一帧
2627: %   y_raw。函数内部会：
2628: %     1. 从 y_raw 提取与当前推荐 ModernTCN 数据集一致的 19 维特征；
2629: %     2. 使用数据集内保存的 TCN scaler 做归一化；
2630: %     3. 维护 128 步滑动窗口；
2631: %     4. 调用当前推荐 ModernTCN ONNX predictor，输出分类、置信度和 theta_hat。
2632: %
2633: % 输入：
2634: %   y_raw           : 当前时刻 AGV 输出，至少 18 维；闭环模型中实际为 [34,1]。
2635: %   reset           : 非 0 时重置在线状态。
2636: %   seed            : 可选，默认 21。
2637: %   request_predict : 可选。
2638: %                     []  = 按 cfg.infer_period_steps 自动决定是否推理；
2639: %                     true  = 本步强制推理；
2640: %                     false = 本步只更新滑窗，不调用 ONNX。
```

```
%
% 输出：
%   out : 结构体，字段包括 label_main、label_turn、theta_hat_rad、conf_main、
%         buffer_count、ready、did_predict 等。函数也会把最近一次输出写入
%         base workspace 的 modern_tcn_online_last_out，便于 Simulink 薄封装读取。

if nargin < 1
    y_raw = [];
end
if nargin < 2 || isempty(reset)
    reset = 0;
end
if nargin < 3 || isempty(seed)
    default_cfg = ModernTCN_default_config(local_project_root());
    seed = default_cfg.seed;
end
if nargin < 4
    request_predict = [];
end

persistent state

if reset ~= 0 || isempty(state) || ~isfield(state, 'seed') || state.seed ~= seed
    state = local_init_state(seed);
end

out = local_default_output(state);

if isempty(y_raw)
    local_publish_output(out);
    return;
end
if numel(y_raw) < 18
    out.debug.warning = "y_raw 维度不足，至少需要前 18 维。";
    local_publish_output(out);
    return;
end

[feature_raw, state] = local_extract_feature(double(y_raw(:)), state);
feature_norm = (feature_raw - state.scaler_mean) ./ (state.scaler_std + state.eps);

% 滚动维护 [128,19] 归一化窗口。模型训练时使用的就是这个归一化窗口。
state.buffer(1:end-1, :) = state.buffer(2:end, :);
state.buffer(end, :) = single(feature_norm);
state.buffer_count = min(state.buffer_count + 1, state.seq_len);
state.step = state.step + 1;

out = local_default_output(state);
out.feature_raw = feature_raw;
out.feature_norm = feature_norm;

if state.buffer_count < state.seq_len
    out.debug.warning = "窗口未满，保持默认输出。";
    state.last_out = out;
    local_publish_output(out);
    return;
end

do_predict = local_should_predict(state, request_predict);
if do_predict
```

```
2701:     pred = ModernTCN_predict_window(state.predictor, state.buffer);
2702:     out = local_output_from_prediction(state, pred);
2703:     out.feature_raw = feature_raw;
2704:     out.feature_norm = feature_norm;
2705:     out.did_predict = true;
2706:     state.last_out = out;
2707: else
2708:     out = state.last_out;
2709:     out.step = state.step;
2710:     out.buffer_count = state.buffer_count;
2711:     out.ready = true;
2712:     out.did_predict = false;
2713: end
2714:
2715: local_publish_output(out);
2716: end
2717:
2718: function state = local_init_state(seed)
2719: % 初始化在线状态。注意：这里读取的是 TCN/ModernTCN 数据集 scaler，不是 GRU scaler。
2720: root = local_project_root();
2721: default_cfg = ModernTCN_default_config(root);
2722: dataset_file = default_cfg.dataset_file;
2723: if exist(dataset_file, 'file') ~= 2
2724:     error('ModernTCN:MissingDataset', '找不到数据集: %s', dataset_file);
2725: end
2726:
2727: S = load(dataset_file, 'dataset');
2728: dataset = S.dataset;
2729: cfg = local_read_sim_cfg(seed);
2730:
2731: params = parameters();
2732: state = struct();
2733: state.seed = seed;
2734: state.params = params;
2735: state.Ts = dataset.meta.Ts;
2736: state.seq_len = dataset.meta.seq_len;
2737: state.feat_dim = numel(dataset.scaler.mean);
2738: state.buffer = zeros(state.seq_len, state.feat_dim, 'single');
2739: state.buffer_count = 0;
2740: state.step = 0;
2741: state.eps = 1e-8;
2742:
2743: state.scaler_mean = double(dataset.scaler.mean(:).');
2744: state.scaler_std = double(dataset.scaler.std(:).');
2745: state.tau_accel_lp = dataset.scaler.tau_accel_lp;
2746: state.tau_diff = dataset.scaler.tau_diff;
2747: state.tau_pitch = dataset.scaler.tau_pitch;
2748: state.alpha_diff = state.Ts / (state.tau_diff + state.Ts);
2749: state.alpha_accel = state.Ts / (state.tau_accel_lp + state.Ts);
2750: state.lambda_pitch = exp(-state.Ts / state.tau_pitch);
2751:
2752: state.has_feature_prev = false;
2753: state.v_hat_prev = 0.0;
2754: state.dv_hat_dt_prev = 0.0;
2755: state.accel_x_lp_prev = 0.0;
2756: state.pitch_angle_est_prev = 0.0;
2757:
2758: state.infer_period_steps = cfg.infer_period_steps;
2759: state.predictor = ModernTCN_load_predictor(seed);
2760: state.last_out = local_default_output(state);
```

```
2761: end
2762:
2763: function cfg = local_read_sim_cfg(seed)
2764: % 允许用户在 base workspace 里用 modern_tcn_sim_cfg 覆写仿真配置。
2765: cfg = struct();
2766: cfg.seed = seed;
2767: cfg.infer_period_steps = 5;  % 默认每 0.05s 推理一次，降低闭环仿真开销。
2768: try
2769:     if evalin('base', 'exist(''modern_tcn_sim_cfg'', ''var'') == 1')
2770:         user_cfg = evalin('base', 'modern_tcn_sim_cfg');
2771:         if isstruct(user_cfg)
2772:             if isfield(user_cfg, 'infer_period_steps') && ~isempty(user_cfg.infer_period_steps)
2773:                 cfg.infer_period_steps = max(1, round(double(user_cfg.infer_period_steps)));
2774:             end
2775:         end
2776:     end
2777: catch
2778:     % base workspace 不可用时保留默认配置。
2779: end
2780: end
2781:
2782: function [feature_raw, state] = local_extract_feature(y_raw, state)
2783: % 复刻 TCN_prepare_dataset 中的 GRU_compatible_observable_19 特征契约。
2784: p = state.params;
2785: r = p.wheel_radius;
2786: W = p.W;
2787:
2788: accel_x = y_raw(9);
2789: gyro_y = y_raw(10);
2790: gyro_z = y_raw(11);
2791: I_lf = y_raw(12);
2792: I_rr = y_raw(13);
2793: omega_wheel_lf = y_raw(17);
2794: omega_wheel_rr = y_raw(18);
2795: delta_lf = y_raw(6);
2796: delta_rr = y_raw(7);
2797:
2798: v_hat = r * (omega_wheel_lf + omega_wheel_rr) / 2;
2799: if ~state.has_feature_prev
2800:     % 离线数据预处理在每个 run 的第一个有效样本处使用 dv_raw=0、
2801:     % accel_x_lp=accel_x、pitch_angle_est=0。这里保持相同初值，避免
2802:     % 在线窗口与训练窗口出现开头偏移。
2803:     dv_hat_dt = 0.0;
2804:     accel_x_lp = accel_x;
2805:     pitch_angle_est = 0.0;
2806:     state.has_feature_prev = true;
2807: else
2808:     dv_raw = (v_hat - state.v_hat_prev) / state.Ts;
2809:     dv_hat_dt = state.alpha_diff * dv_raw + (1 - state.alpha_diff) * state.dv_hat_dt_prev;
2810:     accel_x_lp = state.alpha_accel * accel_x + (1 - state.alpha_accel) * state.accel_x_lp_prev;
2811:     pitch_angle_est = state.lambda_pitch * state.pitch_angle_est_prev + gyro_y * state.Ts;
2812: end
2813:
2814: ws_imbalance = abs(omega_wheel_lf - omega_wheel_rr);
2815: I_sum = abs(I_lf) + abs(I_rr);
2816: I_diff_signed = I_lf - I_rr;
2817: I_diff_abs = abs(I_lf) - abs(I_rr);
2818: kappa_proxy = (tan(delta_lf) - tan(delta_rr)) / W;
2819: accel_per_current = accel_x_lp / max(I_sum, 0.1);
2820:
```

```
state.v_hat_prev = v_hat;
state.dv_hat_dt_prev = dv_hat_dt;
state.accel_x_lp_prev = accel_x_lp;
state.pitch_angle_est_prev = pitch_angle_est;

feature_raw = [accel_x, gyro_z, I_lf, I_rr, omega_wheel_lf, omega_wheel_rr, ...
    delta_lf, delta_rr, gyro_y, v_hat, dv_hat_dt, ws_imbalance, ...
    I_sum, I_diff_signed, I_diff_abs, accel_x_lp, kappa_proxy, ...
    accel_per_current, pitch_angle_est];
end

function tf = local_should_predict(state, request_predict)
if isempty(request_predict)
    tf = mod(state.step - state.seq_len, state.infer_period_steps) == 0;
else
    tf = logical(request_predict);
end
end

function out = local_output_from_prediction(state, pred)
out = struct();
out.seed = state.seed;
out.step = state.step;
out.buffer_count = state.buffer_count;
out.ready = state.buffer_count >= state.seq_len;
out.did_predict = false;
out.label_main = double(pred.main_state);
out.label_turn = double(pred.turn_state);
out.theta_hat_rad = double(pred.theta_hat_rad);
out.theta_hat_deg = double(pred.theta_hat_deg);
out.conf_main = double(pred.main_confidence);
out.conf_turn = double(pred.turn_confidence);
out.main_prob = double(pred.main_prob(:));
out.turn_prob = double(pred.turn_prob(:));
out.logits_main = single(pred.logits_main(:).');
out.logits_turn = single(pred.logits_turn(:).');
out.X_window_norm = state.buffer;
out.debug = struct();
out.debug.infer_period_steps = state.infer_period_steps;
out.debug.note = "ModernTCN raw output; theta_hat 建议先只记录，不直接接入 RhoFilter。";
end

function out = local_default_output(state)
out = struct();
out.seed = state.seed;
out.step = state.step;
out.buffer_count = state.buffer_count;
out.ready = false;
out.did_predict = false;
out.label_main = 1.0;
out.label_turn = 0.0;
out.theta_hat_rad = 0.0;
out.theta_hat_deg = 0.0;
out.conf_main = 1.0;
out.conf_turn = 1.0;
out.main_prob = [1; 0; 0];
out.turn_prob = [0; 1; 0];
out.logits_main = single([0 0 0]);
out.logits_turn = single([0 0 0]);
out.X_window_norm = state.buffer;
```

```
2881: out.feature_raw = zeros(1, state.feat_dim);
2882: out.feature_norm = zeros(1, state.feat_dim);
2883: out.debug = struct();
2884: out.debug.infer_period_steps = state.infer_period_steps;
2885: end
2886:
2887: function local_publish_output(out)
2888: % Simulink 薄封装通过 base workspace 读取标量，普通 MATLAB 调试也可查看。
2889: try
2890:     assignin('base', 'modern_tcn_online_last_out', out);
2891: catch
2892: end
2893: end
2894:
2895: function root = local_project_root()
2896: if exist('project_root', 'file') == 2
2897:     root = project_root();
2898: else
2899:     root = pwd;
2900: end
2901: end
2902:
2903: ===== FILE: src/ModernTCN/ModernTCN_state_classifier.m =====
2904: function [state, out] = ModernTCN_state_classifier(action, varargin)
2905: %MODERNTCN_STATE_CLASSIFIER ModernTCN 在线工况识别状态机。
2906: %
2907: % 功能说明：
2908: %   本函数从每个仿真步的 34 维 y_raw 中提取与 ModernTCN_dataset_v4_industrial
2909: %   完全一致的 19 维可观测特征，使用该数据集内保存的 scaler 做归一化，
2910: %   维护 128 步滑动窗口，然后调用当前推荐 ModernTCN ONNX predictor 推理。
2911: %
2912: % 设计边界：
2913: %   1. 本函数只用于 MATLAB/Simulink 仿真，不用于代码生成部署。
2914: %   2. 输入是原始 y_raw 单帧，不是已经归一化的 [128,19] 窗口。
2915: %   3. theta_hat 先作为诊断输出；第一阶段不建议直接接入 RhoFilter。
2916: %
2917: % 用法：
2918: %   params = parameters();
2919: %   state = ModernTCN_state_classifier('init', params);
2920: %   [state, out] = ModernTCN_state_classifier('update', state, y_raw_t);
2921:
2922: switch lower(action)
2923:     case 'init'
2924:         if nargin < 2
2925:             error('ModernTCN:init', '初始化需要提供 params。');
2926:         end
2927:         cfg = struct();
2928:         if nargin >= 3 && isstruct(varargin{2})
2929:             cfg = varargin{2};
2930:         end
2931:         state = local_init(varargin{1}, cfg);
2932:         out = [];
2933:
2934:     case 'update'
2935:         if nargin < 3
2936:             error('ModernTCN:update', '更新需要提供 state 和 y_raw_t。');
2937:         end
2938:         [state, out] = local_update(varargin{1}, varargin{2});
2939:
2940:     otherwise
```

```
2941:         error('ModernTCN:BadAction', '未知操作: %s，应为 init 或 update。', action);
2942: end
2943: end
2944:
2945: function state = local_init(params, cfg)
2946: root = local_project_root();
2947: default_cfg = ModernTCN_default_config(root);
2948:
2949: if ~isfield(cfg, 'seed') || isempty(cfg.seed)
2950:     cfg.seed = default_cfg.seed;
2951: end
2952: if ~isfield(cfg, 'dataset_file') || isempty(cfg.dataset_file)
2953:     cfg.dataset_file = default_cfg.dataset_file;
2954: end
2955: if (~isfield(cfg, 'run_tag') || isempty(cfg.run_tag)) && (~isfield(cfg, 'onnx_file') ||
isempty(cfg.onnx_file))
2956:     cfg.run_tag = default_cfg.run_tag;
2957: end
2958: if ~isfield(cfg, 'theta_output_gain') || isempty(cfg.theta_output_gain)
2959:     cfg.theta_output_gain = local_field_or_default(default_cfg, 'theta_output_gain', 1.0);
2960: end
2961: if ~isfield(cfg, 'theta_abs_limit') || isempty(cfg.theta_abs_limit)
2962:     cfg.theta_abs_limit = local_field_or_default(default_cfg, 'theta_abs_limit', inf);
2963: end
2964: if ~isfield(cfg, 'theta_rate_limit') || isempty(cfg.theta_rate_limit)
2965:     cfg.theta_rate_limit = local_field_or_default(default_cfg, 'theta_rate_limit', inf);
2966: end
2967: if ~isfield(cfg, 'theta_mpc_deadzone') || isempty(cfg.theta_mpc_deadzone)
2968:     cfg.theta_mpc_deadzone = local_field_or_default(default_cfg, 'theta_mpc_deadzone',
deg2rad(2.0));
2969: end
2970: onnx_file = '';
2971: if isfield(cfg, 'onnx_file') && ~isempty(cfg.onnx_file)
2972:     onnx_file = char(cfg.onnx_file);
2973: elseif isfield(cfg, 'run_tag') && ~isempty(cfg.run_tag)
2974:     onnx_file = fullfile(root, 'results', 'modern_tcn', char(cfg.run_tag), ...
2975:         sprintf('modern_tcn_seed%d.onnx', cfg.seed));
2976: end
2977:
2978: if exist(cfg.dataset_file, 'file') ~= 2
2979:     error('ModernTCN:MissingDataset', '找不到 ModernTCN 数据集: %s', cfg.dataset_file);
2980: end
2981:
2982: S = load(cfg.dataset_file, 'dataset');
2983: dataset = S.dataset;
2984: scaler = dataset.scaler;
2985:
2986: state = struct();
2987: state.params = params;
2988: state.Ts = local_field_or_default(params, 'Ts', 0.01);
2989: state.seed = cfg.seed;
2990: state.dataset_file = cfg.dataset_file;
2991: state.onnx_file = onnx_file;
2992: state.scaler = scaler;
2993: state.feat_names = dataset.feat_names;
2994: state.seq_len = local_field_or_default(dataset.meta, 'seq_len', 128);
2995: state.feat_dim = numel(scaler.mean);
2996: state.buffer = zeros(state.seq_len, state.feat_dim, 'single');
2997: state.buffer_count = 0;
2998:
2999: % 与 TCN_prepare_dataset.m 保持一致：训练数据跳过起始 1s 后再建窗。
3000: state.skip_initial_sec = local_field_or_default(dataset.meta, 'skip_initial_sec', 1.0);
```

```
3001: state.skip_initial_steps = round(state.skip_initial_sec / state.Ts);
3002: state.step = 0;
3003:
3004: % 特征提取参数。字段来自 TCN scaler，缺省值与 TCN_prepare_dataset.m 一致。
3005: state.tau_accel_lp = local_field_or_default(scaler, 'tau_accel_lp', 0.4);
3006: state.tau_diff = local_field_or_default(scaler, 'tau_diff', 0.3);
3007: state.tau_pitch = local_field_or_default(scaler, 'tau_pitch', 2.0);
3008: state.alpha_accel = state.Ts / (state.tau_accel_lp + state.Ts);
3009: state.alpha_diff = state.Ts / (state.tau_diff + state.Ts);
3010: state.lambda_pitch = exp(-state.Ts / state.tau_pitch);
3011:
3012: % 特征递推状态。首个有效样本按离线 local_lowpass/local_leaky_integral 的
3013: % 初值约定初始化，避免在线/离线开头一段不一致。
3014: state.feature_started = false;
3015: state.v_hat_prev = 0.0;
3016: state.dv_hat_dt_prev = 0.0;
3017: state.accel_x_lp_prev = 0.0;
3018: state.pitch_angle_est_prev = 0.0;
3019:
3020: % 标签驻留时间。ModernTCN 的 offline 指标较稳，但 Simulink 中单步更新仍需
3021: % 抑制边界跳变，先沿用 GRU 侧成熟参数。
3022: state.dwell_main = 0.20;
3023: state.dwell_turn = 0.40;
3024: state.dwell_main_steps = max(1, round(state.dwell_main / state.Ts));
3025: state.dwell_turn_steps = max(1, round(state.dwell_turn / state.Ts));
3026: state.label_main_current = 1;
3027: state.label_turn_current = 0;
3028: state.label_main_candidate = 1;
3029: state.label_turn_candidate = 0;
3030: state.label_main_stable_count = 0;
3031: state.label_turn_stable_count = 0;
3032: state.conf_main_current = 1.0;
3033: state.conf_turn_current = 1.0;
3034:
3035: % theta raw/conditioned output remains a model estimate. MPC scheduling can
3036: % use theta_hat_for_mpc, which applies the deployment deadzone outside the
3037: % learned regressor.
3038: state.tau_theta = 0.15;
3039: state.alpha_theta = state.Ts / (state.tau_theta + state.Ts);
3040: state.theta_hat_current = 0.0;
3041: state.theta_mpc_deadzone = double(cfg.theta_mpc_deadzone);
3042: state.theta_output_gain = double(cfg.theta_output_gain);
3043: state.theta_abs_limit = double(cfg.theta_abs_limit);
3044: state.theta_rate_limit = double(cfg.theta_rate_limit);
3045: state.theta_hat_output_prev = 0.0;
3046:
3047: % 19 维特征对应的 y_raw 索引，与 TCN_prepare_dataset.m 的 local_extract_runs 一致。
3048: state.feat_indices = struct();
3049: state.feat_indices.accel_x = 9;
3050: state.feat_indices.gyro_y = 10;
3051: state.feat_indices.gyro_z = 11;
3052: state.feat_indices.I_lf = 12;
3053: state.feat_indices.I_rr = 13;
3054: state.feat_indices.omega_wheel_lf = 17;
3055: state.feat_indices.omega_wheel_rr = 18;
3056: state.feat_indices.delta_lf = 6;
3057: state.feat_indices.delta_rr = 7;
3058: state.eps = 1e-8;
3059:
3060: % ONNX predictor 只在初始化时加载一次，避免每个仿真步重复导入网络。
```

```
3061: if isempty(state.onnx_file)
3062:     state.predictor = ModernTCN_load_predictor(state.seed);
3063: else
3064:     state.predictor = ModernTCN_load_predictor(state.seed, state.onnx_file);
3065: end
3066: state.is_ready = true;
3067: end
3068:
3069: function [state, out] = local_update(state, y_raw_t)
3070: state.step = state.step + 1;
3071:
3072: if isempty(y_raw_t) || numel(y_raw_t) < 18
3073:     out = local_default_output(state, 'y_raw 维度不足');
3074:     return;
3075: end
3076:
3077: % 与训练预处理保持一致：跳过起始 transient，跳过期间不更新特征滤波状态。
3078: if state.step <= state.skip_initial_steps
3079:     out = local_default_output(state, 'skip_initial_sec 内输出默认值');
3080:     return;
3081: end
3082:
3083: [features, state] = local_extract_features(double(y_raw_t(:)), state);
3084: if any(~isfinite(features))
3085:     out = local_default_output(state, '特征含 NaN/Inf');
3086:     return;
3087: end
3088:
3089: state.buffer(1:end-1, :) = state.buffer(2:end, :);
3090: state.buffer(end, :) = single(features);
3091: state.buffer_count = min(state.buffer_count + 1, state.seq_len);
3092:
3093: if state.buffer_count < state.seq_len
3094:     out = local_default_output(state, '滑动窗口未填满');
3095:     out.features = double(features(:));
3096:     out.features_norm = double(((features(:).' - double(state.scaler.mean)) ./ ...
3097:         (double(state.scaler.std) + state.eps)).');
3098:     return;
3099: end
3100:
3101: X_norm = (double(state.buffer) - double(state.scaler.mean)) ./ ...
3102:     (double(state.scaler.std) + state.eps);
3103:
3104: try
3105:     pred = ModernTCN_predict_window(state.predictor, single(X_norm));
3106: catch ME
3107:     warning('ModernTCN:InferenceFailed', 'ModernTCN 推理失败: %s', ME.message);
3108:     out = local_default_output(state, 'ModernTCN 推理失败');
3109:     return;
3110: end
3111:
3112: label_main_raw = double(pred.main_state);
3113: label_turn_raw = double(pred.turn_state);
3114: conf_main_raw = double(pred.main_confidence);
3115: conf_turn_raw = double(pred.turn_confidence);
3116: theta_hat_raw = double(pred.theta_hat_rad);
3117:
3118: [state.label_main_current, state.label_main_candidate, state.label_main_stable_count] = ...
3119:     local_apply_dwell(label_main_raw, state.label_main_current, ...
3120:     state.label_main_candidate, state.label_main_stable_count, state.dwell_main_steps);
```

```
3121:
3122: [state.label_turn_current, state.label_turn_candidate, state.label_turn_stable_count] = ...
3123:     local_apply_dwell(label_turn_raw, state.label_turn_current, ...
3124:     state.label_turn_candidate, state.label_turn_stable_count, state.dwell_turn_steps);
3125:
3126: state.conf_main_current = conf_main_raw;
3127: state.conf_turn_current = conf_turn_raw;
3128: state.theta_hat_current = state.alpha_theta * theta_hat_raw + ...
3129:     (1 - state.alpha_theta) * state.theta_hat_current;
3130:
3131: theta_hat_out = local_apply_theta_conditioning(state.theta_hat_current, state);
3132: theta_hat_for_mpc = local_apply_theta_mpc_deadzone(theta_hat_out, state);
3133: state.theta_hat_output_prev = theta_hat_out;
3134:
3135: out = struct();
3136: out.label_main = state.label_main_current;
3137: out.label_turn = state.label_turn_current;
3138: out.theta_hat = theta_hat_out;
3139: out.theta_hat_for_mpc = theta_hat_for_mpc;
3140: out.conf_main = state.conf_main_current;
3141: out.conf_turn = state.conf_turn_current;
3142: out.label_main_raw = label_main_raw;
3143: out.label_turn_raw = label_turn_raw;
3144: out.theta_hat_raw = theta_hat_raw;
3145: out.main_prob = double(pred.main_prob(:));
3146: out.turn_prob = double(pred.turn_prob(:));
3147: out.features = double(features(:));
3148: out.features_norm = double(X_norm(end, :).');
3149: out.debug = struct();
3150: out.debug.step = state.step;
3151: out.debug.buffer_count = state.buffer_count;
3152: out.debug.ready = true;
3153: out.debug.did_predict = true;
3154: out.debug.skip_initial_steps = state.skip_initial_steps;
3155: out.debug.feature_contract = 'GRU_compatible_observable_19';
3156: out.debug.theta_output_gain = state.theta_output_gain;
3157: out.debug.theta_abs_limit = state.theta_abs_limit;
3158: out.debug.theta_rate_limit = state.theta_rate_limit;
3159: out.debug.theta_mpc_deadzone = state.theta_mpc_deadzone;
3160: out.debug.note = 'ModernTCN online output';
3161: end
3162:
3163: function [features, state] = local_extract_features(y_raw, state)
3164: params = state.params;
3165: Ts = state.Ts;
3166: r = local_field_or_default(params, 'wheel_radius', 0.1);
3167: W = local_field_or_default(params, 'W', 1.0);
3168:
3169: accel_x = y_raw(state.feat_indices.accel_x);
3170: gyro_y = y_raw(state.feat_indices.gyro_y);
3171: gyro_z = y_raw(state.feat_indices.gyro_z);
3172: I_lf = y_raw(state.feat_indices.I_lf);
3173: I_rr = y_raw(state.feat_indices.I_rr);
3174: omega_wheel_lf = y_raw(state.feat_indices.omega_wheel_lf);
3175: omega_wheel_rr = y_raw(state.feat_indices.omega_wheel_rr);
3176: delta_lf = y_raw(state.feat_indices.delta_lf);
3177: delta_rr = y_raw(state.feat_indices.delta_rr);
3178:
3179: v_hat = r * (omega_wheel_lf + omega_wheel_rr) / 2;
3180: if ~state.feature_started
```

```
3181:     dv_raw = 0.0;
3182:     dv_hat_dt = 0.0;
3183:     accel_x_lp = accel_x;
3184:     pitch_angle_est = 0.0;
3185:     state.feature_started = true;
3186: else
3187:     dv_raw = (v_hat - state.v_hat_prev) / Ts;
3188:     dv_hat_dt = state.alpha_diff * dv_raw + ...
3189:         (1 - state.alpha_diff) * state.dv_hat_dt_prev;
3190:     accel_x_lp = state.alpha_accel * accel_x + ...
3191:         (1 - state.alpha_accel) * state.accel_x_lp_prev;
3192:     pitch_angle_est = state.lambda_pitch * state.pitch_angle_est_prev + gyro_y * Ts;
3193: end
3194:
3195: state.v_hat_prev = v_hat;
3196: state.dv_hat_dt_prev = dv_hat_dt;
3197: state.accel_x_lp_prev = accel_x_lp;
3198: state.pitch_angle_est_prev = pitch_angle_est;
3199:
3200: ws_imbalance = abs(omega_wheel_lf - omega_wheel_rr);
3201: I_sum = abs(I_lf) + abs(I_rr);
3202: I_diff_signed = I_lf - I_rr;
3203: I_diff_abs = abs(I_lf) - abs(I_rr);
3204: kappa_proxy = (tan(delta_lf) - tan(delta_rr)) / W;
3205: accel_per_current = accel_x_lp / max(I_sum, 0.1);
3206:
3207: features = [accel_x, gyro_z, I_lf, I_rr, omega_wheel_lf, omega_wheel_rr, ...
3208:     delta_lf, delta_rr, gyro_y, v_hat, dv_hat_dt, ws_imbalance, ...
3209:     I_sum, I_diff_signed, I_diff_abs, accel_x_lp, kappa_proxy, ...
3210:     accel_per_current, pitch_angle_est];
3211: end
3212:
3213: function [current, candidate, count] = local_apply_dwell(raw_label, current, candidate, count,
required_count)
3214: if raw_label == candidate
3215:     count = count + 1;
3216:     if count >= required_count
3217:         current = raw_label;
3218:     end
3219: else
3220:     candidate = raw_label;
3221:     count = 1;
3222: end
3223: end
3224:
3225: function theta_hat = local_apply_theta_conditioning(theta_hat, state)
3226: theta_hat = state.theta_output_gain * theta_hat;
3227:
3228: if isfinite(state.theta_abs_limit) && state.theta_abs_limit > 0
3229:     theta_hat = min(max(theta_hat, -state.theta_abs_limit), state.theta_abs_limit);
3230: end
3231:
3232: if isfinite(state.theta_rate_limit) && state.theta_rate_limit > 0
3233:     max_step = state.theta_rate_limit * state.Ts;
3234:     theta_delta = theta_hat - state.theta_hat_output_prev;
3235:     theta_delta = min(max(theta_delta, -max_step), max_step);
3236:     theta_hat = state.theta_hat_output_prev + theta_delta;
3237: end
3238: end
3239:
3240: function theta_hat = local_apply_theta_mpc_deadzone(theta_hat, state)
```

```
3241: theta_abs = abs(theta_hat);
3242: if theta_abs <= state.theta_mpc_deadzone
3243:     theta_hat = 0.0;
3244: end
3245: end
3246:
3247: function out = local_default_output(state, note)
3248: out = struct();
3249: out.label_main = state.label_main_current;
3250: out.label_turn = state.label_turn_current;
3251: out.theta_hat = state.theta_hat_current;
3252: out.theta_hat_for_mpc = local_apply_theta_mpc_deadzone(state.theta_hat_current, state);
3253: out.conf_main = state.conf_main_current;
3254: out.conf_turn = state.conf_turn_current;
3255: out.label_main_raw = state.label_main_current;
3256: out.label_turn_raw = state.label_turn_current;
3257: out.theta_hat_raw = state.theta_hat_current;
3258: out.main_prob = [1; 0; 0];
3259: out.turn_prob = [0; 1; 0];
3260: out.features = nan(19, 1);
3261: out.features_norm = nan(19, 1);
3262: out.debug = struct();
3263: out.debug.step = state.step;
3264: out.debug.buffer_count = state.buffer_count;
3265: out.debug.ready = state.buffer_count >= state.seq_len;
3266: out.debug.did_predict = false;
3267: out.debug.skip_initial_steps = state.skip_initial_steps;
3268: out.debug.note = note;
3269: end
3270:
3271: function v = local_field_or_default(s, field_name, default_value)
3272: if isstruct(s) && isfield(s, field_name) && ~isempty(s.(field_name))
3273:     v = s.(field_name);
3274: else
3275:     v = default_value;
3276: end
3277: end
3278:
3279: function root = local_project_root()
3280: if exist('project_root', 'file') == 2
3281:     root = project_root();
3282: else
3283:     root = pwd;
3284: end
3285: end
3286:
3287: ===== FILE: src/ModernTCN/ModernTCN_State_Classifier_sim.m =====
3288: function [theta_hat, label_main, label_turn, conf_main] = ModernTCN_State_Classifier_sim(y_raw,
reset)
3289: %MODERNTCN_STATE_CLASSIFIER_SIM Simulink MATLAB Function 块调用入口。
3290: %#codegen
3291: %
3292: % 功能说明:
3293: %   这是替换 GRU_State_Classifier_gru_sim 的薄封装。Simulink 每个仿真步给
3294: %   一帧 34 维 y_raw，本函数通过 extrinsic 调用 MATLAB 侧的
3295: %   ModernTCN_state_classifier，内部维护特征滤波、128 步滑窗、归一化和
3296: %   ONNX 推理状态。
3297: %
3298: % 手动接入时，MATLAB Function block 内建议改为:
3299: %   [theta_hat, label_main, label_turn, conf_main] = ...
3300: %       ModernTCN_State_Classifier_sim(y_raw, reset);
```

```
3301:
3302: coder.extrinsic('evalin');
3303: coder.extrinsic('assignin');
3304: coder.extrinsic('ModernTCN_state_classifier');
3305:
3306: persistent state is_initialized
3307:
3308: theta_hat = 0.0;
3309: label_main = 1.0;
3310: label_turn = 0.0;
3311: conf_main = 1.0;
3312:
3313: if isempty(is_initialized)
3314:     is_initialized = false;
3315: end
3316:
3317: need_reset = (~is_initialized) || (reset ~= 0);
3318: if need_reset
3319:     has_required = evalin('base', 'exist(''params'', ''var'')==1');
3320:     if ~has_required
3321:         return;
3322:     end
3323:
3324:     params = evalin('base', 'params');
3325:     has_cfg = evalin('base', 'exist(''modern_tcn_sim_cfg'', ''var'')==1');
3326:     if has_cfg
3327:         cfg = evalin('base', 'modern_tcn_sim_cfg');
3328:         state = ModernTCN_state_classifier('init', params, cfg);
3329:     else
3330:         state = ModernTCN_state_classifier('init', params);
3331:     end
3332:     is_initialized = true;
3333:     return;
3334: end
3335:
3336: if isempty(state)
3337:     return;
3338: end
3339:
3340: if isempty(y_raw) || numel(y_raw) < 18
3341:     return;
3342: end
3343:
3344: [state, out] = ModernTCN_state_classifier('update', state, double(y_raw(:)));
3345: if isempty(out)
3346:     return;
3347: end
3348:
3349: assignin('base', 'modern_tcn_out_temp', out);
3350: theta_hat = evalin('base', 'double(modern_tcn_out_temp.theta_hat)');
3351: label_main = evalin('base', 'double(modern_tcn_out_temp.label_main)');
3352: label_turn = evalin('base', 'double(modern_tcn_out_temp.label_turn)');
3353: conf_main = evalin('base', 'double(modern_tcn_out_temp.conf_main)');
3354: end
3355:
3356: ===== FILE: src/ModernTCN/ModernTCN_check_matlab_onnx.m =====
3357: function result = ModernTCN_check_matlab_onnx(onnx_file, sample_file)
3358: %MODERNTCN_CHECK_MATLAB_ONNX 导入 ModernTCN ONNX 并和 PyTorch 输出做离线一致性检查。
3359: %
3360: % 用法：
```

```
%   init_project;
%   addpath(fullfile(project_root(), 'src', 'ModernTCN'));
%   result = ModernTCN_check_matlab_onnx();
%
% 说明：
%   该脚本只做离线 test window 推理检查，不接入 Simulink。输入固定为
%   [batch, time=128, feature=19]，输出固定为 logits_main、logits_turn、
%   theta_hat。通过该检查后，才建议继续写 MATLAB 在线封装。

if nargin < 1 || isempty(onnx_file)
    default_cfg = ModernTCN_default_config(project_root());
    onnx_file = default_cfg.onnx_file;
end
if nargin < 2 || isempty(sample_file)
    default_cfg = ModernTCN_default_config(project_root());
    sample_file = default_cfg.pytorch_reference_file;
end

if exist(onnx_file, 'file') ~= 2
    error('ModernTCN:MissingONNX', '找不到 ONNX 文件: %s', onnx_file);
end
if exist(sample_file, 'file') ~= 2
    error('ModernTCN:MissingSample', '找不到 PyTorch 参考样本: %s', sample_file);
end

S = load(sample_file);
X = single(S.X_sample);

% R2024b 的 importNetworkFromONNX 已直接返回 dlnetwork，不支持旧
% importONNXNetwork 中的 TargetNetwork 参数。ONNX importer 对部分算子会
% 自动生成 MATLAB custom layer package；这里把生成目录固定到
% src/ModernTCN/generated_layers，避免在项目根目录生成 +modern_tcn_seed*
% 这类临时 package 文件夹。
net = local_import_modern_tcn_onnx(onnx_file);
[logits_main, logits_turn, theta_hat] = local_predict_all_windows(net, X);

result = struct();
result.onnx_file = onnx_file;
result.sample_file = sample_file;
result.logits_main = local_diff(logits_main, single(S.logits_main_pytorch));
result.logits_turn = local_diff(logits_turn, single(S.logits_turn_pytorch));
result.theta_hat = local_diff(theta_hat, single(S.theta_hat_pytorch));
result.max_abs_tol = 1e-4;
result.mean_abs_tol = 1e-5;
result.pass = result.logits_main.max_abs_error <= result.max_abs_tol ...
    && result.logits_main.mean_abs_error <= result.mean_abs_tol ...
    && result.logits_turn.max_abs_error <= result.max_abs_tol ...
    && result.logits_turn.mean_abs_error <= result.mean_abs_tol ...
    && result.theta_hat.max_abs_error <= result.max_abs_tol ...
    && result.theta_hat.mean_abs_error <= result.mean_abs_tol;

[out_dir, onnx_name] = fileparts(onnx_file);
mat_file = fullfile(out_dir, sprintf('%s_matlab_consistency.mat', onnx_name));
md_file = fullfile(out_dir, sprintf('%s_matlab_consistency.md', onnx_name));
save(mat_file, 'result');
local_write_report(md_file, result);

fprintf('[ModernTCN MATLAB ONNX] pass=%d\n', result.pass);
fprintf('  mat: %s\n', mat_file);
fprintf('  report: %s\n', md_file);
```

```
end

function net = local_import_modern_tcn_onnx(onnx_file)
if exist('project_root', 'file') == 2
    root = project_root();
else
    root = pwd;
end
layer_root = fullfile(root, 'src', 'ModernTCN', 'generated_layers');
if exist(layer_root, 'dir') ~= 7
    mkdir(layer_root);
end
addpath(layer_root);
old_dir = pwd;
cleanup = onCleanup(@() cd(old_dir));
cd(layer_root);
net = importNetworkFromONNX(onnx_file, Namespace=local_onnx_namespace(onnx_file));
end

function namespace = local_onnx_namespace(onnx_file)
if contains(lower(char(onnx_file)), 'causal')
    namespace = "modern_tcn_causal_onnx_layers";
else
    namespace = "modern_tcn_onnx_layers";
end
end

function [logits_main, logits_turn, theta_hat] = local_predict_all_windows(net, X)
% ONNX 第一版固定 batch=1，更接近在线单窗口部署；离线一致性检查逐窗口拼接。
n = size(X, 1);
logits_main = zeros(n, 3, 'single');
logits_turn = zeros(n, 3, 'single');
theta_hat = zeros(n, 1, 'single');
for i = 1:n
    [lm, lt, th] = local_predict_modern_tcn(net, X(i,:,:));
    logits_main(i,:) = lm(1,:);
    logits_turn(i,:) = lt(1,:);
    theta_hat(i,:) = th(1,:);
end
end

function [logits_main, logits_turn, theta_hat] = local_predict_modern_tcn(net, X)
% 尽量兼容不同 MATLAB ONNX importer 对 NTC/CBT 维度的解释。
try
    [logits_main, logits_turn, theta_hat] = predict(net, X);
catch
    try
        [logits_main, logits_turn, theta_hat] = predict(net, dlarray(X, "BTC"));
    catch
        % 若 importer 将输入解释为 CBT，可手动转成 [feature,time,batch]。
        Xcbt = permute(X, [3 2 1]);
        [logits_main, logits_turn, theta_hat] = predict(net, dlarray(Xcbt, "CTB"));
    end
end

logits_main = local_extract(logits_main);
logits_turn = local_extract(logits_turn);
theta_hat = local_extract(theta_hat);
logits_main = local_to_batch_first(logits_main, 3);
logits_turn = local_to_batch_first(logits_turn, 3);
```

```
theta_hat = local_to_batch_first(theta_hat, 1);
end

function A = local_extract(A)
if isa(A, 'dlarray')
    A = gather(extractdata(A));
else
    A = gather(A);
end
end

function A = local_to_batch_first(A, width)
A = squeeze(A);
if size(A, 2) ~= width && size(A, 1) == width
    A = A.';
end
if width == 1
    A = reshape(A, [], 1);
end
A = single(A);
end

function d = local_diff(A, B)
d = struct();
D = abs(single(A) - single(B));
d.max_abs_error = max(D(:));
d.mean_abs_error = mean(D(:));
end

function local_write_report(md_file, result)
fid = fopen(md_file, 'w');
if fid < 0
    warning('ModernTCN:ReportFailed', '无法写入报告: %s', md_file);
    return;
end
cleanup = onCleanup(@() fclose(fid));
fprintf(fid, '# ModernTCN MATLAB ONNX 一致性检查\n\n');
fprintf(fid, '- onnx: `%s`\n', result.onnx_file);
fprintf(fid, '- sample: `%s`\n', result.sample_file);
fprintf(fid, '- pass: `%d`\n\n', result.pass);
fprintf(fid, '| output | max abs error | mean abs error |\n');
fprintf(fid, '|---|---:|---:|\n');
fprintf(fid, '| logits_main | %.6g | %.6g |\n', ...
    result.logits_main.max_abs_error, result.logits_main.mean_abs_error);
fprintf(fid, '| logits_turn | %.6g | %.6g |\n', ...
    result.logits_turn.max_abs_error, result.logits_turn.mean_abs_error);
fprintf(fid, '| theta_hat | %.6g | %.6g |\n', ...
    result.theta_hat.max_abs_error, result.theta_hat.mean_abs_error);
end

===== FILE: src/core/preloadfcn_modern_tcn.m =====
function preloadfcn_modern_tcn()
%PRELOADFCN_MODERN_TCN PreLoadFcn entry for the ModernTCN closed-loop model.
%
% The common LPV/MPC initialization lives in preloadfcn_gru. The mode flag
% selects the frozen ModernTCN artifact instead of loading a GRU artifact.

preloadfcn_gru('modern_tcn');
end

```

```
3541: ===== FILE: src/ModernTCN/ModernTCN_analyze_closed_loop_out.m =====
3542: function result = ModernTCN_analyze_closed_loop_out(out_file)
3543: %MODERNTCN_ANALYZE_CLOSED_LOOP_OUT Analyze ModernTCN closed-loop Simulink logs.
3544: %
3545: %   result = ModernTCN_analyze_closed_loop_out('out.mat')
3546: %
3547: % The analysis expects logsout to contain diag.y_raw plus the public diag.*
3548: % signals from LPVMPC_AGV_simulink_Modern_TCN. It does not modify the model.
3549:
3550: if nargin < 1 || isempty(out_file)
3551:     out_file = fullfile(local_project_root(), 'out.mat');
3552: end
3553: if exist('init_project', 'file') == 2
3554:     init_project();
3555: end
3556:
3557: root = local_project_root();
3558: S = load(out_file, 'logsout');
3559: logs = S.logsout;
3560:
3561: t = local_time(logs, 'diag.y_raw');
3562: y_raw = local_data(logs, 'diag.y_raw');
3563: theta_hat = local_resample(logs, 'diag.theta_hat', t);
3564: label_main = local_resample(logs, 'diag.label_main', t);
3565: label_turn = local_resample(logs, 'diag.label_turn', t);
3566: conf_main = local_resample(logs, 'diag.conf_main', t);
3567: theta_ground = local_resample(logs, 'diag.theta_ground', t);
3568: theta_ref = local_resample(logs, 'diag.theta_ref', t);
3569: omega_ref = local_resample(logs, 'diag.omega_ref', t);
3570: v_ref = local_resample(logs, 'diag.v_ref', t);
3571:
3572: e_y = local_resample(logs, 'diag.e_y', t);
3573: e_psi = local_resample(logs, 'diag.e_psi', t);
3574: e_v = local_resample(logs, 'diag.e_v', t);
3575: e_omega = local_resample(logs, 'diag.e_omega', t);
3576: F_cmd = local_resample(logs, 'diag.F_cmd', t);
3577: omega_cmd = local_resample(logs, 'diag.omega_cmd', t);
3578:
3579: default_cfg = ModernTCN_default_config(root);
3580: dataset_file = default_cfg.dataset_file;
3581: Ds = load(dataset_file, 'dataset');
3582: dataset = Ds.dataset;
3583:
3584: params = parameters();
3585: [feature_raw, feature_norm] = local_extract_features_from_yraw(y_raw, t, params, dataset);
3586:
3587: zones = local_get_zones(root, t);
3588: zone_names = fieldnames(zones);
3589:
3590: truth = local_make_truth(theta_ground, omega_ref);
3591:
3592: rows = repmat(local_empty_zone_row(), numel(zone_names), 1);
3593: for i = 1:numel(zone_names)
3594:     name = zone_names{i};
3595:     tr = zones.(name);
3596:     mask = t >= tr(1) & t < tr(2);
3597:     rows(i) = local_zone_row(name, tr, mask, t, e_y, e_psi, e_v, e_omega, ...
3598:         theta_hat, theta_ground, label_main, label_turn, conf_main, truth, ...
3599:         F_cmd, omega_cmd);
3600: end
```

```
zone_table = struct2table(rows);

feature_zone = local_feature_zone_summary(zone_names, zones, t, feature_norm, dataset.feat_names);
feature_compare = local_feature_compare_to_training(dataset, feature_norm, t, zones);
assessment = local_assess_closed_loop(zone_table, feature_zone);

out_dir = fullfile(root, 'results', 'modern_tcn', 'closed_loop_diag');
if exist(out_dir, 'dir') ~= 7
    mkdir(out_dir);
end
[~, base_name, ~] = fileparts(out_file);
mat_file = fullfile(out_dir, sprintf('%s_modern_tcn_closed_loop_diag.mat', base_name));
report_file = fullfile(out_dir, sprintf('%s_modern_tcn_closed_loop_diag_report.md', base_name));

result = struct();
result.out_file = out_file;
result.dataset_file = dataset_file;
result.zone_table = zone_table;
result.feature_zone = feature_zone;
result.feature_compare = feature_compare;
result.assessment = assessment;
result.feature_names = dataset.feat_names(:);
result.theta_error_deg = rad2deg(theta_hat - theta_ground);
result.report_file = report_file;
result.mat_file = mat_file;

save(mat_file, 'result');
local_write_report(report_file, result);

fprintf('[ModernTCN closed-loop diag] report: %s\n', report_file);
fprintf('[ModernTCN closed-loop diag] mat   : %s\n', mat_file);
disp(zone_table(:, {'zone','ey_rmse','epsi_rmse','ev_rmse','theta_mae_deg', ...
    'main_acc_pct','turn_acc_pct','pred_main_1_pct','pred_main_3_pct', ...
    'pred_turn_0_pct','pred_turn_1_pct'}));
disp(assessment(:, {'check','value','threshold','pass'}));
end

function row = local_empty_zone_row()
row = struct('zone', "", 't0', NaN, 't1', NaN, ...
    'ey_rmse', NaN, 'ey_peak', NaN, 'epsi_rmse', NaN, 'epsi_peak', NaN, ...
    'ev_rmse', NaN, 'ev_peak', NaN, 'eomega_rmse', NaN, 'eomega_peak', NaN, ...
    'theta_mae_deg', NaN, 'theta_rmse_deg', NaN, 'theta_peak_deg', NaN, ...
    'theta_hat_min_deg', NaN, 'theta_hat_max_deg', NaN, ...
    'main_acc_pct', NaN, 'turn_acc_pct', NaN, ...
    'true_main_1_pct', NaN, 'true_main_3_pct', NaN, ...
    'pred_main_1_pct', NaN, 'pred_main_2_pct', NaN, 'pred_main_3_pct', NaN, ...
    'true_turn_m1_pct', NaN, 'true_turn_0_pct', NaN, 'true_turn_1_pct', NaN, ...
    'pred_turn_m1_pct', NaN, 'pred_turn_0_pct', NaN, 'pred_turn_1_pct', NaN, ...
    'conf_median', NaN, 'conf_p10', NaN, ...
    'F_min', NaN, 'F_max', NaN, 'omega_cmd_min', NaN, 'omega_cmd_max', NaN);
end

function row = local_zone_row(name, tr, mask, ~, e_y, e_psi, e_v, e_omega, ...
    theta_hat, theta_ground, label_main, label_turn, conf_main, truth, F_cmd, omega_cmd)
row = local_empty_zone_row();
row.zone = string(name);
row.t0 = tr(1);
row.t1 = tr(2);
if nnz(mask) < 2
    return;
```

```
3661: end
3662:
3663: th_err = theta_hat(mask) - theta_ground(mask);
3664: row.ey_rmse = local_rms(e_y(mask));
3665: row.ey_peak = max(abs(e_y(mask)));
3666: row.epsi_rmse = local_rms(e_psi(mask));
3667: row.epsi_peak = max(abs(e_psi(mask)));
3668: row.ev_rmse = local_rms(e_v(mask));
3669: row.ev_peak = max(abs(e_v(mask)));
3670: row.eomega_rmse = local_rms(e_omega(mask));
3671: row.eomega_peak = max(abs(e_omega(mask)));
3672: row.theta_mae_deg = rad2deg(mean(abs(th_err), 'omitnan'));
3673: row.theta_rmse_deg = rad2deg(local_rms(th_err));
3674: row.theta_peak_deg = rad2deg(max(abs(th_err)));
3675: row.theta_hat_min_deg = rad2deg(min(theta_hat(mask)));
3676: row.theta_hat_max_deg = rad2deg(max(theta_hat(mask)));
3677:
3678: lm = round(label_main(mask));
3679: lt = round(label_turn(mask));
3680: tm = truth.main(mask);
3681: tt = truth.turn(mask);
3682: row.main_acc_pct = 100 * mean(lm == tm, 'omitnan');
3683: row.turn_acc_pct = 100 * mean(lt == tt, 'omitnan');
3684: row.true_main_1_pct = 100 * mean(tm == 1, 'omitnan');
3685: row.true_main_3_pct = 100 * mean(tm == 3, 'omitnan');
3686: row.pred_main_1_pct = 100 * mean(lm == 1, 'omitnan');
3687: row.pred_main_2_pct = 100 * mean(lm == 2, 'omitnan');
3688: row.pred_main_3_pct = 100 * mean(lm == 3, 'omitnan');
3689: row.true_turn_m1_pct = 100 * mean(tt == -1, 'omitnan');
3690: row.true_turn_0_pct = 100 * mean(tt == 0, 'omitnan');
3691: row.true_turn_1_pct = 100 * mean(tt == 1, 'omitnan');
3692: row.pred_turn_m1_pct = 100 * mean(lt == -1, 'omitnan');
3693: row.pred_turn_0_pct = 100 * mean(lt == 0, 'omitnan');
3694: row.pred_turn_1_pct = 100 * mean(lt == 1, 'omitnan');
3695: row.conf_median = median(conf_main(mask), 'omitnan');
3696: row.conf_p10 = prctile(conf_main(mask), 10);
3697: row.F_min = min(F_cmd(mask));
3698: row.F_max = max(F_cmd(mask));
3699: row.omega_cmd_min = min(omega_cmd(mask));
3700: row.omega_cmd_max = max(omega_cmd(mask));
3701: end
3702:
3703: function truth = local_make_truth(theta_ground, omega_ref)
3704: truth = struct();
3705: truth.main = ones(size(theta_ground));
3706: truth.main(abs(theta_ground) > deg2rad(2.0)) = 3;
3707: truth.turn = zeros(size(omega_ref));
3708: truth.turn(omega_ref > 0.02) = 1;
3709: truth.turn(omega_ref < -0.02) = -1;
3710: end
3711:
3712: function [feature_raw, feature_norm] = local_extract_features_from_yraw(y_raw, t, params, dataset)
3713: Ts = median(diff(t));
3714: scaler = dataset.scaler;
3715: seq_skip = round(dataset.meta.skip_initial_sec / Ts);
3716: n = size(y_raw, 1);
3717: feat_dim = numel(scaler.mean);
3718: feature_raw = nan(n, feat_dim);
3719:
3720: r = local_field_or_default(params, 'wheel_radius', 0.1);
```

```
W = local_field_or_default(params, 'W', 1.0);
alpha_accel = Ts / (scaler.tau_accel_lp + Ts);
alpha_diff = Ts / (scaler.tau_diff + Ts);
lambda_pitch = exp(-Ts / scaler.tau_pitch);

started = false;
v_hat_prev = 0.0;
dv_hat_dt_prev = 0.0;
accel_x_lp_prev = 0.0;
pitch_angle_est_prev = 0.0;

for k = (seq_skip + 1):n
    y = double(y_raw(k, :));
    accel_x = y(9);
    gyro_y = y(10);
    gyro_z = y(11);
    I_lf = y(12);
    I_rr = y(13);
    omega_wheel_lf = y(17);
    omega_wheel_rr = y(18);
    delta_lf = y(6);
    delta_rr = y(7);
    v_hat = r * (omega_wheel_lf + omega_wheel_rr) / 2;

    if ~started
        dv_hat_dt = 0.0;
        accel_x_lp = accel_x;
        pitch_angle_est = 0.0;
        started = true;
    else
        dv_raw = (v_hat - v_hat_prev) / Ts;
        dv_hat_dt = alpha_diff * dv_raw + (1 - alpha_diff) * dv_hat_dt_prev;
        accel_x_lp = alpha_accel * accel_x + (1 - alpha_accel) * accel_x_lp_prev;
        pitch_angle_est = lambda_pitch * pitch_angle_est_prev + gyro_y * Ts;
    end

    ws_imbalance = abs(omega_wheel_lf - omega_wheel_rr);
    I_sum = abs(I_lf) + abs(I_rr);
    I_diff_signed = I_lf - I_rr;
    I_diff_abs = abs(I_lf) - abs(I_rr);
    kappa_proxy = (tan(delta_lf) - tan(delta_rr)) / W;
    accel_per_current = accel_x_lp / max(I_sum, 0.1);

    feature_raw(k, :) = [accel_x, gyro_z, I_lf, I_rr, omega_wheel_lf, omega_wheel_rr, ...
        delta_lf, delta_rr, gyro_y, v_hat, dv_hat_dt, ws_imbalance, ...
        I_sum, I_diff_signed, I_diff_abs, accel_x_lp, kappa_proxy, ...
        accel_per_current, pitch_angle_est];

    v_hat_prev = v_hat;
    dv_hat_dt_prev = dv_hat_dt;
    accel_x_lp_prev = accel_x_lp;
    pitch_angle_est_prev = pitch_angle_est;
end

feature_norm = (feature_raw - double(scaler.mean(:).')) ./ ...
    (double(scaler.std(:).') + 1e-8);
end

function T = local_feature_zone_summary(zone_names, zones, t, feature_norm, feat_names)
rows = repmat(struct('zone', "", 'feature', "", 'median_z', NaN, ...
```

```
    'p95_abs_z', NaN, 'max_abs_z', NaN, 'pct_abs_z_gt_3', NaN), ...
    numel(zone_names) * numel(feat_names), 1);
idx = 0;
for zi = 1:numel(zone_names)
    zn = zone_names{zi};
    tr = zones.(zn);
    mask = t >= tr(1) & t < tr(2);
    for fi = 1:numel(feat_names)
        idx = idx + 1;
        z = feature_norm(mask, fi);
        z = z(isfinite(z));
        rows(idx).zone = string(zn);
        rows(idx).feature = string(feat_names{fi});
        if isempty(z)
            continue;
        end
        rows(idx).median_z = median(z);
        rows(idx).p95_abs_z = prctile(abs(z), 95);
        rows(idx).max_abs_z = max(abs(z));
        rows(idx).pct_abs_z_gt_3 = 100 * mean(abs(z) > 3);
    end
end
T = struct2table(rows(1:idx));
end

function T = local_feature_compare_to_training(dataset, feature_norm, t, zones)
feat_names = dataset.feat_names(:);
X = cat(1, dataset.X_train, dataset.X_val, dataset.X_test);
y_main = [dataset.y_main_train; dataset.y_main_val; dataset.y_main_test];
y_turn = [dataset.y_turn_train; dataset.y_turn_val; dataset.y_turn_test];

cases = {
    'train_flat_straight',  y_main == 1 & y_turn == 0;
    'train_flat_left',      y_main == 1 & y_turn == 1;
    'train_slope_straight', y_main == 3 & y_turn == 0;
    'sim_pure_turn',        [];
    'sim_pure_slope',       [];
    'sim_composite',        [];
};

rows = repmat(struct('group', "", 'feature', "", 'median_z', NaN, ...
    'p95_abs_z', NaN, 'max_abs_z', NaN), numel(cases) * numel(feat_names), 1);
idx = 0;
for ci = 1:size(cases, 1)
    group = cases{ci, 1};
    train_mask = cases{ci, 2};
    if startsWith(group, 'train')
        if ~any(train_mask)
            Z = [];
        else
            Xg = X(train_mask, :, :);
            Z = reshape(Xg, [], numel(feat_names));
        end
    else
        switch group
            case 'sim_pure_turn'
                tr = zones.pure_turn;
            case 'sim_pure_slope'
                tr = zones.pure_slope;
            otherwise
```

```
                tr = zones.composite;
            end
            m = t >= tr(1) & t < tr(2);
            Z = feature_norm(m, :);
        end
        for fi = 1:numel(feat_names)
            idx = idx + 1;
            z = Z(:, fi);
            z = z(isfinite(z));
            rows(idx).group = string(group);
            rows(idx).feature = string(feat_names{fi});
            if isempty(z)
                continue;
            end
            rows(idx).median_z = median(z);
            rows(idx).p95_abs_z = prctile(abs(z), 95);
            rows(idx).max_abs_z = max(abs(z));
        end
    end
    T = struct2table(rows(1:idx));
    end

    function assessment = local_assess_closed_loop(zone_table, feature_zone)
    rows = repmat(struct('check', "", 'value', NaN, 'threshold', "", ...
        'pass', false, 'comment', ""), 0, 1);

    pure_turn = local_zone_or_empty(zone_table, 'pure_turn');
    pure_slope = local_zone_or_empty(zone_table, 'pure_slope');
    composite = local_zone_or_empty(zone_table, 'composite');
    closure = local_zone_or_empty(zone_table, 'closure');
    if height(closure) == 0
        closure = local_zone_or_empty(zone_table, 'closed_loop');
    end

    if height(pure_turn) > 0
        rows(end+1) = local_assess_row('pure_turn_false_slope_pct', ...
            pure_turn.pred_main_3_pct, '<= 5', pure_turn.pred_main_3_pct <= 5, ...
            'Flat turn should not be classified as slope.');
        rows(end+1) = local_assess_row('pure_turn_left_recall_pct', ...
            pure_turn.pred_turn_1_pct, '>= 80', pure_turn.pred_turn_1_pct >= 80, ...
            'Flat left turn should be detected before labels are used by MPC.');
        rows(end+1) = local_assess_row('pure_turn_theta_mae_deg', ...
            pure_turn.theta_mae_deg, '<= 2', pure_turn.theta_mae_deg <= 2, ...
            'Theta estimate should stay near zero on flat turns.');
    end

    if height(pure_slope) > 0
        rows(end+1) = local_assess_row('pure_slope_main_acc_pct', ...
            pure_slope.main_acc_pct, '>= 90', pure_slope.main_acc_pct >= 90, ...
            'Pure slope segment should be recognized consistently.');
        rows(end+1) = local_assess_row('pure_slope_theta_mae_deg', ...
            pure_slope.theta_mae_deg, '<= 2', pure_slope.theta_mae_deg <= 2, ...
            'Slope magnitude error should remain below controller-use tolerance.');
    end

    if height(composite) > 0
        rows(end+1) = local_assess_row('composite_turn_recall_pct', ...
            composite.pred_turn_1_pct, '>= 60', composite.pred_turn_1_pct >= 60, ...
            'Composite slope-turn segment should preserve turn information.');
    end
```

```
3901:
3902: if height(closure) > 0
3903:     rows(end+1) = local_assess_row('closure_main_acc_pct', ...
3904:         closure.main_acc_pct, '>= 90', closure.main_acc_pct >= 90, ...
3905:         'Low-speed closure should not collapse into false slope labels.');
3906: end
3907:
3908: ood = local_feature_ood(feature_zone, closure, 'v_hat');
3909: if isfinite(ood)
3910:     rows(end+1) = local_assess_row('closure_v_hat_ood_pct', ...
3911:         ood, '<= 5', ood <= 5, ...
3912:         'High low-speed OOD rate means the dataset needs low-speed closure samples.');
3913: end
3914:
3915: ood = local_feature_ood(feature_zone, closure, 'accel_per_current');
3916: if isfinite(ood)
3917:     rows(end+1) = local_assess_row('closure_accel_per_current_ood_pct', ...
3918:         ood, '<= 5', ood <= 5, ...
3919:         'Large accel/current ratio OOD points to low-load sample coverage gap.');
3920: end
3921:
3922: assessment = struct2table(rows);
3923: end
3924:
3925: function row = local_assess_row(check, value, threshold, pass, comment)
3926: row = struct('check', string(check), 'value', double(value), ...
3927:     'threshold', string(threshold), 'pass', logical(pass), ...
3928:     'comment', string(comment));
3929: end
3930:
3931: function Z = local_zone_or_empty(T, name)
3932: Z = T(T.zone == string(name), :);
3933: end
3934:
3935: function ood = local_feature_ood(feature_zone, zone_row, feature_name)
3936: ood = NaN;
3937: if height(zone_row) == 0
3938:     return;
3939: end
3940: zone_name = zone_row.zone(1);
3941: m = feature_zone.zone == zone_name & feature_zone.feature == string(feature_name);
3942: if any(m)
3943:     ood = feature_zone.pct_abs_z_gt_3(find(m, 1, 'first'));
3944: end
3945: end
3946:
3947: function zones = local_get_zones(root, t)
3948: path_file = fullfile(root, 'data', 'paths', 'path_industrial_lite.mat');
3949: if exist(path_file, 'file') == 2
3950:     S = load(path_file, 'ref');
3951:     if isfield(S, 'ref') && isfield(S.ref, 'meta') && isfield(S.ref.meta, 'zones')
3952:         zones = S.ref.meta.zones;
3953:         return;
3954:     end
3955: end
3956: zones = struct();
3957: zones.startup = [min(t), 10];
3958: zones.golden_test = [10, 50];
3959: zones.pure_turn = [50, 78];
3960: zones.pure_slope = [78, 98];
```

```
zones.composite = [98, 118];
zones.closure = [118, max(t)];
end

function data = local_resample(logs, name, t)
ts = local_timeseries(logs, name);
data = squeeze(double(ts.Data));
if isvector(data)
    data = data(:);
else
    data = reshape(data, [], size(data, ndims(data)));
    data = data(:);
end
tt = double(ts.Time(:));
if numel(tt) ~= numel(data)
    data = squeeze(double(ts.Data));
    data = reshape(data, [], numel(tt)).';
end
if numel(tt) == numel(t) && max(abs(tt - t)) < 1e-12
    return;
end
data = interp1(tt, data, t, 'linear', 'extrap');
end

function data = local_data(logs, name)
ts = local_timeseries(logs, name);
data = squeeze(double(ts.Data));
end

function t = local_time(logs, name)
ts = local_timeseries(logs, name);
t = double(ts.Time(:));
end

function ts = local_timeseries(logs, name)
hit = [];
for i = 1:logs.numElements
    el = logs.getElement(i);
    if strcmp(el.Name, name)
        hit(end + 1) = i; %#ok<AGROW>
    end
end
if isempty(hit)
    error('ModernTCN:MissingLogSignal', 'logsout missing signal: %s', name);
end
% Duplicate names exist for some diag signals. Prefer the last logged signal,
% which is the high-rate diagnostic line in this model.
ts = logs.getElement(hit(end)).Values;
end

function local_write_report(report_file, result)
fid = fopen(report_file, 'w');
if fid < 0
    warning('ModernTCN:ReportFailed', 'Cannot write report: %s', report_file);
    return;
end
cleanup = onCleanup(@() fclose(fid));

fprintf(fid, '# ModernTCN Closed-loop Diagnostic\n\n');
fprintf(fid, '- out file: `%s`\n', result.out_file);
```

```
4021: fprintf(fid, '- dataset: `%s`\n\n', result.dataset_file);
4022:
4023: fprintf(fid, '## Decision Summary\n\n');
4024: fprintf(fid, '- Keep ModernTCN output diagnostic-only for now; do not feed `theta_hat` or labels
into MPC scheduling yet.\n');
4025: fprintf(fid, '- The controller is not the current bottleneck: tracking errors stay small in the
neutral-control run, while classifier errors concentrate in path zones.\n');
4026: fprintf(fid, '- Main data gap: low-load flat turns are being confused with slope, and turn labels
are missed in flat/composite turn zones.\n');
4027: fprintf(fid, '- Next training revision should add targeted short samples for flat low-load turns
and mild slope-turn composites, then rerun full-test and this closed-loop diagnostic.\n\n');
4028:
4029: fprintf(fid, '## Readiness Checks\n\n');
4030: A = result.assessment;
4031: fprintf(fid, '| check | value | threshold | pass | comment |\n');
4032: fprintf(fid, '|---|---:|---:|---:|---|\n');
4033: for i = 1:height(A)
4034:     if A.pass(i)
4035:         pass_text = "yes";
4036:     else
4037:         pass_text = "no";
4038:     end
4039:     fprintf(fid, '| %s | %.3f | %s | %s | %s |\n', ...
4040:         A.check(i), A.value(i), A.threshold(i), pass_text, A.comment(i));
4041: end
4042: fprintf(fid, '\n');
4043:
4044: fprintf(fid, '## Per-zone Metrics\n\n');
4045: fprintf(fid, '| zone | ey rmse | epsi rmse | ev rmse | theta MAE deg | main acc | turn acc | pred
main 1/2/3 | pred turn -1/0/1 |\n');
4046: fprintf(fid, '|---|---:|---:|---:|---:|---:|---:|---:|---:|\n');
4047: T = result.zone_table;
4048: for i = 1:height(T)
4049:     fprintf(fid, '| %s | %.4f | %.4f | %.4f | %.3f | %.1f | %.1f | %.1f/%.1f/%.1f | %.1f/%.1f/%.1f
|\n', ...
4050:         T.zone(i), T.ey_rmse(i), T.epsi_rmse(i), T.ev_rmse(i), ...
4051:         T.theta_mae_deg(i), T.main_acc_pct(i), T.turn_acc_pct(i), ...
4052:         T.pred_main_1_pct(i), T.pred_main_2_pct(i), T.pred_main_3_pct(i), ...
4053:         T.pred_turn_m1_pct(i), T.pred_turn_0_pct(i), T.pred_turn_1_pct(i));
4054: end
4055:
4056: fprintf(fid, '\n## Top Feature Z-score by Zone\n\n');
4057: F = result.feature_zone;
4058: zones = unique(F.zone, 'stable');
4059: for zi = 1:numel(zones)
4060:     zname = zones(zi);
4061:     Fz = F(F.zone == zname, :);
4062:     Fz = sortrows(Fz, 'p95_abs_z', 'descend');
4063:     fprintf(fid, '### %s\n\n', zname);
4064:     fprintf(fid, '| feature | median z | p95 abs z | max abs z | pct abs z > 3 |\n');
4065:     fprintf(fid, '|---|---:|---:|---:|---:|\n');
4066:     n = min(8, height(Fz));
4067:     for i = 1:n
4068:         fprintf(fid, '| %s | %.3f | %.3f | %.3f | %.1f |\n', ...
4069:             Fz.feature(i), Fz.median_z(i), Fz.p95_abs_z(i), ...
4070:             Fz.max_abs_z(i), Fz.pct_abs_z_gt_3(i));
4071:     end
4072:     fprintf(fid, '\n');
4073: end
4074:
4075: fprintf(fid, '## Feature Distribution Compare\n\n');
4076: groups = unique(result.feature_compare.group, 'stable');
4077: for gi = 1:numel(groups)
4078:     g = groups(gi);
4079:     G = result.feature_compare(result.feature_compare.group == g, :);
4080:     G = sortrows(G, 'p95_abs_z', 'descend');
```

```
4081:     fprintf(fid, '### %s\n\n', g);
4082:     fprintf(fid, '| feature | median z | p95 abs z | max abs z |\n');
4083:     fprintf(fid, '|---|---:|---:|---:|\n');
4084:     n = min(8, height(G));
4085:     for i = 1:n
4086:         fprintf(fid, '| %s | %.3f | %.3f | %.3f |\n', ...
4087:             G.feature(i), G.median_z(i), G.p95_abs_z(i), G.max_abs_z(i));
4088:     end
4089:     fprintf(fid, '\n');
4090: end
4091: end
4092:
4093: function y = local_rms(x)
4094: x = x(isfinite(x));
4095: if isempty(x)
4096:     y = NaN;
4097: else
4098:     y = sqrt(mean(x.^2));
4099: end
4100: end
4101:
4102: function v = local_field_or_default(s, field_name, default_value)
4103: if isstruct(s) && isfield(s, field_name) && ~isempty(s.(field_name))
4104:     v = s.(field_name);
4105: else
4106:     v = default_value;
4107: end
4108: end
4109:
4110: function root = local_project_root()
4111: if exist('project_root', 'file') == 2
4112:     root = project_root();
4113: else
4114:     root = pwd;
4115: end
4116: end
4117:
4118: ===== FILE: src/ModernTCN/ModernTCN_replay_closed_loop_yraw.m =====
4119: function result = ModernTCN_replay_closed_loop_yraw(out_file, max_steps, cfg)
4120: %MODERNTCN_REPLAY_CLOSED_LOOP_YRAW Replay logged y_raw through a ModernTCN wrapper.
4121: %
4122: % This diagnostic does not rerun or modify the Simulink model. It reuses
4123: % diag.y_raw from an existing out.mat and calls the same
4124: % ModernTCN_State_Classifier_sim(y_raw, reset) entry used by Simulink.
4125: %
4126: % Optional cfg fields:
4127: %   seed      : default 21.
4128: %   run_tag   : results/modern_tcn/<run_tag>/modern_tcn_seed<seed>.onnx.
4129: %   onnx_file : explicit ONNX path, overrides run_tag.
4130:
4131: if nargin < 1 || isempty(out_file)
4132:     out_file = fullfile(local_project_root(), 'out.mat');
4133: end
4134: if nargin < 2 || isempty(max_steps)
4135:     max_steps = 0;
4136: end
4137: if nargin < 3 || isempty(cfg)
4138:     cfg = struct();
4139: end
4140: if exist('init_project', 'file') == 2
```

```
4141:     init_project();
4142: end
4143:
4144: root = local_project_root();
4145: cfg = local_normalize_cfg(root, cfg);
4146: params = parameters();
4147: assignin('base', 'params', params);
4148: [had_sim_cfg, old_sim_cfg] = local_install_sim_cfg(cfg);
4149: cleanup_cfg = onCleanup(@() local_restore_sim_cfg(had_sim_cfg, old_sim_cfg)); %#ok<NASGU>
4150:
4151: S = load(out_file, 'logsout');
4152: logs = S.logsout;
4153:
4154: t = local_time(logs, 'diag.y_raw');
4155: y_raw = local_data(logs, 'diag.y_raw');
4156: theta_ground = local_resample(logs, 'diag.theta_ground', t);
4157: omega_ref = local_resample(logs, 'diag.omega_ref', t);
4158:
4159: if max_steps > 0
4160:     n = min(max_steps, numel(t));
4161:     t = t(1:n);
4162:     y_raw = y_raw(1:n, :);
4163:     theta_ground = theta_ground(1:n);
4164:     omega_ref = omega_ref(1:n);
4165: else
4166:     n = numel(t);
4167: end
4168:
4169: theta_hat = nan(n, 1);
4170: label_main = nan(n, 1);
4171: label_turn = nan(n, 1);
4172: conf_main = nan(n, 1);
4173: label_main_raw = nan(n, 1);
4174: label_turn_raw = nan(n, 1);
4175: theta_hat_raw = nan(n, 1);
4176: conf_turn = nan(n, 1);
4177: main_prob = nan(n, 3);
4178: turn_prob = nan(n, 3);
4179: features = nan(n, 19);
4180: features_norm = nan(n, 19);
4181:
4182: ModernTCN_State_Classifier_sim(y_raw(1, :).', 1);
4183: fprintf('[ModernTCN replay] steps=%d | out=%s\n', n, out_file);
4184: fprintf('  onnx=%s\n', cfg.onnx_file);
4185: for k = 1:n
4186:     [theta_hat(k), label_main(k), label_turn(k), conf_main(k)] = ...
4187:         ModernTCN_State_Classifier_sim(y_raw(k, :).', 0);
4188:     debug_out = evalin('base', 'modern_tcn_out_temp');
4189:     [label_main_raw(k), label_turn_raw(k), theta_hat_raw(k), conf_turn(k), ...
4190:         main_prob(k, :), turn_prob(k, :), features(k, :), features_norm(k, :)] = ...
4191:         local_debug_fields(debug_out);
4192:     if mod(k, 1000) == 0 || k == n
4193:         fprintf('  replayed %d/%d\n', k, n);
4194:     end
4195: end
4196:
4197: truth02 = local_make_truth(theta_ground, omega_ref, 0.02);
4198: truth05 = local_make_truth(theta_ground, omega_ref, 0.05);
4199: zones = local_get_zones(root, t);
4200: zone_names = fieldnames(zones);
```

```
rows = repmat(local_empty_zone_row(), numel(zone_names), 1);
for i = 1:numel(zone_names)
    name = zone_names{i};
    tr = zones.(name);
    mask = t >= tr(1) & t < tr(2);
    rows(i) = local_zone_row(name, tr, mask, theta_hat, theta_ground, ...
        label_main, label_turn, label_main_raw, label_turn_raw, conf_main, ...
        conf_turn, turn_prob, truth02, truth05, omega_ref);
end
zone_table = struct2table(rows);

out_dir = fullfile(root, 'results', 'modern_tcn', 'closed_loop_replay');
if exist(out_dir, 'dir') ~= 7
    mkdir(out_dir);
end
[~, base_name, ~] = fileparts(out_file);
report_tag = local_report_tag(cfg);
mat_file = fullfile(out_dir, sprintf('%s_modern_tcn_replay_%s.mat', base_name, report_tag));
report_file = fullfile(out_dir, sprintf('%s_modern_tcn_replay_%s_report.md', base_name,
report_tag));

result = struct();
result.out_file = out_file;
result.cfg = cfg;
result.onnx_file = cfg.onnx_file;
result.zone_table = zone_table;
result.t = t;
result.theta_hat = theta_hat;
result.theta_hat_raw = theta_hat_raw;
result.label_main = label_main;
result.label_main_raw = label_main_raw;
result.label_turn = label_turn;
result.label_turn_raw = label_turn_raw;
result.conf_main = conf_main;
result.conf_turn = conf_turn;
result.main_prob = main_prob;
result.turn_prob = turn_prob;
result.features = features;
result.features_norm = features_norm;
result.theta_ground = theta_ground;
result.omega_ref = omega_ref;
result.truth02 = truth02;
result.truth05 = truth05;
result.mat_file = mat_file;
result.report_file = report_file;

save(mat_file, 'result');
local_write_report(report_file, result);

fprintf('[ModernTCN replay] report: %s\n', report_file);
disp(zone_table(:, {'zone','theta_mae_deg','main_acc_pct','turn_acc05_pct', ...
    'left_recall05_pct','left_raw_recall05_pct','pred_main_3_pct', ...
    'pred_turn_1_pct','pred_turn_raw_1_pct','left_prob_p90'}));
end

function cfg = local_normalize_cfg(root, cfg)
default_cfg = ModernTCN_default_config(root);
if ~isfield(cfg, 'seed') || isempty(cfg.seed)
    cfg.seed = default_cfg.seed;
end
if ~isfield(cfg, 'run_tag')
```

```
4261:     cfg.run_tag = default_cfg.run_tag;
4262: end
4263: if ~isfield(cfg, 'onnx_file')
4264:     cfg.onnx_file = '';
4265: end
4266: if ~isfield(cfg, 'dataset_file') || isempty(cfg.dataset_file)
4267:     cfg.dataset_file = default_cfg.dataset_file;
4268: end
4269: if isempty(cfg.onnx_file)
4270:     if ~isempty(cfg.run_tag)
4271:         cfg.onnx_file = fullfile(root, 'results', 'modern_tcn', char(cfg.run_tag), ...
4272:             sprintf('modern_tcn_seed%d.onnx', cfg.seed));
4273:     else
4274:         cfg.onnx_file = default_cfg.onnx_file;
4275:     end
4276: end
4277: end
4278:
4279: function [had_cfg, old_cfg] = local_install_sim_cfg(cfg)
4280: had_cfg = evalin('base', 'exist(''modern_tcn_sim_cfg'', ''var'')==1');
4281: if had_cfg
4282:     old_cfg = evalin('base', 'modern_tcn_sim_cfg');
4283: else
4284:     old_cfg = [];
4285: end
4286: assignin('base', 'modern_tcn_sim_cfg', cfg);
4287: end
4288:
4289: function local_restore_sim_cfg(had_cfg, old_cfg)
4290: if had_cfg
4291:     assignin('base', 'modern_tcn_sim_cfg', old_cfg);
4292: else
4293:     evalin('base', 'clear modern_tcn_sim_cfg');
4294: end
4295: end
4296:
4297: function [label_main_raw, label_turn_raw, theta_hat_raw, conf_turn, main_prob, turn_prob, features,
features_norm] = ...
4298:     local_debug_fields(debug_out)
4299: label_main_raw = NaN;
4300: label_turn_raw = NaN;
4301: theta_hat_raw = NaN;
4302: conf_turn = NaN;
4303: main_prob = nan(1, 3);
4304: turn_prob = nan(1, 3);
4305: features = nan(1, 19);
4306: features_norm = nan(1, 19);
4307: if ~isstruct(debug_out)
4308:     return;
4309: end
4310: if isfield(debug_out, 'label_main_raw'); label_main_raw = double(debug_out.label_main_raw); end
4311: if isfield(debug_out, 'label_turn_raw'); label_turn_raw = double(debug_out.label_turn_raw); end
4312: if isfield(debug_out, 'theta_hat_raw'); theta_hat_raw = double(debug_out.theta_hat_raw); end
4313: if isfield(debug_out, 'conf_turn'); conf_turn = double(debug_out.conf_turn); end
4314: if isfield(debug_out, 'main_prob')
4315:     p = double(debug_out.main_prob(:)).';
4316:     main_prob(1:min(3, numel(p))) = p(1:min(3, numel(p)));
4317: end
4318: if isfield(debug_out, 'turn_prob')
4319:     p = double(debug_out.turn_prob(:)).';
4320:     turn_prob(1:min(3, numel(p))) = p(1:min(3, numel(p)));
```

```
4321: end
4322: if isfield(debug_out, 'features')
4323:     p = double(debug_out.features(:)).';
4324:     features(1:min(19, numel(p))) = p(1:min(19, numel(p)));
4325: end
4326: if isfield(debug_out, 'features_norm')
4327:     p = double(debug_out.features_norm(:)).';
4328:     features_norm(1:min(19, numel(p))) = p(1:min(19, numel(p)));
4329: end
4330: end
4331:
4332: function row = local_empty_zone_row()
4333: row = struct('zone', "", 't0', NaN, 't1', NaN, ...
4334:     'theta_mae_deg', NaN, 'theta_rmse_deg', NaN, 'theta_peak_deg', NaN, ...
4335:     'theta_hat_min_deg', NaN, 'theta_hat_max_deg', NaN, ...
4336:     'main_acc_pct', NaN, ...
4337:     'turn_acc_pct', NaN, 'turn_acc02_pct', NaN, 'turn_acc05_pct', NaN, ...
4338:     'turn_raw_acc05_pct', NaN, ...
4339:     'left_recall02_pct', NaN, 'left_recall05_pct', NaN, ...
4340:     'left_raw_recall05_pct', NaN, 'right_recall05_pct', NaN, ...
4341:     'right_raw_recall05_pct', NaN, ...
4342:     'true_main_1_pct', NaN, 'true_main_3_pct', NaN, ...
4343:     'pred_main_1_pct', NaN, 'pred_main_2_pct', NaN, 'pred_main_3_pct', NaN, ...
4344:     'true_turn_m1_pct', NaN, 'true_turn_0_pct', NaN, 'true_turn_1_pct', NaN, ...
4345:     'true_turn05_m1_pct', NaN, 'true_turn05_0_pct', NaN, 'true_turn05_1_pct', NaN, ...
4346:     'pred_turn_m1_pct', NaN, 'pred_turn_0_pct', NaN, 'pred_turn_1_pct', NaN, ...
4347:     'pred_turn_raw_m1_pct', NaN, 'pred_turn_raw_0_pct', NaN, 'pred_turn_raw_1_pct', NaN, ...
4348:     'left_prob_median', NaN, 'left_prob_p90', NaN, ...
4349:     'straight_prob_median', NaN, 'conf_median', NaN, 'conf_p10', NaN, ...
4350:     'omega_ref_min', NaN, 'omega_ref_max', NaN);
4351: end
4352:
4353: function row = local_zone_row(name, tr, mask, theta_hat, theta_ground, ...
4354:     label_main, label_turn, label_main_raw, label_turn_raw, conf_main, ...
4355:     conf_turn, turn_prob, truth02, truth05, omega_ref)
4356: row = local_empty_zone_row();
4357: row.zone = string(name);
4358: row.t0 = tr(1);
4359: row.t1 = tr(2);
4360: if nnz(mask) < 2
4361:     return;
4362: end
4363:
4364: th_err = theta_hat(mask) - theta_ground(mask);
4365: lm = round(label_main(mask));
4366: lt = round(label_turn(mask));
4367: ltr = round(label_turn_raw(mask));
4368: tm = truth05.main(mask);
4369: tt02 = truth02.turn(mask);
4370: tt05 = truth05.turn(mask);
4371: prob = turn_prob(mask, :);
4372:
4373: row.theta_mae_deg = rad2deg(mean(abs(th_err), 'omitnan'));
4374: row.theta_rmse_deg = rad2deg(local_rms(th_err));
4375: row.theta_peak_deg = rad2deg(max(abs(th_err)));
4376: row.theta_hat_min_deg = rad2deg(min(theta_hat(mask)));
4377: row.theta_hat_max_deg = rad2deg(max(theta_hat(mask)));
4378: row.main_acc_pct = 100 * mean(lm == tm, 'omitnan');
4379: row.turn_acc_pct = 100 * mean(lt == tt02, 'omitnan');
4380: row.turn_acc02_pct = row.turn_acc_pct;
```

```
4381: row.turn_acc05_pct = 100 * mean(lt == tt05, 'omitnan');
4382: row.turn_raw_acc05_pct = 100 * mean(ltr == tt05, 'omitnan');
4383: row.left_recall02_pct = local_recall_pct(lt, tt02, 1);
4384: row.left_recall05_pct = local_recall_pct(lt, tt05, 1);
4385: row.left_raw_recall05_pct = local_recall_pct(ltr, tt05, 1);
4386: row.right_recall05_pct = local_recall_pct(lt, tt05, -1);
4387: row.right_raw_recall05_pct = local_recall_pct(ltr, tt05, -1);
4388: row.true_main_1_pct = 100 * mean(tm == 1, 'omitnan');
4389: row.true_main_3_pct = 100 * mean(tm == 3, 'omitnan');
4390: row.pred_main_1_pct = 100 * mean(lm == 1, 'omitnan');
4391: row.pred_main_2_pct = 100 * mean(lm == 2, 'omitnan');
4392: row.pred_main_3_pct = 100 * mean(lm == 3, 'omitnan');
4393: row.true_turn_m1_pct = 100 * mean(tt02 == -1, 'omitnan');
4394: row.true_turn_0_pct = 100 * mean(tt02 == 0, 'omitnan');
4395: row.true_turn_1_pct = 100 * mean(tt02 == 1, 'omitnan');
4396: row.true_turn05_m1_pct = 100 * mean(tt05 == -1, 'omitnan');
4397: row.true_turn05_0_pct = 100 * mean(tt05 == 0, 'omitnan');
4398: row.true_turn05_1_pct = 100 * mean(tt05 == 1, 'omitnan');
4399: row.pred_turn_m1_pct = 100 * mean(lt == -1, 'omitnan');
4400: row.pred_turn_0_pct = 100 * mean(lt == 0, 'omitnan');
4401: row.pred_turn_1_pct = 100 * mean(lt == 1, 'omitnan');
4402: row.pred_turn_raw_m1_pct = 100 * mean(ltr == -1, 'omitnan');
4403: row.pred_turn_raw_0_pct = 100 * mean(ltr == 0, 'omitnan');
4404: row.pred_turn_raw_1_pct = 100 * mean(ltr == 1, 'omitnan');
4405: row.left_prob_median = median(prob(:, 3), 'omitnan');
4406: row.left_prob_p90 = local_prctile(prob(:, 3), 90);
4407: row.straight_prob_median = median(prob(:, 2), 'omitnan');
4408: row.conf_median = median(conf_main(mask), 'omitnan');
4409: row.conf_p10 = local_prctile(conf_main(mask), 10);
4410: row.omega_ref_min = min(omega_ref(mask));
4411: row.omega_ref_max = max(omega_ref(mask));
4412:
4413: % Keep the raw main label read alive in reports if a future diagnostic needs it.
4414: if all(~isfinite(label_main_raw(mask))) && all(~isfinite(conf_turn(mask)))
4415:     return;
4416: end
4417: end
4418:
4419: function truth = local_make_truth(theta_ground, omega_ref, omega_thresh)
4420: truth = struct();
4421: truth.main = ones(size(theta_ground));
4422: truth.main(abs(theta_ground) > deg2rad(2.0)) = 3;
4423: truth.turn = zeros(size(omega_ref));
4424: truth.turn(omega_ref > omega_thresh) = 1;
4425: truth.turn(omega_ref < -omega_thresh) = -1;
4426: truth.omega_thresh = omega_thresh;
4427: end
4428:
4429: function zones = local_get_zones(root, t)
4430: path_file = fullfile(root, 'data', 'paths', 'path_industrial_lite.mat');
4431: if exist(path_file, 'file') == 2
4432:     S = load(path_file, 'ref');
4433:     if isfield(S, 'ref') && isfield(S.ref, 'meta') && isfield(S.ref.meta, 'zones')
4434:         zones = S.ref.meta.zones;
4435:         return;
4436:     end
4437: end
4438: zones = struct();
4439: zones.startup = [min(t), 10];
4440: zones.golden_test = [10, 50];
```

```
zones.pure_turn = [50, 78];
zones.pure_slope = [78, 98];
zones.composite = [98, 118];
zones.closure = [118, max(t)];
end

function data = local_resample(logs, name, t)
ts = local_timeseries(logs, name);
data = squeeze(double(ts.Data));
if isvector(data)
    data = data(:);
else
    data = reshape(data, [], size(data, ndims(data)));
    data = data(:);
end
tt = double(ts.Time(:));
if numel(tt) ~= numel(data)
    data = squeeze(double(ts.Data));
    data = reshape(data, [], numel(tt)).';
end
if numel(tt) == numel(t) && max(abs(tt - t)) < 1e-12
    return;
end
data = interp1(tt, data, t, 'linear', 'extrap');
end

function data = local_data(logs, name)
ts = local_timeseries(logs, name);
data = squeeze(double(ts.Data));
end

function t = local_time(logs, name)
ts = local_timeseries(logs, name);
t = double(ts.Time(:));
end

function ts = local_timeseries(logs, name)
hit = [];
for i = 1:logs.numElements
    el = logs.getElement(i);
    if strcmp(el.Name, name)
        hit(end + 1) = i; %#ok<AGROW>
    end
end
if isempty(hit)
    error('ModernTCN:MissingLogSignal', 'logsout missing signal: %s', name);
end
ts = logs.getElement(hit(end)).Values;
end

function local_write_report(report_file, result)
fid = fopen(report_file, 'w');
if fid < 0
    warning('ModernTCN:ReportFailed', 'Cannot write report: %s', report_file);
    return;
end
cleanup = onCleanup(@() fclose(fid)); %#ok<NASGU>

fprintf(fid, '# ModernTCN y_raw Replay Diagnostic\n\n');
fprintf(fid, '- out file: `%s`\n', result.out_file);
```

```
4501: fprintf(fid, '- onnx: `%s`\n', result.onnx_file);
4502: fprintf(fid, '- turn truth thresholds: `0.02` and `0.05` rad/s; training labels use `0.05`
rad/s.\n\n');
4503: fprintf(fid, '| zone | theta MAE deg | main acc | turn acc 0.05 | left recall 0.05 | raw left
recall 0.05 | pred main 3 | pred turn 1 | raw pred turn 1 | left p90 |\n');
4504: fprintf(fid, '|---|---:|---:|---:|---:|---:|---:|---:|---:|---:|\n');
4505: T = result.zone_table;
4506: for i = 1:height(T)
4507:     fprintf(fid, '| %s | %.3f | %.1f | %.1f | %.1f | %.1f | %.1f | %.1f | %.3f |\n', ...
4508:         T.zone(i), T.theta_mae_deg(i), T.main_acc_pct(i), T.turn_acc05_pct(i), ...
4509:         T.left_recall05_pct(i), T.left_raw_recall05_pct(i), T.pred_main_3_pct(i), ...
4510:         T.pred_turn_1_pct(i), T.pred_turn_raw_1_pct(i), T.left_prob_p90(i));
4511: end
4512: fprintf(fid, '\n## Full Zone Table\n\n');
4513: fprintf(fid, 'Saved in `%s` as `result.zone_table`.\n', result.mat_file);
4514: end
4515:
4516: function y = local_rms(x)
4517: x = x(isfinite(x));
4518: if isempty(x)
4519:     y = NaN;
4520: else
4521:     y = sqrt(mean(x.^2));
4522: end
4523: end
4524:
4525: function p = local_recall_pct(pred, truth, cls)
4526: mask = truth == cls;
4527: if ~any(mask)
4528:     p = NaN;
4529: else
4530:     p = 100 * mean(pred(mask) == cls, 'omitnan');
4531: end
4532: end
4533:
4534: function p = local_prctile(x, q)
4535: x = x(isfinite(x));
4536: if isempty(x)
4537:     p = NaN;
4538: else
4539:     p = prctile(x, q);
4540: end
4541: end
4542:
4543: function tag = local_report_tag(cfg)
4544: if isfield(cfg, 'run_tag') && ~isempty(cfg.run_tag)
4545:     tag = char(cfg.run_tag);
4546: else
4547:     tag = 'current_model';
4548: end
4549: tag = regexprep(tag, '[^A-Za-z0-9_\-]+', '_');
4550: end
4551:
4552: function root = local_project_root()
4553: if exist('project_root', 'file') == 2
4554:     root = project_root();
4555: else
4556:     root = pwd;
4557: end
4558: end
4559:
4560: ===== FILE: src/Compare/benchmark_modern_tcn_onnx_runtime.py =====
```

```
4561: """Benchmark ModernTCN single-window ONNXRuntime inference latency.
4562:
4563: The benchmark intentionally uses batch size 1 and input shape [1, 128, 19],
4564: matching the online closed-loop deployment path.  It writes raw timings and a
4565: small summary under results/compare/realtime_benchmark by default.
4566: """
4567:
4568: from __future__ import annotations
4569:
4570: import argparse
4571: import csv
4572: import json
4573: import os
4574: import platform
4575: import statistics
4576: import sys
4577: import time
4578: from pathlib import Path
4579: from typing import Dict, List
4580:
4581: import h5py
4582: import numpy as np
4583:
4584:
4585: def parse_args() -> argparse.Namespace:
4586:     root = find_project_root()
4587:     default_out = root / "results" / "compare" / "realtime_benchmark"
4588:     default_onnx = (
4589:         root
4590:         / "results"
4591:         / "modern_tcn"
4592:         / "modern_tcn_theta10_uniform_h0_v2_seed21"
4593:         / "modern_tcn_seed21.onnx"
4594:     )
4595:     default_dataset = (
4596:         root
4597:         / "data"
4598:         / "tcn"
4599:         / "ModernTCN_dataset_agv_dualsteer_theta10_uniform_conf_h0_v2.mat"
4600:     )
4601:
4602:     p = argparse.ArgumentParser(description="ModernTCN ONNXRuntime latency benchmark")
4603:     p.add_argument("--onnx-file", type=Path, default=default_onnx)
4604:     p.add_argument("--dataset-file", type=Path, default=default_dataset)
4605:     p.add_argument("--out-dir", type=Path, default=default_out)
4606:     p.add_argument("--sample-count", type=int, default=512)
4607:     p.add_argument("--warmup", type=int, default=200)
4608:     p.add_argument("--repeat", type=int, default=5000)
4609:     p.add_argument("--provider", type=str, default="CPUExecutionProvider")
4610:     p.add_argument("--intra-op-num-threads", type=int, default=1)
4611:     return p.parse_args()
4612:
4613:
4614: def find_project_root(start: Path | None = None) -> Path:
4615:     if start is None:
4616:         start = Path(__file__).resolve()
4617:     for p in (start, *start.parents):
4618:         if (p / "init_project.m").exists() and (p / "data" / "tcn").exists():
4619:             return p
4620:     cwd = Path.cwd().resolve()
```

```
4621:     for p in (cwd, *cwd.parents):
4622:         if (p / "init_project.m").exists() and (p / "data" / "tcn").exists():
4623:             return p
4624:     raise FileNotFoundError("Could not locate project root.")
4625: 
4626: 
4627: def read_test_windows(dataset_file: Path, sample_count: int) -> np.ndarray:
4628:     if not dataset_file.exists():
4629:         raise FileNotFoundError(f"Dataset not found: {dataset_file}")
4630:     with h5py.File(dataset_file, "r") as f:
4631:         ds = f["dataset"]["X_test"]
4632:         if ds.ndim != 3:
4633:             raise ValueError(f"Expected dataset.X_test to be 3-D, got shape={ds.shape}")
4634:         n_total = int(ds.shape[2])
4635:         n = min(max(1, int(sample_count)), n_total)
4636:         indices = np.linspace(0, n_total - 1, n, dtype=np.int64)
4637:         raw = np.asarray(ds[:, :, indices], dtype=np.float32)
4638:     # MATLAB v7.3 stores this as [feature, time, window].  ONNX expects
4639:     # [batch, time, feature].
4640:     return np.transpose(raw, (2, 1, 0)).copy()
4641: 
4642: 
4643: def make_session(onnx_file: Path, provider: str, intra_op_num_threads: int):
4644:     try:
4645:         import onnxruntime as ort
4646:     except ImportError as exc:
4647:         raise SystemExit("onnxruntime is not installed in the active Python.") from exc
4648: 
4649:     if not onnx_file.exists():
4650:         raise FileNotFoundError(f"ONNX file not found: {onnx_file}")
4651: 
4652:     opts = ort.SessionOptions()
4653:     opts.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL
4654:     if intra_op_num_threads > 0:
4655:         opts.intra_op_num_threads = int(intra_op_num_threads)
4656:     session = ort.InferenceSession(str(onnx_file), sess_options=opts, providers=[provider])
4657:     return session, ort
4658: 
4659: 
4660: def run_benchmark(session, windows: np.ndarray, warmup: int, repeat: int) -> Dict[str, object]:
4661:     input_name = session.get_inputs()[0].name
4662:     output_names = [o.name for o in session.get_outputs()]
4663:     n = int(windows.shape[0])
4664: 
4665:     for i in range(int(warmup)):
4666:         x = windows[i % n : i % n + 1]
4667:         session.run(output_names, {input_name: x})
4668: 
4669:     elapsed_ms: List[float] = []
4670:     sample_idx: List[int] = []
4671:     for i in range(int(repeat)):
4672:         j = i % n
4673:         x = windows[j : j + 1]
4674:         t0 = time.perf_counter_ns()
4675:         session.run(output_names, {input_name: x})
4676:         t1 = time.perf_counter_ns()
4677:         elapsed_ms.append((t1 - t0) / 1e6)
4678:         sample_idx.append(j)
4679: 
4680:     arr = np.asarray(elapsed_ms, dtype=np.float64)
```

```
4681:     return {
4682:         "input_name": input_name,
4683:         "output_names": output_names,
4684:         "sample_idx": sample_idx,
4685:         "elapsed_ms": elapsed_ms,
4686:         "summary": {
4687:             "n": int(arr.size),
4688:             "mean_ms": float(arr.mean()),
4689:             "std_ms": float(arr.std(ddof=0)),
4690:             "min_ms": float(arr.min()),
4691:             "p50_ms": float(np.percentile(arr, 50)),
4692:             "p95_ms": float(np.percentile(arr, 95)),
4693:             "p99_ms": float(np.percentile(arr, 99)),
4694:             "max_ms": float(arr.max()),
4695:         },
4696:     }
4697:
4698:
4699: def write_outputs(args: argparse.Namespace, ort, windows: np.ndarray, result: Dict[str, object]) ->
None:
4700:     out_dir = args.out_dir
4701:     out_dir.mkdir(parents=True, exist_ok=True)
4702:
4703:     raw_file = out_dir / "realtime_onnx_runtime_raw.csv"
4704:     summary_file = out_dir / "realtime_onnx_runtime_summary.csv"
4705:     metadata_file = out_dir / "realtime_onnx_runtime_metadata.json"
4706:     report_file = out_dir / "realtime_onnx_runtime_report.md"
4707:
4708:     elapsed = result["elapsed_ms"]
4709:     sample_idx = result["sample_idx"]
4710:     with raw_file.open("w", newline="", encoding="utf-8") as f:
4711:         w = csv.writer(f)
4712:         w.writerow(["iteration", "sample_index", "elapsed_ms"])
4713:         for i, (j, ms) in enumerate(zip(sample_idx, elapsed), start=1):
4714:             w.writerow([i, j, f"{ms:.9g}"])
4715:
4716:     s = result["summary"]
4717:     row = {
4718:         "metric": "onnxruntime_single_window",
4719:         "n": s["n"],
4720:         "mean_ms": s["mean_ms"],
4721:         "std_ms": s["std_ms"],
4722:         "min_ms": s["min_ms"],
4723:         "p50_ms": s["p50_ms"],
4724:         "p95_ms": s["p95_ms"],
4725:         "p99_ms": s["p99_ms"],
4726:         "max_ms": s["max_ms"],
4727:         "batch_size": 1,
4728:         "sample_count": int(windows.shape[0]),
4729:         "warmup": int(args.warmup),
4730:         "repeat": int(args.repeat),
4731:         "provider": args.provider,
4732:         "intra_op_num_threads": int(args.intra_op_num_threads),
4733:         "onnx_file": str(args.onnx_file),
4734:         "dataset_file": str(args.dataset_file),
4735:     }
4736:     with summary_file.open("w", newline="", encoding="utf-8") as f:
4737:         w = csv.DictWriter(f, fieldnames=list(row.keys()))
4738:         w.writeheader()
4739:         w.writerow(row)
4740:
```

```
    metadata = {
        "python": sys.version,
        "python_executable": sys.executable,
        "platform": platform.platform(),
        "processor": platform.processor(),
        "cpu_count": os.cpu_count(),
        "onnxruntime_version": ort.__version__,
        "numpy_version": np.__version__,
        "input_name": result["input_name"],
        "output_names": result["output_names"],
        "windows_shape": list(windows.shape),
    }
    metadata_file.write_text(json.dumps(metadata, indent=2), encoding="utf-8")

    with report_file.open("w", encoding="utf-8") as f:
        f.write("# ModernTCN ONNXRuntime Latency Benchmark\n\n")
        f.write(f"- onnx: `{args.onnx_file}`\n")
        f.write(f"- dataset: `{args.dataset_file}`\n")
        f.write(f"- provider: `{args.provider}`\n")
        f.write(f"- batch size: `1`\n")
        f.write(f"- warmup: `{args.warmup}`\n")
        f.write(f"- repeat: `{args.repeat}`\n\n")
        f.write("| metric | value (ms) |\n")
        f.write("|---|---:|\n")
        for key in ["mean_ms", "p50_ms", "p95_ms", "p99_ms", "max_ms"]:
            f.write(f"| {key} | {s[key]:.6g} |\n")

    print(f"[onnx benchmark] summary: {summary_file}")
    print(f"[onnx benchmark] report:  {report_file}")


def main() -> None:
    args = parse_args()
    windows = read_test_windows(args.dataset_file, args.sample_count)
    session, ort = make_session(args.onnx_file, args.provider, args.intra_op_num_threads)
    result = run_benchmark(session, windows, args.warmup, args.repeat)
    write_outputs(args, ort, windows, result)


if __name__ == "__main__":
    main()

===== FILE: src/Compare/run_realtime_benchmark.m =====
function result = run_realtime_benchmark(cfg)
%RUN_REALTIME_BENCHMARK Summarize ModernTCN closed-loop timing metrics.
%
% This script combines:
%   1) ONNXRuntime single-window timing written by benchmark_modern_tcn_onnx_runtime.py
%   2) MATLAB online replay timing for ModernTCN_state_classifier('update', ...)
%   3) MPC solve-time samples already logged as diag_solve_time_ms
%   4) Simulink wall-time metadata from existing closed-loop outputs

if nargin < 1 || ~isstruct(cfg)
    cfg = struct();
end

init_project();
root = project_root();
cfg = local_defaults(cfg, root);
local_ensure_dir(cfg.out_root);
```

```
4801:
4802: params = parameters();
4803: Ts = local_field_or_default(params, 'Ts', 0.01);
4804: Ts_ms = 1000 * double(Ts);
4805:
4806: fprintf('[realtime] output root: %s\n', cfg.out_root);
4807: fprintf('[realtime] Ts = %.6g ms\n', Ts_ms);
4808:
4809: [replay_raw, replay_summary] = local_run_matlab_replay(cfg, params, Ts_ms);
4810: [mpc_raw, mpc_summary, wall_summary] = local_collect_closed_loop_timing(cfg, Ts_ms);
4811: onnx_summary = local_read_onnx_summary(cfg.onnx_summary_file, Ts_ms);
4812: summary = local_build_summary(onnx_summary, replay_summary, mpc_summary, wall_summary, Ts_ms);
4813:
4814: raw_replay_file = fullfile(cfg.out_root, 'realtime_matlab_replay_raw.csv');
4815: replay_summary_file = fullfile(cfg.out_root, 'realtime_matlab_replay_summary.csv');
4816: mpc_raw_file = fullfile(cfg.out_root, 'realtime_mpc_solve_raw.csv');
4817: mpc_summary_file = fullfile(cfg.out_root, 'realtime_mpc_solve_summary.csv');
4818: wall_summary_file = fullfile(cfg.out_root, 'realtime_simulink_wall_summary.csv');
4819: summary_file = fullfile(cfg.out_root, 'realtime_summary.csv');
4820: report_file = fullfile(cfg.out_root, 'realtime_report.md');
4821: mat_file = fullfile(cfg.out_root, 'realtime_result.mat');
4822:
4823: if ~isempty(replay_raw), writetable(replay_raw, raw_replay_file); end
4824: if ~isempty(replay_summary), writetable(replay_summary, replay_summary_file); end
4825: if ~isempty(mpc_raw), writetable(mpc_raw, mpc_raw_file); end
4826: if ~isempty(mpc_summary), writetable(mpc_summary, mpc_summary_file); end
4827: if ~isempty(wall_summary), writetable(wall_summary, wall_summary_file); end
4828: if ~isempty(summary), writetable(summary, summary_file); end
4829:
4830: result = struct();
4831: result.timestamp = datestr(now, 'yyyy-mm-dd HH:MM:SS');
4832: result.cfg = cfg;
4833: result.Ts = Ts;
4834: result.Ts_ms = Ts_ms;
4835: result.onnx_summary = onnx_summary;
4836: result.replay_raw = replay_raw;
4837: result.replay_summary = replay_summary;
4838: result.mpc_raw = mpc_raw;
4839: result.mpc_summary = mpc_summary;
4840: result.wall_summary = wall_summary;
4841: result.summary = summary;
4842: result.summary_file = summary_file;
4843: result.report_file = report_file;
4844: result.mat_file = mat_file;
4845:
4846: save(mat_file, 'result', '-v7.3');
4847: local_write_report(report_file, result);
4848:
4849: fprintf('[realtime] summary: %s\n', summary_file);
4850: fprintf('[realtime] report:  %s\n', report_file);
4851: end
4852:
4853: function cfg = local_defaults(cfg, root)
4854: cfg.out_root = local_cfg(cfg, 'out_root', ...
4855:     fullfile(root, 'results', 'compare', 'realtime_benchmark'));
4856: cfg.warmup_sec = local_cfg(cfg, 'warmup_sec', 1.0);
4857: cfg.predict_warmup_count = local_cfg(cfg, 'predict_warmup_count', 20);
4858: cfg.max_replay_steps = local_cfg(cfg, 'max_replay_steps', 0);
4859: cfg.onnx_summary_file = local_cfg(cfg, 'onnx_summary_file', ...
4860:     fullfile(cfg.out_root, 'realtime_onnx_runtime_summary.csv'));
```

```
4861:
4862: default_files = { ...
4863:     fullfile(root, 'results', 'compare', 'multipath_closed_loop', ...
4864:         'path_closed_loop_long_updown_theta10_v1', 'ModernTCN_out.mat')
4865:     fullfile(root, 'results', 'compare', 'multipath_closed_loop', ...
4866:         'path_closed_loop_sharp_turn_transition_theta10_v1', 'ModernTCN_out.mat')};
4867: default_files = default_files(cellfun(@(p) exist(p, 'file') == 2, default_files));
4868: if isempty(default_files)
4869:     d = dir(fullfile(root, 'results', 'compare', '**', 'ModernTCN_out.mat'));
4870:     default_files = arrayfun(@(x) fullfile(x.folder, x.name), d, 'UniformOutput', false);
4871: end
4872: cfg.closed_loop_files = local_cfg(cfg, 'closed_loop_files', default_files);
4873: if ischar(cfg.closed_loop_files) || isstring(cfg.closed_loop_files)
4874:     cfg.closed_loop_files = cellstr(cfg.closed_loop_files);
4875: end
4876: cfg.replay_file = local_cfg(cfg, 'replay_file', cfg.closed_loop_files{1});
4877: end
4878:
4879: function v = local_cfg(cfg, name, default_value)
4880: if isstruct(cfg) && isfield(cfg, name) && ~isempty(cfg.(name))
4881:     v = cfg.(name);
4882: else
4883:     v = default_value;
4884: end
4885: end
4886:
4887: function local_ensure_dir(d)
4888: if exist(d, 'dir') ~= 7
4889:     mkdir(d);
4890: end
4891: end
4892:
4893: function [raw_table, summary_table] = local_run_matlab_replay(cfg, params, Ts_ms)
4894: raw_table = table();
4895: summary_table = table();
4896: if isempty(cfg.replay_file) || exist(cfg.replay_file, 'file') ~= 2
4897:     warning('run_realtime_benchmark:MissingReplay', ...
4898:         'Replay file not found: %s', string(cfg.replay_file));
4899:     return;
4900: end
4901:
4902: S = load(cfg.replay_file, 'logsout');
4903: yts = local_get_signal(S.logsout, 'diag.y_raw');
4904: if isempty(yts)
4905:     warning('run_realtime_benchmark:MissingYRaw', ...
4906:         'diag.y_raw not found in %s', cfg.replay_file);
4907:     return;
4908: end
4909:
4910: t = double(yts.Time(:));
4911: Y = local_timeseries_rows(yts.Data, numel(t));
4912: n = size(Y, 1);
4913: if cfg.max_replay_steps > 0
4914:     n = min(n, cfg.max_replay_steps);
4915:     Y = Y(1:n, :);
4916:     t = t(1:n);
4917: end
4918:
4919: clear ModernTCN_state_classifier ModernTCN_load_predictor ModernTCN_predict_window
4920: init_timer = tic;
```

```
4921: state = ModernTCN_state_classifier('init', params);
4922: init_ms = 1000 * toc(init_timer);
4923:
4924: elapsed_ms = nan(n, 1);
4925: did_predict = false(n, 1);
4926: ready = false(n, 1);
4927: buffer_count = nan(n, 1);
4928: predict_index = nan(n, 1);
4929: predict_count = 0;
4930:
4931: for i = 1:n
4932:     yi = double(Y(i, :)).';
4933:     one_timer = tic;
4934:     [state, out] = ModernTCN_state_classifier('update', state, yi);
4935:     elapsed_ms(i) = 1000 * toc(one_timer);
4936:     if isstruct(out) && isfield(out, 'debug')
4937:         dbg = out.debug;
4938:         if isfield(dbg, 'did_predict'), did_predict(i) = logical(dbg.did_predict); end
4939:         if isfield(dbg, 'ready'), ready(i) = logical(dbg.ready); end
4940:         if isfield(dbg, 'buffer_count'), buffer_count(i) = double(dbg.buffer_count); end
4941:     end
4942:     if did_predict(i)
4943:         predict_count = predict_count + 1;
4944:         predict_index(i) = predict_count;
4945:     end
4946: end
4947:
4948: raw_table = table( ...
4949:     repmat(string(cfg.replay_file), n, 1), (1:n).', t(:), elapsed_ms, ...
4950:     did_predict, ready, buffer_count, predict_index, ...
4951:     'VariableNames', {'source_file','step_index','time_s','elapsed_ms', ...
4952:     'did_predict','ready','buffer_count','predict_index'});
4953:
4954: warm_mask = t(:) >= double(cfg.warmup_sec);
4955: predict_steady_mask = warm_mask & did_predict & ...
4956:     predict_index > double(cfg.predict_warmup_count);
4957: rows = repmat(local_stats_row("", [], Ts_ms), 0, 1);
4958: rows(end+1) = local_stats_row("matlab_replay_update_all", elapsed_ms(warm_mask),
Ts_ms); %#ok<AGROW>
4959: rows(end+1) = local_stats_row("matlab_replay_update_predict_all", ...
4960:     elapsed_ms(warm_mask & did_predict), Ts_ms); %#ok<AGROW>
4961: rows(end+1) = local_stats_row("matlab_replay_update_predict_steady", ...
4962:     elapsed_ms(predict_steady_mask), Ts_ms); %#ok<AGROW>
4963: rows(end+1) = local_stats_row("matlab_replay_init_once", init_ms, Ts_ms); %#ok<AGROW>
4964: summary_table = struct2table(rows);
4965: summary_table.source_file = repmat(string(cfg.replay_file), height(summary_table), 1);
4966: summary_table.warmup_sec = repmat(double(cfg.warmup_sec), height(summary_table), 1);
4967: summary_table.predict_warmup_count = repmat(double(cfg.predict_warmup_count),
height(summary_table), 1);
4968: end
4969:
4970: function [raw_table, summary_table, wall_table] = local_collect_closed_loop_timing(cfg, Ts_ms)
4971: raw_table = table();
4972: summary_table = table();
4973: wall_table = table();
4974:
4975: all_values = [];
4976: all_sources = strings(0, 1);
4977: all_t = [];
4978: wall_rows = repmat(local_wall_row(), 0, 1);
4979:
4980: for i = 1:numel(cfg.closed_loop_files)
```

```
4981:     file = cfg.closed_loop_files{i};
4982:     if exist(file, 'file') ~= 2
4983:         warning('run_realtime_benchmark:MissingClosedLoopFile', ...
4984:             'Closed-loop output not found: %s', file);
4985:         continue;
4986:     end
4987:     S = load(file, 'logsout', 'SimulationMetadata');
4988:     sts = local_get_signal(S.logsout, 'diag_solve_time_ms');
4989:     if ~isempty(sts)
4990:         vals = double(sts.Data(:));
4991:         t = double(sts.Time(:));
4992:         if numel(t) ~= numel(vals)
4993:             t = (0:numel(vals)-1).' * (Ts_ms / 1000);
4994:         end
4995:         mask = isfinite(vals) & t >= double(cfg.warmup_sec);
4996:         vals = vals(mask);
4997:         t = t(mask);
4998:         all_values = [all_values; vals(:)]; %#ok<AGROW>
4999:         all_sources = [all_sources; repmat(string(file), numel(vals), 1)]; %#ok<AGROW>
5000:         all_t = [all_t; t(:)]; %#ok<AGROW>
5001:     end
5002: 
5003:     if isfield(S, 'SimulationMetadata')
5004:         wall_rows(end+1) = local_extract_wall_row(file, S.SimulationMetadata, S.logsout,
Ts_ms); %#ok<AGROW>
5005:     end
5006: end
5007: 
5008: if ~isempty(all_values)
5009:     raw_table = table(all_sources, all_t, all_values, ...
5010:         'VariableNames', {'source_file','time_s','solve_time_ms'});
5011:     rows = repmat(local_stats_row("", [], Ts_ms), 0, 1);
5012:     rows(end+1) = local_stats_row("mpc_solve_time", all_values, Ts_ms); %#ok<AGROW>
5013:     summary_table = struct2table(rows);
5014:     summary_table.source_file = repmat("aggregate_closed_loop_files", height(summary_table), 1);
5015:     summary_table.warmup_sec = repmat(double(cfg.warmup_sec), height(summary_table), 1);
5016: end
5017: 
5018: if ~isempty(wall_rows)
5019:     wall_table = struct2table(wall_rows);
5020: end
5021: end
5022: 
5023: function T = local_read_onnx_summary(summary_file, Ts_ms)
5024: T = table();
5025: if exist(summary_file, 'file') ~= 2
5026:     warning('run_realtime_benchmark:MissingONNXSummary', ...
5027:         'ONNX summary not found: %s', summary_file);
5028:     return;
5029: end
5030: S = readtable(summary_file, 'TextType', 'string');
5031: if isempty(S)
5032:     return;
5033: end
5034: row = local_stats_row("onnxruntime_single_window", S.mean_ms(1), Ts_ms);
5035: row.n = S.n(1);
5036: row.mean_ms = S.mean_ms(1);
5037: row.std_ms = local_table_value(S, 'std_ms', 1, NaN);
5038: row.min_ms = local_table_value(S, 'min_ms', 1, NaN);
5039: row.p50_ms = S.p50_ms(1);
5040: row.p95_ms = S.p95_ms(1);
```

```
5041: row.p99_ms = local_table_value(S, 'p99_ms', 1, NaN);
5042: row.max_ms = S.max_ms(1);
5043: row.overrun_rate_vs_Ts = double(row.max_ms > Ts_ms);
5044: row.margin_p95_ms = Ts_ms - row.p95_ms;
5045: row.pass_p95 = row.p95_ms < Ts_ms;
5046: T = struct2table(row);
5047: T.source_file = string(summary_file);
5048: end
5049:
5050: function summary = local_build_summary(onnx_summary, replay_summary, mpc_summary, wall_summary,
Ts_ms)
5051: rows = repmat(local_summary_row(), 0, 1);
5052:
5053: if ~isempty(onnx_summary)
5054:     rows(end+1) = local_summary_from_stats(onnx_summary(1, :), ...
5055:         "onnxruntime_single_window", "python_onnxruntime", Ts_ms); %#ok<AGROW>
5056: end
5057: if ~isempty(replay_summary)
5058:     idx = find(strcmp(string(replay_summary.metric), "matlab_replay_update_predict_steady"), 1);
5059:     if ~isempty(idx)
5060:         rows(end+1) = local_summary_from_stats(replay_summary(idx, :), ...
5061:             "matlab_replay_update_predict_steady", "matlab_replay", Ts_ms); %#ok<AGROW>
5062:     end
5063:     idx = find(strcmp(string(replay_summary.metric), "matlab_replay_update_predict_all"), 1);
5064:     if ~isempty(idx)
5065:         rows(end+1) = local_summary_from_stats(replay_summary(idx, :), ...
5066:             "matlab_replay_update_predict_all", "matlab_replay", Ts_ms); %#ok<AGROW>
5067:     end
5068:     idx = find(strcmp(string(replay_summary.metric), "matlab_replay_update_all"), 1);
5069:     if ~isempty(idx)
5070:         rows(end+1) = local_summary_from_stats(replay_summary(idx, :), ...
5071:             "matlab_replay_update_all", "matlab_replay", Ts_ms); %#ok<AGROW>
5072:     end
5073: end
5074: if ~isempty(mpc_summary)
5075:     rows(end+1) = local_summary_from_stats(mpc_summary(1, :), ...
5076:         "mpc_solve_time", "closed_loop_logsout", Ts_ms); %#ok<AGROW>
5077: end
5078:
5079: if ~isempty(onnx_summary) && ~isempty(mpc_summary)
5080:     rows(end+1) = local_cycle_row("cycle_onnxruntime_plus_mpc", ...
5081:         onnx_summary(1, :), mpc_summary(1, :), "p95 sum of ONNXRuntime and MPC",
Ts_ms); %#ok<AGROW>
5082: end
5083: if ~isempty(replay_summary) && ~isempty(mpc_summary)
5084:     idx = find(strcmp(string(replay_summary.metric), "matlab_replay_update_predict_steady"), 1);
5085:     if ~isempty(idx)
5086:         rows(end+1) = local_cycle_row("cycle_matlab_replay_plus_mpc", ...
5087:             replay_summary(idx, :), mpc_summary(1, :), "p95 sum of MATLAB replay and MPC",
Ts_ms); %#ok<AGROW>
5088:     end
5089: end
5090:
5091: if ~isempty(wall_summary)
5092:     row = local_summary_row();
5093:     row.metric = "simulink_wall_per_step";
5094:     row.source = "SimulationMetadata.TimingInfo";
5095:     row.n = height(wall_summary);
5096:     vals = wall_summary.wall_ms_per_step;
5097:     row.mean_ms = mean(vals, 'omitnan');
5098:     row.p50_ms = median(vals, 'omitnan');
5099:     row.p95_ms = max(vals);
5100:     row.max_ms = max(vals);
```

```
    row.margin_p95_ms = Ts_ms - row.p95_ms;
    row.pass_p95 = row.p95_ms < Ts_ms;
    row.note = "desktop simulation wall time, not embedded controller compute time";
    rows(end+1) = row; %#ok<AGROW>
end

summary = struct2table(rows);
end

function row = local_stats_row(metric, values, Ts_ms)
values = double(values(:));
values = values(isfinite(values));
row = struct();
row.metric = string(metric);
row.n = numel(values);
row.mean_ms = NaN;
row.std_ms = NaN;
row.min_ms = NaN;
row.p50_ms = NaN;
row.p95_ms = NaN;
row.p99_ms = NaN;
row.max_ms = NaN;
row.overrun_rate_vs_Ts = NaN;
row.margin_p95_ms = NaN;
row.pass_p95 = false;
if isempty(values)
    return;
end
row.mean_ms = mean(values);
row.std_ms = std(values, 0);
row.min_ms = min(values);
row.p50_ms = prctile(values, 50);
row.p95_ms = prctile(values, 95);
row.p99_ms = prctile(values, 99);
row.max_ms = max(values);
row.overrun_rate_vs_Ts = mean(values > Ts_ms);
row.margin_p95_ms = Ts_ms - row.p95_ms;
row.pass_p95 = row.p95_ms < Ts_ms;
end

function row = local_summary_row()
row = struct();
row.metric = "";
row.source = "";
row.n = NaN;
row.mean_ms = NaN;
row.p50_ms = NaN;
row.p95_ms = NaN;
row.max_ms = NaN;
row.overrun_rate_vs_Ts = NaN;
row.margin_p95_ms = NaN;
row.pass_p95 = false;
row.note = "";
end

function row = local_summary_from_stats(T, metric, source, Ts_ms)
row = local_summary_row();
row.metric = string(metric);
row.source = string(source);
row.n = T.n(1);
```

```
5161: row.mean_ms = T.mean_ms(1);
5162: row.p50_ms = T.p50_ms(1);
5163: row.p95_ms = T.p95_ms(1);
5164: row.max_ms = T.max_ms(1);
5165: row.overrun_rate_vs_Ts = T.overrun_rate_vs_Ts(1);
5166: row.margin_p95_ms = Ts_ms - row.p95_ms;
5167: row.pass_p95 = row.p95_ms < Ts_ms;
5168: end
5169:
5170: function row = local_cycle_row(metric, A, B, note, Ts_ms)
5171: row = local_summary_row();
5172: row.metric = string(metric);
5173: row.source = "computed_sum";
5174: row.n = min(A.n(1), B.n(1));
5175: row.mean_ms = A.mean_ms(1) + B.mean_ms(1);
5176: row.p50_ms = A.p50_ms(1) + B.p50_ms(1);
5177: row.p95_ms = A.p95_ms(1) + B.p95_ms(1);
5178: row.max_ms = A.max_ms(1) + B.max_ms(1);
5179: row.overrun_rate_vs_Ts = NaN;
5180: row.margin_p95_ms = Ts_ms - row.p95_ms;
5181: row.pass_p95 = row.p95_ms < Ts_ms;
5182: row.note = string(note);
5183: end
5184:
5185: function row = local_wall_row()
5186: row = struct();
5187: row.source_file = "";
5188: row.num_steps = NaN;
5189: row.execution_wall_s = NaN;
5190: row.total_wall_s = NaN;
5191: row.wall_ms_per_step = NaN;
5192: row.execution_to_model_time_ratio = NaN;
5193: end
5194:
5195: function row = local_extract_wall_row(file, meta, logsout, Ts_ms)
5196: row = local_wall_row();
5197: row.source_file = string(file);
5198: row.num_steps = local_num_steps(logsout);
5199: if isprop(meta, 'TimingInfo') || isfield(meta, 'TimingInfo')
5200:     ti = meta.TimingInfo;
5201:     row.execution_wall_s = local_field_or_default(ti, 'ExecutionElapsedWallTime', NaN);
5202:     row.total_wall_s = local_field_or_default(ti, 'TotalElapsedWallTime', NaN);
5203: end
5204: if isfinite(row.execution_wall_s) && row.num_steps > 0
5205:     row.wall_ms_per_step = 1000 * row.execution_wall_s / row.num_steps;
5206:     model_time_s = row.num_steps * Ts_ms / 1000;
5207:     row.execution_to_model_time_ratio = row.execution_wall_s / model_time_s;
5208: end
5209: end
5210:
5211: function n = local_num_steps(logsout)
5212: n = NaN;
5213: for i = 1:logsout.numElements
5214:     e = logsout.get(i);
5215:     if isa(e.Values, 'timeseries')
5216:         n = numel(e.Values.Time);
5217:         return;
5218:     end
5219: end
5220: end
```

```
5221:
5222: function ts = local_get_signal(logsout, name)
5223: ts = [];
5224: if ~isa(logsout, 'Simulink.SimulationData.Dataset')
5225:     return;
5226: end
5227: for i = 1:logsout.numElements
5228:     e = logsout.get(i);
5229:     if strcmp(e.Name, name)
5230:         ts = e.Values;
5231:         return;
5232:     end
5233: end
5234: end
5235:
5236: function Y = local_timeseries_rows(data, n_time)
5237: D = squeeze(data);
5238: sz = size(D);
5239: if isvector(D)
5240:     Y = double(D(:));
5241:     return;
5242: end
5243: if sz(1) == n_time
5244:     Y = double(reshape(D, n_time, []));
5245: elseif sz(end) == n_time
5246:     Y = double(reshape(D, [], n_time).');
5247: elseif numel(sz) >= 2 && sz(2) == n_time
5248:     Y = double(reshape(permute(D, [2 1 3:ndims(D)]), n_time, []));
5249: else
5250:     error('run_realtime_benchmark:BadTimeseriesShape', ...
5251:         'Cannot align data shape %s with %d time samples.', mat2str(sz), n_time);
5252: end
5253: end
5254:
5255: function v = local_table_value(T, name, row, default_value)
5256: if ismember(name, T.Properties.VariableNames)
5257:     v = T.(name)(row);
5258: else
5259:     v = default_value;
5260: end
5261: end
5262:
5263: function v = local_field_or_default(s, name, default_value)
5264: if isstruct(s) && isfield(s, name)
5265:     v = s.(name);
5266: else
5267:     try
5268:         v = s.(name);
5269:     catch
5270:         v = default_value;
5271:     end
5272: end
5273: end
5274:
5275: function local_write_report(report_file, result)
5276: fid = fopen(report_file, 'w');
5277: if fid < 0
5278:     warning('run_realtime_benchmark:ReportFailed', 'Cannot write %s', report_file);
5279:     return;
5280: end
```

```
cleanup = onCleanup(@() fclose(fid));

fprintf(fid, '# Real-Time Benchmark\n\n');
fprintf(fid, '- timestamp: `%s`\n', result.timestamp);
fprintf(fid, '- Ts: `%.6g ms`\n', result.Ts_ms);
fprintf(fid, '- output root: `%s`\n\n', result.cfg.out_root);

fprintf(fid, '## Summary\n\n');
local_write_markdown_table(fid, result.summary);

fprintf(fid, '\n## Notes\n\n');
fprintf(fid, '- `onnxruntime_single_window` is pure batch=1 ONNXRuntime inference.\n');
fprintf(fid, '- `matlab_replay_update_predict_steady` replays closed-loop `diag.y_raw` through the MATLAB online wrapper after the sliding window is ready and after the configured first-predict warmup count.\n');
fprintf(fid, '- `mpc_solve_time` comes from `diag_solve_time_ms` in closed-loop logs after the configured warmup period.\n');
fprintf(fid, '- `simulink_wall_per_step` is desktop Simulink wall time and is not used as embedded controller compute time.\n');
end

function local_write_markdown_table(fid, T)
if isempty(T)
    fprintf(fid, '_No timing summary available._\n');
    return;
end
fprintf(fid, '| metric | mean ms | p50 ms | p95 ms | max ms | p95 margin ms | pass p95 |\n');
fprintf(fid, '|---|---:|---:|---:|---:|---:|---:|\n');
for i = 1:height(T)
    fprintf(fid, '| %s | %.6g | %.6g | %.6g | %.6g | %.6g | %d |\n', ...
        string(T.metric(i)), T.mean_ms(i), T.p50_ms(i), T.p95_ms(i), ...
        T.max_ms(i), T.margin_p95_ms(i), T.pass_p95(i));
end
end
```